

第 20 章：Comprehensions and Generations (推导式与生成)

原书：《Learning Python》（6th Edition），作者 Mark Lutz 本章地位：本章是一系列高级函数工具的收官之战，也是第 14 章（迭代与推导式）的“续集”。它把列表推导式推广到集合/字典推导式和生成器，系统讲解生成器函数（yield）、生成器表达式、惰性求值的内存优势，以及 map/filter/zip 等函数式工具与推导式的对比，最后用浅显篇幅预告 async 协程。第 21 章将用计时脚本验证本章各种工具的性能差异。

20.1 开篇引言 (Chapter Introduction)

英文原文

This chapter explores a set of advanced function-related tools and topics. Its main subjects are generator functions and their generator expression relatives—user-defined ways to produce results on demand the same way that many built-ins do. Because generators apply the iteration protocol and generator expressions reuse comprehension syntax, this chapter is also partl

y a follow-up to Chapter 14 (hence its title). We'll extend these topics to their completion here and demo with larger examples that tie ideas together. Finally, this chapter provides just enough of an intro to get you started with async coroutines—tools that build on generators, but assume knowledge of parallel programming, which is outside the scope of this book and the needs of most Python learners. You won't become an async master here, but you'll get a head start for further explorations. Iteration and generation in Python also encompasses user-defined classes, but we'll defer that final part of this story until Part VI, when we study operator overloading. The next chapter continues threads spun here by timing the relative performance of some of this chapter's tools as a larger case study. Before that, though, let's pick up the comprehensions and iterations story where it was last left, and extend it to include value generators.

中文翻译

本章探讨一组与函数相关的高级工具和主题。其主角是生成器函数 (generator functions) 及其近亲生成器表达式 (generator expressions) ——它们是用自定义的方式，像许多内置工具那样“按需” (on demand) 产出结果。由于生成器应用了迭代协议 (iteration protocol)、而生成器表达式复用了推导式 (comprehension) 的语法，本章在很大程度上也是第 14 章的后续 (这正是本章标题的由来)。我们将在这里把这些主题讲完，并用更大的、把各种思想

串联起来的示例来演示。最后，本章就异步协程（async coroutines）给出入门级的简要介绍——这类工具建立在生成器之上，但假设读者了解并行编程，而并行编程超出了本书的范畴，也超出了大多数 Python 学习者的需求。你不会在这里成为 async 大师，但会为进一步探索提供一个好的起点。Python 中的迭代（iteration）与生成（generation）同样涵盖用户自定义的类（classes），但我们要把这一部分的最终结局推迟到第六部分（Part VI）学习运算符重载（operator overloading）时再讲。下一章将继续本章抛出的线索——通过对本章某些工具的相对性能进行计时，构成一个更大的案例研究。在此之前，我们不妨从上次停下的地方接续推导式与迭代的故事，并把它扩展到“值生成器”（value generators）上。

深度理解

·核心概念：本章回答三个问题——推导式还能怎么写、生成器（generator）到底是什么、惰性求值（lazy evaluation）能带来哪些好处。作者把生成器定位为“函数式编程（functional programming）”大工具箱里的一员，与闭包、lambda、map、装饰器等并列。

·底层视角：三次强调“迭代协议”——生成器的本质就是实现 iter / next 协议的对象。Python 的 for 循环、list()、sum() 等所有迭代工具都认这套协议，所以生成器可以无缝放进任何迭代上下文。

·设计原因：Python 鼓励“按需产出”而非“一次产出全部”。文件按行读取、map/zip 边取边出，都是同一设计哲学的体现；用户自定义生成器只是把这种思想下放到自己的

代码里。

·实际问题：处理大数据流、无限序列、不想让调用方长时间等待结果时，惰性生成是唯一现实的选择。

·初学者误区：以为生成器是“很神奇的黑魔法”。其实它只是“实现了迭代协议的对象”，机制完全公开、可手动用 `next()` 驱动。

20.2 Comprehensions: The Final Act (推导式：最后一幕)

英文原文

As mentioned early in this book, Python supports the procedural, object-oriented, and function programming paradigms. Among these, Python has a host of tools that most observers would consider functional in nature, which we enumerated in the preceding chapter—closures, generators, lambdas, comprehensions, maps, decorators, function objects, and more. These tools allow us to apply and combine functions in powerful ways, and often offer state retention and coding solutions that are alternatives to classes and OOP. For instance, the prior chapter explored tools such as `map` and `filter`—key members of Python's early functional-programming toolset inspired by the Lisp language—that map operations over iterables and collect results. Because this is such a common task in coding,

Python eventually sprouted a new expression—the list comprehension—a device inspired by languages like Haskell, which is even more flexible than the original functional toolset. As we've seen, list comprehensions apply an arbitrary expression to items in an iterable, rather than applying a function. Accordingly, they can be more general tools. In later Pythons, the list comprehension was extended to other roles—sets, dictionaries, and even the value generator expressions we'll explore in this chapter. Hence, it's not just for lists anymore, and we'll simply call it comprehension when referring to all its forms collectively. We first met comprehensions in Chapter 4's preview, introduced them in Part II's coverage of sets, lists, and dictionaries, and studied them further in Chapter 14, in conjunction with looping statements. Because they're also related to functional programming tools like the map and filter calls and repurposed by generator expressions, though, let's resurrect the topic here for one last look.

中文翻译

正如本书早先提到的，Python 支持过程式（procedural）、面向对象（object-oriented）和函数式（function）三种编程范式。在这三者之中，Python 拥有一大堆多数观察者会认为是函数式（functional）性质的工具——上一章列举过：闭包（closures）、生成器（generators）、lambda、推导式（comprehensions）、map、装饰器（decor

ators)、函数对象(function objects)等等。这些工具让我们能以强大的方式应用和组合函数,并且常常提供状态保持(state retention)和编码方案,作为类(classes)和 OOP 的替代品。例如,上一章探讨了 map 和 filter——Python 早期函数式编程工具集的关键成员,灵感来自 Lisp 语言——它们在可迭代对象上映射操作并收集结果。因为这在编码中是如此常见的任务,Python 最终催生了一种新的表达式(expression)——列表推导式(list comprehension),这个机制受 Haskell 等语言的启发,甚至比原来的函数式工具集更加灵活。正如我们所看到的,列表推导式对可迭代对象中的每一项应用一个任意的表达式,而不是应用一个函数。相应地,它们可以是更通用的工具。在后续的 Python 版本里,列表推导式被扩展到其他角色——集合(sets)、字典(dictionaries),甚至本章将要探索的值生成器表达式。因此,它不再只是为列表服务的了,当我们需要统称它的一切形式时,我们就直接叫它推导式(comprehension)。我们最初在第 4 章的预览中见过推导式,在第二部分(Part II)讲集合、列表、字典时正式介绍过它,并在第 14 章结合循环语句进一步研究过。不过,由于它和 map、filter 之类的函数式编程工具相关,又被生成器表达式改造复用,让我们在此把这个话题复活,作最后一次审视。

深度理解

·核心概念:推导式是一种"以表达式为中心的循环"。普通 for 循环是语句(statement),而推导式是表达式(expression)——可以直接嵌在 lambda、函数调用、字面量等任何表达式允许出现的地方。这是它相对循环语句的

根本优势。

·底层实现：列表推导式在解释器内部以"专门字节码" (LISTAPPEND) 执行，几乎等价于一个隐含的 for 循环加 append，但由 C 层驱动，比在 PVM 里逐条执行 Python 循环字节码更快（本章稍后明确说明）。

·设计原因：推导式受 Haskell 等纯函数式语言影响。把"映射 (map, 对每项做变换) + 过滤 (filter, 用 if 挑选)"这两个最常用的操作合并进一个表达式，这正对应了上文"map 一个函数、推导式映射一个表达式"的对比。

·实际问题：从旧代码迁移角度来看，Python 社区长期以来建议"能用推导式的地方优先用推导式"，但不建议为了炫技而嵌套过深（本章下文专门写了一小节警告）。

·初学者误区：以为推导式只能处理列表、只能是单层循环。实际上它可迭代任意可迭代对象，支持多层 for 嵌套和 if 过滤，并且有 list / set / dict / generator 四种外形。

20.3 List Comprehensions Review (列表推导式回顾)

英文原文

Here's a brief example to refresh a few neurons. As we learned in Chapter 7, Python's built-in ord function returns the integer code point of a single character. If we wish to collect code points of all characters in an entire string, a straightforward approach is to use a si

mple for loop and append the results to a list. In a REPL:

```
> >> res = [] >> for x in 'text':  
  
    res.append(ord(x))          # Manual results collection  
> >> res  
  
[116, 101, 120, 116]
```

Now that we know about map, though, we can achieve similar results with a single function call without having to manage list construction in the code:

```
> >> res = list(map(ord, 'text')) # Apply function to iterable ...  
  
[116, 101, 120, 116]
```

However, we can get the same results from a list comprehension expression—while map maps a function over an iterable, list comprehensions map an expression:

```
> >> res = [ord(x) for x in 'text'] # Apply expression to itera...  
  
[116, 101, 120, 116]
```

List comprehensions collect the results of applying an expression to an iterable of values, and return them in a new list. Syntactically, list comprehensions are enclosed in square brackets—to remind you that they construct lists. In their basic form, within the brackets you code an expression that names a variable, followed by what looks like a for loop header that names the same variable. You get back the expression's results

for each iteration of the implied loop.

The effect of the preceding example is similar to that of the manual for loop and the map call. List comprehensions become more convenient, though, when we wish to apply an arbitrary expression instead of a function:

```
> >> [x ** 2 for x in range(10)] # Squares of numbers 0 to 9
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

To do similar work with a map call, we may need to invent a little function to implement the square operation. Because we won't need this function elsewhere, we'd typically (but not necessarily) code it inline, with a lambda:

```
> >> list(map((lambda x: x ** 2), range(10)))
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

This does the same job, and it's only a few keystrokes longer than the equivalent list comprehension. It's also only marginally more complex (at least, once you understand the lambda). For more advanced roles, though, list comprehensions will often require considerably less typing.

中文翻译

这里有一个小例子来唤醒几根神经。正如我们在第 7 章学到的, Python 内置的 `ord` 函数返回单个字符的整数码点 (code point)。如果我们想收集整个字符串中所有字符的

码点，一个直截了当的做法是使用简单的 for 循环并把结果 append 到一个列表里。在 REPL（交互式环境）中：

```
>>> res = []
>>> for x in 'text':
    res.append(ord(x))      #
>>> res
[116, 101, 120, 116]
```

不过，现在我们知道 map，就可以用一次函数调用来达到类似的效果，而无需在代码中手动管理列表的构建：

```
>>> res = list(map(ord, 'text')) #
>>> res
[116, 101, 120, 116]
```

然而，我们也可以从一个列表推导式表达式中得到相同的结果——map 是把一个函数映射到可迭代对象上，而列表推导式是把一个表达式映射过去：

```
>>> res = [ord(x) for x in 'text'] #
>>> res
[116, 101, 120, 116]
```

列表推导式收集"把表达式应用到一个可迭代对象的值集合上"的结果，并把它们放进一个新的列表返回。在语法上，列表推导式用方括号括起来——提醒你它们构造的是列表。在其基本形式中，方括号内你写一个命名某个变量的表达式，后面跟着一个看起来像 for 循环头、且命名同一变量的部分。你会为隐含循环的每一次迭代拿到这个表达式的结果。前面例子的效果和手写 for 循环、map 调用是相似的。不过，当我们希望应用的是一段任意表达式、而不是一个函数时，列表推导式就变得方便多了：

```
>>> [x ** 2 for x in range(10)] # 0 9  
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

要用 map 调用做类似的工作，我们可能得发明一个小小的函数来实现平方运算。由于别处用不到这个函数，我们通常会（但并非必须）用 lambda 把它内联（inline）写出来：

```
>>> list(map((lambda x: x ** 2), range(10)))  
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

这完成了同样的工作，只比等价的列表推导式多敲几个键。它也只是略复杂一点（至少，在你理解 lambda 之后）。不过，对于更高级的角色，列表推导式通常能省下可观的键入量。

代码分析

三个方案（手动循环 / map / 列表推导式）殊途同归，但分属不同层次：手动循环是“逐步建造”，map 是“函数映射 + 惰性生成”，列表推导式是“表达式映射 + 立即建表”。

解释器处理 `[ord(x) for x in 'text']` 时，会为“字符串迭代”和“字符视频”直接使用专用字节码：先用 GETITER 取得字符串迭代器，循环中每轮用 FORITER / YIELDVALUE 派生的流程取下一个字符、计算 `ord(x)`，再用 LISTAPPEND 追加进正在构建的列表——append（绑定方法查找）这一步在推导式里被内联优化掉了，这正是它通常比手写循环快的底层原因。

设计思想：作者刻意让三者的输出完全一致，就是为了告诉读者——“语法不同、基建相同”，真正拉开差距的是你能否写出“任意表达式”（而 map 只能应用函数）。

英文原文

For instance, as we learned in Chapter 14, you can code an if clause after the comprehension's for to add selection logic. List comprehensions with if clauses can be thought of as analogous to the filter built-in in the preceding chapter—they skip an iterable's items for which the if clause is not true. As a demo, following are both schemes, along with the equivalent for, picking up even numbers from 0 to 9 ($x \% 2$ is zero for evens only):

```
>>> [x for x in range(10) if x % 2 == 0]
[0, 2, 4, 6, 8]
>>> list(filter((lambda x: x % 2 == 0), range(10)))
[0, 2, 4, 6, 8]
>>> res = []
>>> for x in range(10):
        if x % 2 == 0:
            res.append(x)
>>> res
[0, 2, 4, 6, 8]
```

Though it's penalized by having to code a function to be applied, the filter call here is still not much longer than the list comprehension either. However, the list comprehension can combine an if clause and an arbitrary expression, to give it the effect of a filter and a map—in a single expression:

```
>>> [x ** 2 for x in range(10) if x % 2 == 0]
[0, 4, 16, 36, 64]
```

This time, we collect the squares of the even numbers from 0 through 9: the for loop skips numbers for which the attached if clause on the right is false, and the expression on the left computes the squares. The equivalent map call would require a lot more work on our part—we would have to combine filter selections with map iteration, making for a noticeably more complex expression:

```
>>> list( map((lambda x: x**2), filter((lambda x: x % 2 == 0), ...
[0, 4, 16, 36, 64]
```

中文翻译

例如，正如我们在第 14 章学到的，你可以在推导式的 for 之后写一个 if 子句，来加入选择（selection）逻辑。带 if 子句的列表推导式可以被认为等同于上一章的 filter 内置函数——它们会跳过那些 if 子句不为真的可迭代项。作为演示，下面同时给出这两种方案，以及等价的 for 循环，都是从 0 到 9 中挑出偶数（ $x \% 2$ 只在偶数时为零）：

```
>>> [x for x in range(10) if x % 2 == 0]
[0, 2, 4, 6, 8]
>>> list(filter((lambda x: x % 2 == 0), range(10)))
[0, 2, 4, 6, 8]
>>> res = []
>>> for x in range(10):
    if x % 2 == 0:
        res.append(x)
>>> res
[0, 2, 4, 6, 8]
```

尽管这里的 filter 调用因为必须编码一个待应用的函数而有些吃亏，但它仍然不比列表推导式长多少。然而，列表推导

式可以把 if 子句和任意表达式结合起来，从而在一个表达式里同时起到 filter 和 map 的效果：

```
>>> [x ** 2 for x in range(10) if x % 2 == 0]
[0, 4, 16, 36, 64]
```

这一次，我们收集了 0 到 9 之间偶数的平方：for 循环跳过那些右侧附加的 if 子句为假的数字，而左侧的表达式计算平方。等价的 map 调用就需要我们做多得多的活了——我们必须把 filter 的选择和 map 的迭代组合起来，构成一个显著更复杂的表达式：

```
>>> list( map((lambda x: x**2), filter((lambda x: x % 2 == 0), ...
[0, 4, 16, 36, 64]
```

代码分析

· `[x 2 for x in range(10) if x % 2 == 0]` 的读取顺序是“从右向左”：先看 for 的迭代范围，再看 if 是否过滤，最后才把通过过滤的 x 交给左边的表达式。初学者最容易读反方向。

· 解释器处理这条推导式时，循环体内首先执行 `x % 2 == 0` 这个测试，若为假就直接跳过本次迭代（不进入表达式计算），为真才计算 `x 2` 并 `LISTAPPEND`。过滤在 C 层完成，无 lambda 调用开销。

· 设计思想：map + filter 的组合需要嵌套两个函数调用、写两次 lambda，可读性急剧下降；而推导式把“过滤 + 变换”合体为一行。这正是函数式工具在简单场景下被推导式取代的原因——推导式是“表达式级别的函数式编程”。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.4 Formal Comprehension Syntax (推导式的正式语法)

英文原文

In fact, list comprehensions are more general still. In their simplest form, you must always code an accumulation expression and a single for clause:

```
[ expression for target in iterable ]
```

Though all other parts are optional, they allow richer iterations to be expressed—you can code any number of nested for loops in a list comprehension, and each may have an optional associated if test to act as a filter. The general structure of list comprehensions loo

ks like this:

```
[ expression for target1 in iterable1 if condition1
  for target2 in iterable2 if condition2 ...
  for targetN in iterableN if conditionN ]
```

This exact same syntax is inherited by set and dictionary comprehensions as well as the generator expressions coming up, though these use different enclosing characters—curly braces for the first two and often optional parentheses for the latter—and the dictionary comprehension begins with two expressions separated by a colon (for key and value).

We experimented with the if filter clause in the previous section. When for clauses are nested within a list comprehension, they work like equivalent nested for loop statements. For example:

```
> >> res = [x + y for x in [0, 1, 2] for y in [100, 200, 300]] ...
[100, 200, 300, 101, 201, 301, 102, 202, 302]
```

This has the same effect as this substantially more verbose equivalent:

```
> >> res = []
>>> for x in [0, 1, 2]:
    for y in [100, 200, 300]:
        res.append(x + y)
>>> res
[100, 200, 300, 101, 201, 301, 102, 202, 302]
```

Although list comprehensions construct list results, remember that they can iterate over any iterable object.

ct. Here's a similar bit of code that traverses strings instead of lists of numbers, and so collects concatenation results:

```
> >> [x + y for x in 'orm' for y in 'ORM'] # 3 * 3 results

['oO', 'oR', 'oM', 'rO', 'rR', 'rM', 'mO', 'mR', 'mM']
```

Each for clause can have an associated if filter, no matter how deeply the loops are nested—though use cases for the following sort of code, apart perhaps from the multidimensional arrays ahead, start to become more and more difficult to imagine at this level:

```
> >> [x + y for x in 'orm' if x in 'ro' for y in 'ORM' if y > 'r']

['oO', 'oR', 'rO', 'rR']
> >> [x + y + z for x in 'hack' if x > 'c'

        for y in 'CODE' if y in 'OD'
        for z in '123' if z > '1']
['hO2', 'hO3', 'hD2', 'hD3', 'kO2', 'kO3', 'kD2', 'kD3']
```

Finally, here is a similar list comprehension that illustrates the effect of attached if selections on nested for clauses applied to numeric objects rather than strings:

```
> >> [(x, y) for x in range(5) if x % 2 == 0

        for y in range(5) if y % 2 == 1]
[(0, 1), (0, 3), (2, 1), (2, 3), (4, 1), (4, 3)]
```

This expression combines even numbers from 0 through 4 with odd numbers from 0 through 4. The if clauses filter out items in each iteration. Here is the equi

valent statement-based code:

```
> >> res = [] >> for x in range(5):  
  
    if x % 2 == 0:  
        for y in range(5):  
            if y % 2 == 1:  
                res.append((x, y))  
  
> >> res  
  
[(0, 1), (0, 3), (2, 1), (2, 3), (4, 1), (4, 3)]
```

Recall that if you're confused about what a complex list comprehension does, you can always nest the list comprehension's for and if clauses inside each other like this—indenting each clause successively further to the right—to derive the equivalent statements. The result is longer, but perhaps clearer in intent to some human readers on first glance, especially those more familiar with basic statements.

The map and filter equivalent of this last example would be wildly complex and deeply nested, so this book will leave its coding as an exercise for Zen masters, ex-Lisp programmers, and those with far too much free time.

中文翻译

事实上，列表推导式还要更通用。在其最简单的形式中，你必须始终编写一个累加表达式（accumulation expression）和单个 for 子句：

```
[ expression for target in iterable ]
```

尽管其他所有部分都是可选的，它们却允许表达更丰富的迭代——你可以在列表推导式里编写任意数量的嵌套 for 循环，每个循环都可以带一个可选的 if 测试作为过滤器。列表推导式的一般结构如下：

```
[ expression for target1 in iterable1 if condition1
  for target2 in iterable2 if condition2 ...
  for targetN in iterableN if conditionN ]
```

这一完全相同的语法也被 set（集合）和 dictionary（字典）推导式以及即将出场的 generator expressions（生成器表达式）继承，不过它们使用不同的包围字符——前两者用花括号，后者用经常可省略的圆括号——并且字典推导式以两个用冒号分隔的表达式开头（分别对应键和值）。我们在上一节试验过 if 过滤子句。当 for 子句在列表推导式中嵌套时，它们的行为与等价的嵌套 for 循环语句一样。例如：

```
>>> res = [x + y for x in [0, 1, 2] for y in [100, 200, 300]]
>>> res
[100, 200, 300, 101, 201, 301, 102, 202, 302]
```

这和下面这个冗长得多的等价形式效果相同：

```
>>> res = []
>>> for x in [0, 1, 2]:
    for y in [100, 200, 300]:
        res.append(x + y)
>>> res
[100, 200, 300, 101, 201, 301, 102, 202, 302]
```

尽管列表推导式构造的是列表结果，但要记住它们可以迭代任何可迭代对象（iterable object）。下面是一段类似的代码，它遍历的是字符串而非数字列表，因此收集的是拼接（concatenation）的结果：

```
>>> [x + y for x in 'orm' for y in 'ORM'] # 3 * 3
['oO', 'oR', 'oM', 'rO', 'rR', 'rM', 'mO', 'mR', 'mM']
```

每个 for 子句都可以带一个关联的 if 过滤器，无论循环嵌套得多深——不过下面这类代码的用例（或许除了后面的多维数组），在这个层级上开始变得越来越难以想象了：

```
>>> [x + y for x in 'orm' if x in 'ro' for y in 'ORM' if y > 'M...
['oO', 'oR', 'rO', 'rR']
>>> [x + y + z for x in 'hack' if x > 'c'
      for y in 'CODE' if y in 'OD'
      for z in '123' if z > '1']
['hO2', 'hO3', 'hD2', 'hD3', 'kO2', 'kO3', 'kD2', 'kD3']
```

最后，这里有一个类似的列表推导式，演示附加的 if 选择对嵌套 for 子句的影响——这次应用于数字对象而非字符串：

```
>>> [(x, y) for x in range(5) if x % 2 == 0
      for y in range(5) if y % 2 == 1]
[(0, 1), (0, 3), (2, 1), (2, 3), (4, 1), (4, 3)]
```

这个表达式把 0 到 4 的偶数与 0 到 4 的奇数两两组合。if 子句在每一轮迭代中过滤掉不符合的项。下面是等价的基于语句的代码：

```
>>> res = []
>>> for x in range(5):
    if x % 2 == 0:
        for y in range(5):
            if y % 2 == 1:
                res.append((x, y))
>>> res
[(0, 1), (0, 3), (2, 1), (2, 3), (4, 1), (4, 3)]
```

记住：如果你对一个复杂的列表推导式感到困惑，你总是可以像这样把它的 for 和 if 子句逐层嵌套展开——每一层子

句依次向右缩进——从而推导出等价的语句形式。结果虽然更长，但或许一眼看去对某些读者（尤其是更熟悉基本语句的人）来说意图更清晰。最后一个例子的 `map` 和 `filter` 等价形式将极度复杂且层层嵌套，所以本书把它的编码留给禅宗大师、前 Lisp 程序员，以及那些空闲时间多到没处花的人作为练习。

深度理解

·核心概念：推导式是“声明式”的迭代——你描述“要什么”，而不是“怎么一步步做”。嵌套 `for` 的顺序就是循环嵌套的顺序：外层在前、内层在后，展开成语句时对应逐层缩进。

·底层实现：推导式嵌套循环展开后相当于多重循环，`if` 过滤则相当于循环体里的 `if` 守卫。解释器把它编译成一段隐含循环的专用字节码；所有生成值都通过 `LISTAPPEND` 进同一个列表。

·设计原因：把“循环 + 条件 + 变换”压缩为单一表达式，是为了让“迭代后收集结果”这种最高频的编码模式保持一行、一目了然。三个例子（字符串、数字、组合过滤）证明同一语法通吃一切可迭代对象。

·实际问题：矩阵转置、笛卡尔积（所有组合）、按条件筛选并变换——这类代码用推导式最自然。笛卡尔积就是“两个 `for` 嵌套”，如同后文矩阵示例所示。

·初学者误区：①把两个 `for` 的先后顺序搞反（外内颠倒会得到不同的结果顺序）；②误以为多个 `for` 是“并行的”——其实是“嵌套的”，右侧先写完整个内层循环才轮到外层

下一项；③忘记 if 只作用于紧跟其后的那个 for。

20.5 Example: List Comprehensions and Matrixes (示例：列表推导式与矩阵)

英文原文

Not all list comprehensions are so artificial, of course. Let's look at one more application to stretch our comprehension muscles. As we saw in Chapters 4 and 8, one basic way to code matrixes (a.k.a. multidimensional arrays) in Python is with nested list structures. The following, for example, defines two 3×3 matrixes as lists of nested lists:

```
> >> M = [[1, 2, 3],  
           [4, 5, 6],  
           [7, 8, 9]]  
> >> N = [[2, 2, 2],  
           [3, 3, 3],  
           [4, 4, 4]]
```

Given this structure, we can always index rows, and columns within rows, using normal index operations:

```
> >> M[1] # Row 2  
[4, 5, 6]  
> >> M[1][2] # Row 2, item 3
```

List comprehensions are powerful tools for processing such structures because they automatically scan rows and columns. For instance, although this code stores the matrix by rows, to collect the second column we can simply iterate across the rows and pull out the desired column, or iterate through positions in the rows and index as we go:

```
> >> [row[1] for row in M] # Column 2  
  
[2, 5, 8]  
> >> [M[row][1] for row in (0, 1, 2)] # Using offsets  
  
[2, 5, 8]
```

Given positions, we can also easily perform tasks such as pulling out a diagonal. The first of the following expressions uses `range` to generate the list of offsets and then indexes with the row and column the same, picking out `M[0][0]`, then `M[1][1]`, and so on. The second scales the column index to fetch `M[0][2]`, `M[1][1]`, etc. (we assume the matrix has the same number of rows and columns):

```
> >> [M[i][i] for i in range(len(M))] # Diagonals  
  
[1, 5, 9]  
> >> [M[i][len(M)-1-i] for i in range(len(M))]  
  
[3, 5, 7]
```

Changing such a matrix in place requires assignment to offsets (use `range` twice if shapes differ):

```
> >> L = [[1, 2, 3], [4, 5, 6]] >> for i in range(len(L)):
    for j in range(len(L[i])):
        L[i][j] += 10      # Update in place
> >> L

[[11, 12, 13], [14, 15, 16]]
```

We can't really do the same with list comprehensions, as they make new lists, but we could always assign their results to the original name for a similar effect. For example, we can apply an operation to every item in a matrix, producing results in either a simple vector or a matrix of the same shape:

```
> >> [col + 10 for row in M for col in row] # Assign to M to re...

[11, 12, 13, 14, 15, 16, 17, 18, 19]
> >> [[col + 10 for col in row] for row in M]

[[11, 12, 13], [14, 15, 16], [17, 18, 19]]
```

To understand these, translate to their simple statement form equivalents that follow—indent parts that are further to the right in the expression (as in the first loop in the following), and make a new list when comprehensions are nested on the left (like the second loop in the following).

As its statement equivalent makes clearer, the second expression in the preceding works because the row iteration is an outer loop: for each row, it runs the nested column iteration to build up one row of the result matrix:

```

> >> res = [] >> for row in M: # Statement equivalents

    for col in row:           # Indent parts further right
        res.append(col + 10)

> >> res

[11, 12, 13, 14, 15, 16, 17, 18, 19]

> >> res = [] >> for row in M:

    tmp = []                  # Left-nesting starts new list
    for col in row:
        tmp.append(col + 10)
    res.append(tmp)

> >> res

[[11, 12, 13], [14, 15, 16], [17, 18, 19]]

```

Finally, with a bit of creativity, we can also use list comprehensions to combine values of multiple matrixes . The following first builds a flat list that contains the result of multiplying the matrixes pairwise, and then builds a nested list structure having the same values by nesting list comprehensions again:

```

> >> M

[[1, 2, 3], [4, 5, 6], [7, 8, 9]]

> >> N

[[2, 2, 2], [3, 3, 3], [4, 4, 4]]

> >> [M[row][col] * N[row][col] for row in range(3) for col in ran...

[2, 4, 6, 12, 15, 18, 28, 32, 36]

> >> [[M[row][col] * N[row][col] for col in range(3)] for row in r...

[[2, 4, 6], [12, 15, 18], [28, 32, 36]]

```

This last expression works because the row iteration is an outer loop again; it's equivalent to this statement-based code:

```
res = []
for row in range(3):
    tmp = []
    for col in range(3):
        tmp.append(M[row][col] * N[row][col])
    res.append(tmp)
```

And for more fun, we can use `zip` to pair items to be multiplied—the following comprehension and loop statement both produce the same list-of-lists pairwise multiplication result as the last preceding example (and because `zip` is a generator of values, this isn't as inefficient as it may seem):

```
[[col1 * col2 for (col1, col2) in zip(row1, row2)] for (row1, r...
```

```
res = []
for (row1, row2) in zip(M, N):
    tmp = []
    for (col1, col2) in zip(row1, row2):
        tmp.append(col1 * col2)
    res.append(tmp)
```

Compared to their statement equivalents, the list comprehension versions here require only one line of code, might run substantially faster for large matrixes, and just might make your head explode. Which brings us to the next section.

中文翻译

当然，并非所有的列表推导式都这么“人工”。让我们再看一个应用，来拉伸一下我们的推导式肌肉。正如我们在第 4 章和第 8 章看到的，在 Python 中编写矩阵（matrix，也叫多维数组）的一种基本方式是用嵌套的列表结构。例如，下面用嵌套列表的列表定义了两个 3×3 矩阵：

```
>>> M = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]
>>> N = [[2, 2, 2],
          [3, 3, 3],
          [4, 4, 4]]
```

有了这种结构，我们总可以用普通的索引运算（index operations）来索引行、以及行内的列：

```
>>> M[1]           # 2
[4, 5, 6]
>>> M[1][2]        # 2 3
6
```

列表推导式是处理这种结构的强大工具，因为它们会自动扫描行和列。例如，尽管这段代码是按行存储矩阵的，要收集第二列，我们只需遍历各行、抽出想要的列，或者遍历行内的位置并边走边索引：

```
>>> [row[1] for row in M]    # 2
[2, 5, 8]
>>> [M[row][1] for row in (0, 1, 2)] #
[2, 5, 8]
```

有了位置信息，我们也可以轻松完成诸如抽出对角线之类的任务。下面第一个表达式用 range 生成偏移量列表，然后用相同的行列索引去索引，依次取出 M[0][0]、M[1][1].....

；第二个把列索引缩放，去取 $M[0][2]$ 、 $M[1][1]$ 等（我们假设矩阵的行列数相同）：

```
>>> [M[i][i] for i in range(len(M))]      #  
[1, 5, 9]  
>>> [M[i][len(M)-1-i] for i in range(len(M))]  
[3, 5, 7]
```

要就地 (in place) 修改这样的矩阵，需要向偏移量赋值（如果形状不同，range 要用两次）：

```
>>> L = [[1, 2, 3], [4, 5, 6]]  
>>> for i in range(len(L)):  
    for j in range(len(L[i])):  
        L[i][j] += 10      #  
>>> L  
[[11, 12, 13], [14, 15, 16]]
```

用列表推导式其实做不到同样的事，因为它们制造的是新列表，但我们总可以把它们的结果赋值回原名字，以达到类似的效果。例如，我们可以对矩阵中的每一项应用一个操作，结果要么是一个简单的向量，要么是一个形状相同的矩阵：

```
>>> [col + 10 for row in M for col in row] # M  
[11, 12, 13, 14, 15, 16, 17, 18, 19]  
>>> [[col + 10 for col in row] for row in M]  
[[11, 12, 13], [14, 15, 16], [17, 18, 19]]
```

要理解它们，把它们翻译成下面这些简单的语句等价形式即可——把表达式中靠右的部分缩进（如下面第一个循环），当推导式在左边嵌套时新建一个列表（如下面第二个循环）。从语句等价形式可以看得更清楚：上面第二个表达式之所以成立，是因为行的迭代是外层循环——对每一行，它运行嵌套的列迭代来构建结果矩阵的一行：

```

>>> res = []
>>> for row in M:
    for col in row:
        res.append(col + 10)
>>> res
[11, 12, 13, 14, 15, 16, 17, 18, 19]
>>> res = []
>>> for row in M:
    tmp = []
    for col in row:
        tmp.append(col + 10)
    res.append(tmp)
>>> res
[[11, 12, 13], [14, 15, 16], [17, 18, 19]]

```

最后，稍微发挥一点创造力，我们还可以用列表推导式来组合多个矩阵的值。下面先构建一个扁平列表，包含矩阵逐项相乘的结果；然后再通过嵌套列表推导式，构建一个具有相同值的嵌套列表结构：

```

>>> M
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>>> N
[[2, 2, 2], [3, 3, 3], [4, 4, 4]]
>>> [M[row][col] * N[row][col] for row in range(3) for col in range(3)]
[2, 4, 6, 12, 15, 18, 28, 32, 36]
>>> [[M[row][col] * N[row][col] for col in range(3)] for row in range(3)]
[[2, 4, 6], [12, 15, 18], [28, 32, 36]]

```

最后一个表达式之所以成立，是因为行迭代再次成为外层循环；它等价于下面这段基于语句的代码：

```

res = []
for row in range(3):
    tmp = []
    for col in range(3):
        tmp.append(M[row][col] * N[row][col])
    res.append(tmp)

```

再玩得更有趣一点：我们可以用 `zip` 把要相乘的项配对——下面的推导式和循环语句都产生与上例相同的逐项相乘的"列表的列表"结果（而且因为 `zip` 是一个值的生成器，这并没有看上去那么低效）：

```

[[col1 * col2 for (col1, col2) in zip(row1, row2)] for (row1, r...

```

```

res = []
for (row1, row2) in zip(M, N):
    tmp = []
    for (col1, col2) in zip(row1, row2):
        tmp.append(col1 * col2)
    res.append(tmp)

```

与它们的语句等价形式相比，这里的列表推导式版本只需要一行代码，对大型矩阵可能明显更快，而且可能让你大脑爆炸。这正好把我们带向下一节。

代码分析

·矩阵示例演示了推导式的两种"嵌套模式"：扁平嵌套（两个 `for` 在同一个方括号内 → 输出一维列表）与左侧嵌套（第二个推导式包在第一个的表达式位置上 → 输出二维列表）。区别在于：外层每次迭代是否开始一个新列表。

·解释器处理 `[[col + 10 for col in row] for row in M]` 时，外层推导式每取到一行 `row`，就触发内层推导式的完整执行（内层也是一个独立的、带自身字节码的循环），内

层 LISTAPPEND 结束后，外层再把它 append 进外层结果列表。所以外层循环 N 次，内层循环共执行 $N \times$ 列数次——这就是"嵌套"的真实执行时序。

- zip 版的对角配对利用了解包：for (row1, row2) in zip(M, N) 每次迭代解包出一对行，内层 zip(row1, row2) 再解包出一对列元素。zip 是惰性生成器，按需产出配对，所以不会先造出中间大列表。

- 设计思想：同一份数据、同一个操作，可以用"一个循环 + 一个 append"（语句）、"扁平推导式"（一维）或"嵌套推导式"（二维）表达。作者反复让你做"语句推导式"翻译练习，因为这是读懂任何复杂推导式的唯一可靠方法。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.6 When Not to Use List Comprehensions: Code Obfuscation (何时不要用列表推导式: 代码混淆)

英文原文

With such generality, list comprehensions can quickly become, well, incomprehensible, especially when nested. Some programming tasks are inherently complex, and we can't sugarcoat them to make them any simpler than they are (see the upcoming permutations for a prime example). Tools like comprehensions are powerful solutions when used wisely, and there's nothing inherently wrong with using them in your scripts. At the same time, code like that at the end of the prior section may introduce more complexity than it should—and, frankly, tends to disproportionately spark the interest of those holding the darker and misinformed assumption that code obfuscation somehow implies talent. In the interest of software citizenship, some perspective seems in order here. This book demonstrates advanced comprehensions to teach, but in the real world, programming is not about being clever and obscure—it's about how clearly your program communicates its purpose. Writing tricky comprehensions may be a fun academic recreation, but it doesn't work in programs that others will someday need to understand. In other words, the age-old acronym KISS a

plies here as always: Keep It Simple—traditionally followed either by a word that is now too sexist, or another that is too colorful for a G-rated book like this. If you have to translate code to simpler statements to understand it, it should probably be simpler statements in the first place.

中文翻译

有了如此强的通用性，列表推导式也会很快变得，呃，不可理解（incomprehensible），尤其是嵌套的时候。有些编程任务天生复杂，我们不能粉饰它，让它比实际上更简单（即将出场的排列（permutations）就是绝佳例子）。像推导式这样的工具，用得明智时是强大的解决方案，在你的脚本中使用它们本身也毫无问题。与此同时，上一节结尾那样的代码可能引入了超出必要的复杂性——而且，坦率地说，这种代码往往格外吸引那些抱着阴暗且错误想法（认为代码混淆暗示着才华）的人的兴趣。出于“软件公民意识”（software citizenship），这里似乎有必要给点清醒的看法。本书演示高级推导式是为了教学，但在真实世界里，编程不是关于聪明和晦涩——而是关于你的程序多么清晰地传达它的目的。编写刁钻的推导式也许是种有趣的学术消遣，但在那些将来需要别人理解的程序中行不通。换句话说，古老的缩写 KISS 一如既往地适用：Keep It Simple（保持简单）——传统上后面跟着一个如今已嫌性别歧视的词，或者另一个对这本书这种老少皆宜（G-rated）读物来说太过出格的词。如果你需要把代码翻译成更简单的语句才能理解它，那么它从一开始就该写成更简单的语句。

深度理解

- 核心概念：可读性 > 炫技。作者明确表态：推导式是"教学演示"的，真实代码里"清晰传达意图"才是第一位的。
- 设计原因：本章多次出现"代码被阅读的次数远多于被编写的次数"这一理念——一个表达式如果必须"展开成语句"才能读懂，那它写复杂了。
- 实际问题：团队协作中，后人维护你代码的成本常常超过你当初省下的几行。行数上的"简洁"如果以"理解上的晦涩"为代价，就是负资产。
- 初学者误区：误以为"越短越好"或"一行大师代码 = 高水平"。判断标准从来不是行数，而是"另一双眼睛（包括六个月后的你自己）能否一眼看懂"。
- 底层视角：复杂度不随写法消失——嵌套推导式和嵌套循环在语义上等价，只是语法密度不同。密度越高，解读成本越高。

20.7 When to Use List Comprehensions: Speed, Conciseness, etc. (何时使用列表推导式：速度、简洁等)

英文原文

Nevertheless, in this case, there is currently a performance advantage to the extra complexity: based on tests run under Python today, map calls and list comp

rehensions can be significantly faster than equivalent for loops. This speed difference can vary per usage and Python, but is due to the fact that map and list comprehensions run at compiled-language speed inside the interpreter, which is generally faster than running for loop bytecode within the PVM. Also in the plus column, list comprehensions offer a code conciseness that's compelling and even warranted when that reduction in size doesn't also imply a reduction in meaning for the next programmer; many find the expressiveness of comprehensions to be a powerful ally; and because map and list comprehensions are both expressions, they also can show up syntactically in places that for loop statements cannot, such as in lambda and object literals. Because of all this, list comprehensions and map calls are worth knowing and using for simple sorts of iterations, especially if your application's speed is an important consideration. For example, it's hard to argue with the ease and power of code like either of the following:

```
[line.rstrip() for line in open('myfile')]
map((lambda line: line.rstrip()), open('myfile'))
```

To achieve the same memory economy and execution time division as map, though, list comprehensions must be coded as generator expressions—which is why we've run through this review in the first place, and one of the major topics this chapter turns to next.

中文翻译

然而，在这种情形下，额外的复杂性确实带来性能优势：根据当今 Python 下的测试，map 调用和列表推导式可能显著快于等价的 for 循环。速度差异因用途和 Python 版本而异，但原因在于：map 和列表推导式是在解释器内部以“编译语言速度”运行的，通常快于在 PVM（Python 虚拟机）中逐条执行 for 循环的字节码。在加分项里还有：列表推导式提供的代码简洁性（conciseness）很有说服力，当这种体积缩减并不伴随下一位程序员理解上的损失时，甚至理应使用；许多人觉得推导式的表现力（expressiveness）是强大的盟友；而且由于 map 和列表推导式都是表达式，它们在语法上可以出现在 for 循环语句无法出现的地方，比如 lambda 和对象字面量（object literals）里。正因如此，列表推导式和 map 调用值得你了解并在简单迭代中使用，尤其是当你的应用把速度当作重要考量时。例如，很难反驳下面任一写法的易用与强大：

```
[line.rstrip() for line in open('myfile')]
map((lambda line: line.rstrip()), open('myfile'))
```

不过，要获得与 map 相同的“内存经济性”和“执行时间分割”（即按需产生结果），列表推导式必须改写成生成器表达式——这正是我们首先回顾这一整套内容的原因，也是本章接下来要讲的主要话题之一。

代码分析

·两行代码都对文件对象做逐行处理。open('myfile') 返回的文件对象本身就是惰性的（按需读行），rstrip 去掉行尾换行。

- `[line.rstrip() for line in open('myfile')]` 会一次把整个文件内容全部读进内存建列表；而 `map(...)` 只返回一个 `map` 对象，每次 `next()` 才读一行、处理一行——文件多大都不会先耗尽内存。

- 底层实现：列表推导式的 `LISTAPPEND` 与 `map` 的内部循环都在 C 层执行，因此比"Python 解释器逐条执行 `for` 循环字节码"快；但前者是一次性建表，后者是惰性生成。

- 设计思想：这一小节的落点是"想要 `map` 的惰性 + 推导式的语法 = 生成器表达式"。作者埋下伏笔，让读者带着"为什么需要生成器表达式"这个问题进入下一大节。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.8 Generator Functions and Expressions (生成器函数与表达式)

英文原文

Python today supports procrastination much more than it did in the past—it provides tools that produce results only when needed, instead of all at once. We've seen this at work in built-in tools: files that read lines on request, and functions like `map` and `zip` that produce items on demand. Such laziness isn't confined to Python itself, though. In particular, two language constructs delay result creation whenever possible in user-defined operations:

- Generator functions are coded as normal `def` statements, but use `yield` statements to return results one at a time, suspending and resuming their state between each.
- Generator expressions are similar to the list comprehensions of the prior section, but they return an object that produces results on demand instead of building a result list. Because neither of these constructs a result list all at once, they both save memory space and allow computation to be split across requests to avoid pauses and enable large (and even "infinite") results. As you'll see, both of these ultimately perform their delayed-results magic by implementing the iteration protocol we studied in Chapter 14. These features date back to at least Python 2.2 (and were explored even earlier in lang

languages like Icon), and are common in Python code today. Though they may initially seem unusual if you're accustomed to simpler models, you'll probably find generators to be a powerful tool. Moreover, because they are a natural extension to the function, comprehension, and iteration topics we've already explored, you already know more about coding generators than you might expect.

中文翻译

今天的 Python 比过去“拖延症”严重得多——它提供的工具只在需要时才产生结果，而不是一下子全部给出。我们已经在内置工具中看到过这种工作方式：按需读取行的文件，以及按需产出条目的 `map` 和 `zip` 之类的函数。不过，这种“懒惰”并不局限于 Python 自身。特别是，有两个语言构造在用户自定义的操作中尽可能推迟结果的创建：- 生成器函数 (generator functions)：用普通的 `def` 语句编写，但使用 `yield` 语句一次返回一个结果，并在每次返回之间挂起 (suspend) 和恢复 (resume) 自己的状态。- 生成器表达式 (generator expressions)：与上一节的列表推导式类似，但它们返回的是一个按需产生结果的对象，而不是构建一个结果列表。因为这两者都不会一次性构造出结果列表，所以它们都节省内存空间，并允许把计算拆分到多次请求之间执行，以避免停顿 (pauses)，还能支持庞大 (甚至“无限”) 的结果。你会看到，这两者最终都是通过实现第 14 章研究过的迭代协议 (iteration protocol) 来完成“延迟结果”的把戏的。这些特性至少可以追溯到 Python 2.2 (在更早的语言中，如 Icon，就已经探索过类似概念)，在今天

的 Python 代码中很常见。如果你习惯了更简单的模型，它们起初可能显得不寻常，但你很可能会发现生成器是强大的工具。而且，因为它们是函数、推导式和迭代主题的自然延伸——这些我们都已经探索过了，关于如何编写生成器，你已经知道的可能比你想象的要多。

深度理解

- 核心概念：作者用"procrastination（拖延）"这个俏皮词概括惰性求值（lazy evaluation）——"结果需要时才产生，而非一次全给"。生成器函数与生成器表达式是用户侧的两件惰性工具。
- 底层实现：二者最终都落实为"实现迭代协议的对象"——即拥有 `iter` 和 `next` 方法的对象。`for` 循环不关心对象是列表还是生成器，只要协议对，就能迭代。
- 设计原因：内存与时间权衡。一次性建表：大列表先占满内存、调用方必须干等；惰性生成：内存只存一个"状态机"，时间被分摊到每次请求。
- 实际问题：日志逐行处理、网络流、无限序列、需要"边产边用"的流水线——惰性工具是唯一可行的方案。
- 初学者误区：以为"生成器 = 存储序列的容器"。恰恰相反，生成器几乎不占存储——它只存代码位置和局部变量状态（本章下文详述）。

20.9 Generator Functions: yield Versus return（生成器函数：yield 与 return 的对决）

英文原文

In this part of the book, we've learned about coding normal functions that receive input parameters and send back a single result immediately. It is also possible, however, to write functions that may send back a value and later be resumed, picking up where they left off. Such functions are known as generator functions because they generate a series of values over time instead of stopping after just one. Generator functions are like normal functions in most respects, and in fact are coded with normal `def` statements. However, when created, they are compiled specially into an object that supports the iteration protocol. When called, they don't return a result: they return a result generator that can appear in any iteration context. We studied iterables in Chapter 14, and Figure 14-1 gave a formal and graphic summary of their operation. Here, we'll revisit the subject to see how it applies to generators.

State suspension (状态挂起)

Unlike normal functions that return a value and exit, generator functions automatically suspend and resume their execution and state around the point of value generation. Because of that, they are often a useful alternative to both computing an entire series of values up front and manually saving and restoring state in classes. The state that generator functions retain w

hen they are suspended includes both their code location and their entire local scope. Hence, their local variables retain information between results, and make it available when the functions are resumed. The chief code difference between generator and normal functions is that a generator yields a value, rather than returning one—the yield statement suspends the function and sends a value back to the caller, but retains enough state to enable the function to resume from where it left off. When resumed, the function continues execution immediately after the last yield run. From the function's perspective, this allows its code to produce a series of values over time, rather than computing them all at once and sending them back in something like a list.

Iteration protocol integration (与迭代协议的整合)

To truly understand generator functions, you need to know that they are closely bound up with the notion of the iteration protocol in Python. As we've seen, iterator objects define a next method, which either returns the next item in the iteration, or raises the StopIteration exception to end the iteration. An iterable object's iterator is fetched initially with the iter method, though this step is a no-op for objects that are their own iterator. Python for loops, and all other iteration tools, use this iteration protocol to step through a sequence or value generator, if the protocol is supported.

ed (if not, iteration falls back on repeatedly indexing sequences instead). Any object that supports this interface works in all iteration tools, and the `iter` and `next` built-ins simplify manual iteration by calling the corresponding double-underscore method (or its internal equivalent). To support this protocol, functions containing a `yield` statement are compiled specially as generators—they are not normal functions, but rather are built to return an object with the expected iteration-protocol methods. When later called, such functions return an iterable object that supports the iteration protocol with an automatically created method named `next` to start or resume execution. Besides `yield`, generator functions may also use a `return` statement that, along with falling off the end of the `def` block, terminates the generation of values by automatically raising a `StopIteration` exception. A generator's `return` can also give an object that becomes the `value` attribute of the `StopIteration` exception, but it's ignored by iteration tools and uncommon. From the caller's perspective, the generator's `next` method simply starts or resumes the function and runs until either the next `yield` result is returned, or a `StopIteration` is raised. The net effect is that generator functions, coded as `def` statements containing `yield` statements, are automatically made to support the iteration methods protocol and thus may be used in any iteration tool to produce results over time and on demand. The presence of a `yield`

d in a def suffices to make all this happen, and none of this applies to lambda because it doesn't allow statements like yield (which is yet another reason to prefer def).

Generator functions in action (生成器函数实战)

As usual, this is probably simpler in code than narrative. The following defines a generator function that can be used to produce the squares of a series of numbers—over time:

```
> >> def gensquares(n):  
    for i in range(n):  
        yield i ** 2          # Resume here later
```

This function yields a value, and so returns to its caller, each time through the loop. When it is resumed, its prior state is restored, including the last values of its variables *i* and *n*, and control picks up again immediately after the yield statement. For example, when it's used in the body of a for loop, the first iteration starts the function and gets its first result; thereafter, control returns to the function after its yield statement each time through the loop:

```
> >> for i in gensquares(5): # Resume the function  
    print(i, end=' ' )      # Print last yielded value  
0 1 4 9 16
```

To end the generation of values, functions either use a return statement or simply allow control to fall off t

he end of the function body. As noted, return can give a value attached to the exit exception, but usually does not. To most people, this process seems a bit implicit (if not enchanted) on first encounter. It's actually quite tangible, though. If you really want to see what is going on inside the for, call the generator function directly:

```
> >> x = gensquares(3) # Generator of 0..2 squares >> x
```

```
<generator object gensquares at 0x10dc6d700>
```

You get back a generator object that supports the iteration protocol—the generator function was compiled to return this automatically. The returned generator object in turn has a next method that starts the function or resumes it from where it last yielded a value, and raises a StopIteration exception when the end of the series of values is reached and the function returns. As noted, next(X) here calls X.next() for convenience:

```
> >> next(x) # Run to first yield
```

```
0
```

```
> >> next(x) # Resume after yield, run to next yield
```

```
1
```

```
> >> next(x) # Resume after yield, run to next yield
```

```
4
```

```
> >> next(x) # Resume after yield, exception on return
```

```
StopIteration
```

Per Chapter 14, for loops and other iteration tools work with generators in the same way—by calling the next method repeatedly, until the exit exception is caught. For generators, the result produces yielded values over time. Notice that the top-level iter call of the iteration protocol isn't required here because generators are their own iterator, supporting just one active iteration scan. To put that another way, generators return themselves for iter and support next directly. This also holds true in the generator expressions you'll meet later in this chapter:

```
> >> y = gensquares(5) # Returns a generator which is its own i...
```

```
True
```

```
> >> next(y) # Can run next() immediately
```

```
0
```

Why generator functions? (为什么要生成器函数?)

Given the simple example we're using to illustrate fundamentals, you might be wondering just why you'd ever care to code a generator at all. In code so far, for instance, we could simply build the list of result values all at once:

```
> >> def buildsquares(n):

    res = []
    for i in range(n): res.append(i ** 2)
    return res

> >> for x in buildsquares(5): print(x, end=' ')
```

```
0 1 4 9 16
```

For that matter, we could use any of the for loop, map, or list comprehension techniques we've already mastered:

```
> >> for x in [n ** 2 for n in range(5)]:

    print(x, end=' ')

0 1 4 9 16

> >> for x in map((lambda n: n ** 2), range(5)):
```

```
    print(x, end=' ')

0 1 4 9 16
```

However, generators can be better in terms of both memory use and performance in larger programs. They allow functions to avoid doing all the work up front, which is especially useful when the result lists are large or when it takes a lot of computation to produce each value. Generators distribute the time required to produce the series of values among loop iterations, and can even produce a series of values that has no end at all (an "infinite" result). Moreover, for more advanced uses, generators can provide a simpler alternative to manually saving the state between iterations in class objects—with generators, variables accessible

in the function's scopes are saved and restored automatically. You'll be able to judge this contrast after we discuss class-based iterables in more detail in Part VI. Generator functions are also more broadly focused than implied so far. They can operate on and return any type of object, and as iterables may appear in any of Chapter 14's iteration tools, including tuple calls, enumerations, and dictionary comprehensions:

```
> >> def ups(line):  
  
    for sub in line.split(','):   
        yield sub.upper()      # Substring generator  
> >> tuple(ups('aaa,bbb,ccc')) # All iteration contexts  
  
('AAA', 'BBB', 'CCC')  
> >> {i: s for (i, s) in enumerate(ups('aaa,bbb,ccc'))}  
  
{0: 'AAA', 1: 'BBB', 2: 'CCC'}
```

In a moment you'll observe the same assets for generator expressions—a tool that trades function flexibility for comprehension conciseness. Later in this chapter you'll also see that generators can sometimes enable otherwise impractical tasks, by producing components of result sets that would be far too large to create all at once. First, though, let's explore some advanced generator function features.

中文翻译

在本书的这一部分，我们学到了如何编写接收输入参数、并立即返回单个结果的普通函数。然而，同样有可能编写这样

的函数：先送回一个值，之后被恢复执行，从离开的地方继续。这类函数被称为生成器函数（generator functions），因为它们会随时间生成一系列的值，而不是只产生一个就结束。生成器函数在大多数方面与普通函数类似，事实上它们就是用普通 `def` 语句编写的。然而，在创建时，它们会被专门编译成一个支持迭代协议的对象。当被调用时，它们不返回结果：它们返回一个“结果生成器”（result generator），可以出现在任何迭代上下文中。我们在第 14 章研究过可迭代对象，图 14-1 给出了它们运作的正式而图形化的总结。这里我们将重访这个主题，看看它如何应用于生成器。

状态挂起 (State suspension)

与“返回一个值就退出”的普通函数不同，生成器函数会在值生成的节点自动挂起并恢复其执行和状态。正因如此，它们常常是两种做法的有用替代：一是预先计算一整套值，二是在类（classes）中手动保存和恢复状态。生成器函数在挂起时保留的状态（state）包括它的代码位置（code location）和完整的局部作用域（local scope）。因此，它们的局部变量在两次结果之间保留信息，并在函数被恢复时可供使用。生成器函数与普通函数的主要代码差异是：生成器产出（yields）一个值，而不是返回（returns）一个值——`yield` 语句挂起函数并把一个值送回给调用方，但保留足够的状态使函数能够从离开的地方恢复。恢复后，函数紧接着上一次运行的 `yield` 之后继续执行。从函数自身的角度来看，这使得它的代码可以随时间产生一系列值，而不是一次性全部算好、用某种列表之类的东西一次性送回。

与迭代协议的整合 (Iteration protocol integration)

要真正理解生成器函数，你需要知道它们与 Python 中迭代协议 (iteration protocol) 的概念紧密相连。正如我们所看到的，迭代器 (iterator) 对象定义了一个 `next` 方法，它要么返回迭代中的下一项，要么抛出 `StopIteration` 异常来结束迭代。一个可迭代对象的迭代器最初是通过 `iter` 方法取得的，不过对于"自己就是自己的迭代器"的对象来说，这一步是空操作 (no-op)。Python 的 `for` 循环以及所有其他迭代工具，都使用这个迭代协议来遍历序列或值生成器——前提是协议被支持（如果不支持，迭代会退化为反复对序列做下标索引）。任何支持这个接口的对象都能用于所有迭代工具，`iter` 和 `next` 这两个内置函数通过调用相应的双下划线方法（或它的内部等价物）来简化手动迭代。为了支持这个协议，包含 `yield` 语句的函数会被专门编译为生成器——它们不是普通函数，而是被构建成返回一个带有预期迭代协议方法的对象。这类函数在随后被调用时，返回一个支持迭代协议的可迭代对象，带有一个自动创建的名叫 `next` 的方法来启动或恢复执行。除了 `yield`，生成器函数也可以使用 `return` 语句，它与"执行落到 `def` 块的末尾"一样，会通过自动抛出 `StopIteration` 异常来终止值的生成。生成器的 `return` 也可以给出一个对象，作为 `StopIteration` 异常的 `value` 属性，但迭代工具会忽略它，这种情况也不常见。从调用方的角度看，生成器的 `next` 方法只是启动或恢复该函数并运行，直到返回下一个 `yield` 的结果，或抛出 `StopIteration`。最终效果是：作为包含 `yield` 语句的 `def` 编写的生成器函数，会被自动改造成支持迭代方法协议，从而可以在任何迭代工具中使用，随时间按需产出结果。`def` 中只要出现一个 `yield` 就足以让这一切发生，而这些对 `lam`

bda 一概不适用，因为 lambda 不允许 yield 这样的语句（这是更喜欢 def 的又一个理由）。

生成器函数实战 (Generator functions in action)

照例，用代码比用文字更容易讲明白。下面定义一个生成器函数，它可以随时间产生一系列数字的平方：

```
>>> def gensquares(n):  
    for i in range(n):  
        yield i ** 2
```

这个函数在循环的每一轮都 yield 一个值，从而返回给调用方。当它被恢复时，先前的状态被还原，包括变量 i 和 n 的最后值，控制流会从 yield 语句之后立刻继续。例如，当它在 for 循环体中使用时，第一次迭代启动函数并取得第一个结果；此后，每次循环控制权都会返回到函数中 yield 语句之后的位置：

```
>>> for i in gensquares(5):  
    print(i, end=' ')
```

0 1 4 9 16

要结束值的生成，函数要么使用 return 语句，要么干脆让控制流落到函数体末尾之外。如前所述，return 可以给出一个挂在退出异常上的值，但通常不会这样做。对大多数人来说，这个过程第一次遇到时显得有点隐晦（如果不是神奇的话）。不过，它其实非常具体实在。如果你真的想看看 for 内部发生了什么，直接调用这个生成器函数：

```
>>> x = gensquares(3)  
>>> x  
<generator object gensquares at 0x10dc6d700>
```

你拿回的是一个支持迭代协议的生成器对象（generator o

bject) ——生成器函数被编译成自动返回这个对象。返回的生成器对象又有一个 next 方法，它启动函数或从上次 yield 出值的地方恢复它，并在值的序列走到尽头、函数返回时抛出 StopIteration 异常。如前所述，这里的 next(X) 只是为了方便而调用 X.next()：

```
>>> next(x)          # yield
0
>>> next(x)          # yield yield
1
>>> next(x)          # yield yield
4
>>> next(x)          # yield
StopIteration
```

按照第 14 章的说法，for 循环和其他迭代工具以同样的方式与生成器协作——反复调用 next 方法，直到捕获到退出异常。对生成器来说，结果是随时间产出的值。注意：迭代协议顶层的 iter 调用在这里并不是必需的，因为生成器自己就是自己的迭代器，只支持一次激活的迭代扫描 (one active iteration scan)。换句话说，生成器为 iter 返回它们自身，并直接支持 next。这一点对本章稍后遇到的生成器表达式同样成立：

```
>>> y = gensquares(5)          #
>>> iter(y) is y               # iter()
True
>>> next(y)                    # next()
0
```

为什么要生成器函数？ (Why generator functions?)

面对我们用来讲基础的这个简单例子，你可能会好奇：到底为什么要费心写生成器？比如到目前为止的代码里，我们完

全可以一次性把结果值的列表建好：

```
>>> def buildsquares(n):
    res = []
    for i in range(n): res.append(i ** 2)
    return res
>>> for x in buildsquares(5): print(x, end=' ')
0 1 4 9 16
```

就算退一步，我们也可以用已经掌握的 for 循环、map 或列表推导式中的任何一种技巧：

```
>>> for x in [n ** 2 for n in range(5)]:
    print(x, end=' ')
0 1 4 9 16
>>> for x in map((lambda n: n ** 2), range(5)):
    print(x, end=' ')
0 1 4 9 16
```

然而，在更大的程序中，生成器在内存使用 and 性能两方面都可能更优。它们让函数避免在开头就做完所有工作——当结果列表很大、或者每个值的计算量很大时，这一点尤其有用。生成器把产生这一系列值所需的时间分摊到各次循环迭代中，甚至能产生一个根本没有尽头（“无限”结果）的值序列。而且，对于更高级的用途，生成器可以提供比“在类对象里手动保存迭代间状态”更简单的替代方案——有了生成器，函数作用域内可访问的变量会被自动保存和恢复。等你到第六部分更详细地讨论了基于类的可迭代对象之后，就能体会到这种对比了。生成器函数的适用面也比目前暗示的更广。它们可以操作并返回任何类型的对象，并且作为可迭代对象可以出现在第 14 章的任何迭代工具中，包括 tuple 调用、枚举（enumerations）和字典推导式：

```
>>> def ups(line):
    for sub in line.split(','):
        yield sub.upper() #
>>> tuple(ups('aaa,bbb,ccc')) #
('AAA', 'BBB', 'CCC')
>>> {i: s for (i, s) in enumerate(ups('aaa,bbb,ccc'))}
{0: 'AAA', 1: 'BBB', 2: 'CCC'}
```

稍后你会看到生成器表达式同样具有这些优点——那是一种用"函数的灵活性"换取"推导式的简洁性"的工具。在本章后面你还会看到，生成器有时能完成本来不切实际的任务——它按需产出结果集的组成部分，而这些结果集一次性全部创建会大到不现实。不过，先让我们探索一些更高级的生成器函数特性吧。

代码分析

- gensquares 的核心：yield i²。解释器发现 def 体内有 yield，编译时就为该函数生成两个"版本"：函数体字节码本身 + 一个包装。调用 gensquares(3) 时，函数体一行都不会执行，只返回一个 generator object；真正执行是在第一次 next() 时。
- 状态保存在哪里：CPython 中生成器对象内部有一个 gframe（帧对象），保存着指令指针（IP，指向下一次要执行的字节码）和局部变量槽。yield 把当前帧"冻结"（不销毁），next() 把它重新挂到栈上恢复运行。这就是"状态挂起"的底层真相——不是魔法，是一个暂停的栈帧。
- next(x) 四次调用的执行轨迹：第一次运行到 yield 返回 0；第二次从 yield 之后继续循环到第二次 yield 返回 1；第三次返回 4；第四次循环结束、函数落到末尾，生成器

自动抛出 `StopIteration`——`for` 循环正是靠捕获它来静默退出。

- `iter(y) is y` 为 `True`: 生成器对象实现了 `iter` (返回 `self`) 和 `next`, 因此是"自迭代器"。`for` 拿到它后直接调 `next`, 不会再造第二个迭代器。
 - `ups` 的例子说明生成器可无缝进入 `tuple()`、`enumerate()`、字典推导式等一切迭代上下文——它们全部内部调 `next()`, 与生成器结构无关。
 - 设计思想: `yield` 把"函数"变成了"可暂停的协程雏形"。返回值是一次性答案, 产出值是时间上展开的答案流。两者都是"函数", 但编译路径截然不同。
-

深度理解

- 核心概念: 把握本节在整章知识链中的位置, 弄清它引入或巩固的关键术语与机制。
- 底层实现: 对照 Python 运行时/迭代协议/对象模型, 理解代码实际如何被解释器处理。
- 设计原因: Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题: 可在真实项目中用本节技术解决哪些任务; 何时该用、何时不该用。
- 初学者误区: 最常见的混淆点 (与相似工具的边界、求值时机、可变/共享状态等)。

20.10 Extended Generator Function Protocol: send Versus next (扩展的生成器函数协议: send 与 next)

英文原文

Somewhere along generators' evolutionary path (in Python 2.5), a `send` method was added to the generator function protocol. The `send` method advances to the next item in the series of results, just like `next`, but also provides a way for the caller to communicate with the generator, and hence to affect its operation. Technically, `yield` is an expression form that returns the item passed to `send`, not a statement. It can be coded either way (and usually is a statement), but when used as an expression must be enclosed in parentheses unless it's the only item on the right side of an assignment statement. For example, `X = yield Y` is OK, as is `X = (yield Y) + Z`. When this extra protocol is used, values are sent into a generator `G` by calling `G.send(value)`. The generator's code is then resumed, and the `yield` expression inside the generator returns the value passed to `send` by the caller. If the regular `G.next()` method (or its `next(G)` equivalent) is called to advance, the `yield` simply returns `None`. For example:

```

> >> def gen():
    for i in range(10):
        X = yield i
        print('=>', X)
> >> G = gen() >> next(G) # Must call next() first, to start gener...

0
> >> G.send(77) # Advance, and send value to yield expression

=> 77
1
> >> G.send(88)

=> 88
2
> >> next(G) # next() and X.__next__() send None

=> None
3

```

The send method can be used, for example, to code a generator that its caller can terminate by sending a termination code, or redirect by passing a new position in data being processed inside the generator. In addition, generators also support a throw(type) method to raise an exception inside the generator at the latest yield, and a close method that raises a special GeneratorExit exception inside the generator to terminate the iteration entirely. Together with send, these are advanced features added to make generators more like a tool called coroutines, a role eventually subsumed in part by the upcoming async. Hence, we won't delve further here; see Python's standard manuals for

more information, and watch for more on exceptions in Part VII. Note, though, that while Python provides a `next(X)` convenience built-in that calls the `X.next()` method of an object, other generator methods, like `send`, must be called as methods of generator objects directly (e.g., `G.send(X)`). This makes sense if you realize that these extra methods are implemented on built-in generator objects only, whereas the `next` method applies to all iterable objects—both built-in types and user-defined classes.

中文翻译

在生成器演化之路的某处（Python 2.5），一个 `send` 方法被加入生成器函数协议。`send` 方法像 `next` 一样推进到结果序列中的下一项，但它还提供了一种让调用方与生成器通信、进而影响其运作的方式。严格来说，`yield` 是一种表达式形式，它会返回传给 `send` 的项，而不是一条语句。它两种写法都行（通常写作语句），但当作为表达式使用时必须用括号括起来，除非它是赋值语句右侧唯一的一项。例如 `X = yield Y` 是合法的，`X = (yield Y) + Z` 也合法。当使用这个扩展协议时，值通过调用 `G.send(value)` 被送进生成器 `G`。接着生成器的代码被恢复，生成器内部的 `yield` 表达式会返回调用方传给 `send` 的那个值。如果用常规的 `G.next()` 方法（或其 `next(G)` 等价物）来推进，`yield` 就简单地返回 `None`。例如：

```

>>> def gen():
    for i in range(10):
        X = yield i
        print('=>', X)
>>> G = gen()
>>> next(G)                # next()
0
>>> G.send(77)            # yield
=> 77
1
>>> G.send(88)
=> 88
2
>>> next(G)                # next() X.__next__() None
=> None
3

```

例如，`send` 方法可以用来编写这样的生成器：调用方可以通过发送一个终止码来终止它，或通过传入一个在处理数据中的新位置来重定向它。此外，生成器还支持 `throw(type)` 方法，用来在生成器最新的 `yield` 处抛出一个异常；以及 `close` 方法，它在生成器内部抛出一个特殊的 `GeneratorExit` 异常来彻底终止迭代。与 `send` 一起，这些是为了让生成器更接近一种叫协程（`coroutines`）的工具而加入的高级特性，这个角色后来部分地被即将到来的 `async` 所吸收。因此，这里我们不再深入；更多信息请看 Python 的标准手册，并留意第七部分关于异常的更多内容。不过请注意：虽然 Python 提供了一个调用对象 `X.next()` 方法的便利内置函数 `next(X)`，但其他生成器方法，如 `send`，必须作为生成器对象的方法直接调用（例如 `G.send(X)`）。如果你意识到这些额外的方法只实现在内置的生成器对象上，而 `next` 方法适用于所有可迭代对象——既有内置类型也有用户自定义类——那么这就说得通了。

代码分析

· `X = yield i` 同时做了两件事：把 `i` 产给调用方（`yield` 的右半部分），并把调用方传来的值赋给 `X`（`yield` 表达式的求值结果）。启动时必须用 `next(G)` 先跑"到第一个 `yield`"，因为生成器是惰性启动的，第一次必须先"推"它到暂停点。

· 执行轨迹：`next(G)` → 停在 `yield i`（返回 0），此时 `X` 尚未赋值；`G.send(77)` → 从 `yield` 处恢复，`yield` 表达式求值为 77 赋给 `X`，打印 `=> 77`，循环到下一个 `yield i` 返回 1；`next(G)` → `yield` 表达式求值为 `None` 赋给 `X`，打印 `=> None`。

· 底层实现：CPython 中用 `giframe.flasti` 指向暂停的字节码位置；`send` 把值写入当前帧的栈顶，让 `YIELDVALUE` 恢复后立刻能取到这个值。`send` 与 `next` 的唯一区别就是"是否注入值"。

· 设计思想：这本质上是"双向管道"——调用方可以喂数据给生成器，生成器成为可被外部控制的状态机，这正是协程的前身、也是 `async` 模型的种子。

· 初学者误区：①对尚未启动的生成器直接 `send(non-None)` 会抛 `TypeError`（必须先 `next`）；②认为 `yield i` 的 `i` 与 `X = yield i` 会混淆——前者是产出，后者产出的同时还能接收。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.11 The yield from Extension (yield from 扩展)

英文原文

Even later, Python 3.3 introduced extended syntax for the yield statement with a from generator clause that allows generators to delegate to nested generators (known as subgenerators). In simple cases, it's the equivalent to a yielding for loop. As a demo, the list call in the following forces a generator to produce values from two sources:

```
> >> def both(N):
    for i in range(N): yield i
    for i in map(lambda x: x ** 2, range(N)): yield i
> >> list(both(5))

[0, 1, 2, 3, 4, 0, 1, 4, 9, 16]
```

The `yield from` syntax makes this more concise and explicit, and supports all the usual generator usage contexts. In the following, `both` is called a delegating generator, and the `range` and `map` built-ins serve as subgenerators (join also uses a generator expression here: see the next section):

```
> >> def both(N):
    yield from range(N)
    yield from map(lambda x: x ** 2, range(N))
> >> list(both(5))

[0, 1, 2, 3, 4, 0, 1, 4, 9, 16]
> >> ': '.join(str(i) for i in both(5))

'0 : 1 : 2 : 3 : 4 : 0 : 1 : 4 : 9 : 16'
```

The `yield from` also supports arbitrary chains of generators. In the following, for example, two generators `yield from` others, the last of which ultimately uses `yield` to send results up through the chain:

```

> >> def c(n):

    yield n * 8          # The bottom of the "chain"
    yield n ** 2
> >> def b(n):

    yield from c(n)      # Delegate to subgenerator
> >> def a(n):

    yield from b(n)      # Delegate to subgenerator
> >> g = a(4) # Make top generator, fetch results >> g

<generator object a at 0x10bd17940>
> >> next(g)

32
> >> for i in a(4): print(i) # Force results production

32
16

```

In more advanced roles, though, this extension allows subgenerators to receive sent and thrown values directly from the delegating generator's caller, and return a final value to the delegating generator as the result of the yield from expression.

The net effect allows generators to be split into multiple subgenerators much as a single function can be split into multiple subfunctions, but still operate as though their code appeared inline where the yield from appears.

Since this is both uncommon and beyond this chapter's scope, we'll again defer to Python's standard man

uals for more details.

For an additional yield from example, see the solution to Exercise 11 from "Test Your Knowledge: Part IV Exercises", and stay tuned for a glimpse of async def coroutines ahead—whose await is something like a generator yield from, with a delegation to an event loop to allow other tasks to be run during pauses. Here, let's move on to a tool close enough to yield to be called a fraternal twin.

中文翻译

更早些, Python 3.3 为 yield 语句引入了扩展语法——一个 from generator (来自某个生成器) 子句, 允许生成器把工作委派给嵌套的生成器 (称为子生成器 (subgenerators))。在简单情形下, 它等价于一个不断 yield 的 for 循环。作为演示, 下面的 list 调用强迫一个生成器从两个来源产生值:

```
>>> def both(N):
    for i in range(N): yield i
    for i in map(lambda x: x ** 2, range(N)): yield i
>>> list(both(5))
[0, 1, 2, 3, 4, 0, 1, 4, 9, 16]
```

yield from 语法让这更简洁、更明确, 并支持所有常规的生成器使用场景。在下面代码中, both 被称为委派生成器 (delegating generator), range 和 map 内置函数充当子生成器 (join 在这里也用到了生成器表达式: 见下一节) :

```
>>> def both(N):
    yield from range(N)
    yield from map(lambda x: x ** 2, range(N))
>>> list(both(5))
[0, 1, 2, 3, 4, 0, 1, 4, 9, 16]
>>> ' : '.join(str(i) for i in both(5))
'0 : 1 : 2 : 3 : 4 : 0 : 1 : 4 : 9 : 16'
```

yield from 还支持任意的生成器链 (chains)。例如在下面代码中，两个生成器 yield from 别的生成器，而最后那个生成器最终用 yield 把结果逐层送上传链：

```
>>> def c(n):
    yield n * 8          # ""
    yield n ** 2
>>> def b(n):
    yield from c(n)      #
>>> def a(n):
    yield from b(n)      #
>>> g = a(4)            #
>>> g
<generator object a at 0x10bd17940>
>>> next(g)
32
>>> for i in a(4): print(i)  #
32
16
```

不过在更高级的角色中，这个扩展允许子生成器直接从委派生成器的调用方接收 sent 和 thrown 的值，并把一个最终值作为 yield from 表达式的结果返回给委派生成器。最终效果是：生成器可以被拆分成多个子生成器，正如一个函数可以被拆分成多个子函数一样，但仍然表现得像它们的代码就内联在 yield from 出现的位置一样。由于这既少见又超出本章范围，细节我们再次交给 Python 的标准手册。关于

yield from 的更多示例，参见"Test Your Knowledge: Part IV Exercises"第 11 题的解答；并期待后面 async def 协程的惊鸿一瞥——它的 await 有点像生成器的 yield from，只是把控制委派给事件循环（event loop），以便在暂停期间运行其他任务。这里，我们继续往下，去看一个与 yield 亲近到足以被称为"同胞孪生兄弟"的工具。

代码分析

- both(5) 的两个版本输出完全相同。yield from range(N) 相当于"把这个可迭代对象里的每一项逐个 yield 出来"，把"循环 + yield"两行压成一行，意图更直白。
- 执行轨迹（链式）：next(a(4)) 时，a 的 yield from b(n) 把控制流交给 b，b 再 yield from c(n) 交给 c，最终 c 里 yield n 8 产出 32；再次 next 时 c 继续产出 $n - 2 = 16$ 。整个链只有"最底层"真正 yield，中间层只是无损耗地透传。
- 底层实现：CPython 为 yield from 生成专门字节码（如 SEND、YIELDFROM），使委派生成器直接驱动子生成器（子生成器在运行期间成为激活的唯一帧），比"外层开 for 循环逐个 yield"开销更小、语义更完整（send/throw/return 都能透传）。
- 设计思想：这是"函数拆分"思想在生成器世界的翻版——复杂生成逻辑可以分解成多个小生成器，再像流水管线一样串起来。这和 await 的关系：await 就相当于把控制权交给事件循环去唤醒另一个任务，本质同源。
- 初学者误区：误以为 yield from 只是语法糖、与 for 循

环等价。它真正的价值在于能双向传递 `send/throw` 值并传递返回值，这是对高级协程的支持，`for` 循环做不到。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.12 Generator Expressions: Iterables Meet Comprehensions (生成器表达式：可迭代对象遇见推导式)

英文原文

Because the delayed evaluation of generator functions was so useful, later Pythons eventually combined the notions of iterables and comprehensions in a new tool: generator expressions. Syntactically, generator expressions are just like normal list comprehensions, a

nd support all their syntax—including if filters and for loop nesting—but they are enclosed in parentheses instead of square brackets (like tuples, their enclosing parentheses are often optional):

```
> >> [x ** 2 for x in range(5)] # List comprehension: build a l...  
[0, 1, 4, 9, 16]  
> >> (x ** 2 for x in range(5)) # Generator expression: make an it...  
<generator object <genexpr> at 0x109607ac0>
```

In fact, at least on a functionality basis, coding a list comprehension is essentially the same as wrapping a generator expression in a list built-in call to force it to produce all its results in a list at once:

```
> >> list(x ** 2 for x in range(5)) # List comprehension equiva...  
[0, 1, 4, 9, 16]
```

Operationally, however, generator expressions are very different: instead of building the result list in memory, they return a generator object—an automatically created iterable. This iterable object in turn supports the iteration protocol to yield one piece of the result list at a time in any iteration tool. The iterable object also retains generator state while active—the variable *x* in the preceding expressions, along with the generator's code location.

The net effect is much like that of generator functions, but in the context of a comprehension expression: we get back an object that remembers where it left o

ff after each part of its result is returned. Also like generator functions, looking under the hood at the protocol that these objects automatically support can help demystify them; the `iter` call is again not required at the top here, for reasons we'll expand on ahead:

```
> >> G = (x ** 2 for x in range(3)) >> iter(G) is G # iter(G) o...
True
> >> next(G) # Generator objects: automatic methods
0
> >> next(G)
1
> >> next(G)
4
> >> next(G)
StopIteration
> >> G
<generator object <genexpr> at 0x109607920>
```

Again, we don't typically see the next iterator machinery under the hood of a generator expression like this because for loops and similar tools trigger it for us automatically:

```
> >> for num in (x ** 2 for x in range(3)): # Calls next() auto...
    print(num, num / 2.0, sep=', ')
0, 0.0
1, 0.5
4, 2.0
```

As we've already seen, every iteration tool does this—including for loops; the list call we used earlier; comprehensions of all kinds; the map and filter functional tools; and other iteration tools we explored in Chapter 14, including the sum, sorted, any, and all built-in functions. As iterables, generator expressions can be used in any of these iteration tools, just like both physical sequences and the results of a generator-function call.

For example, the following deploys generator expressions in the string join method call and tuple assignment, iteration tools both. Recall that join with an empty-string delimiter concatenates values produced by its iterable:

```
> >> ''.join(x.upper() for x in 'aaa,bbb,ccc'.split(','))  
  
'AAABBBCCC'  
> >> a, b, c = (x + '\n' for x in 'aaa,bbb,ccc'.split(',')) >> a, c  
  
('aaa\n', 'ccc\n')
```

Notice how the join call in the preceding doesn't require extra parentheses around the generator. Syntactically, parentheses are not required around a generator expression that is the sole item already enclosed in parentheses used for other purposes—like those of a function call. Parentheses are required in all other cases, however, even if they seem extra, as in the second call to sorted that follows:

```
> >> sum(x ** 2 for x in range(3)) # Parens optional
5
> >> sorted(x ** 2 for x in range(3)) # Parens optional

[0, 1, 4]
> >> sorted((x ** 2 for x in range(3)), reverse=True) # Parens req...

[4, 1, 0]
```

Like the often-optional parentheses in tuples, there is no widely accepted rule on this, though a generator expression does not have as clear a role as a fixed collection of other objects as a tuple, making extra parentheses seem perhaps more spurious here.

Why generator expressions? (为什么要生成器表达式?)

Just like generator functions, generator expressions are a memory-space optimization—they do not require the entire result list to be constructed all at once, as the square-bracketed list comprehension does. Also like generator functions, they divide the work of results production into smaller time slices—they yield results in piecemeal fashion, instead of making the caller wait for the full set to be created in a single call.

On the other hand, because generator expressions often run slower today than list comprehensions, they may be best reserved for result sets that are very large, or programs that cannot wait for full results generation. A more authoritative statement about perform

ance, though, will have to await the timing scripts we'll code in the next chapter.

On the subjective upside, generator expressions also offer significant coding advantages—as the next sections show.

中文翻译

由于生成器函数的延迟求值（delayed evaluation）如此有用，后来的 Python 版本最终把“可迭代对象”和“推导式”这两个概念组合成一个新工具：生成器表达式（generator expressions）。在语法上，生成器表达式与普通列表推导式完全一样，支持它们的所有语法——包括 if 过滤器和 for 循环嵌套——但它们用圆括号而非方括号包围（和元组一样，它们的包围括号常常可以省略）：

```
>>> [x ** 2 for x in range(5)]    #  
[0, 1, 4, 9, 16]  
>>> (x ** 2 for x in range(5))    #  
<generator object <genexpr> at 0x109607ac0>
```

事实上，至少从功能角度讲，编写一个列表推导式，本质上等同于把一个生成器表达式包进 list 内置函数调用里，以强迫它一次性把全部结果产生到一个列表中：

```
>>> list(x ** 2 for x in range(5)) #  
[0, 1, 4, 9, 16]
```

不过，操作层面（operationally）上，生成器表达式大不相同：它们不把结果列表构建在内存中，而是返回一个生成器对象（generator object）——一个自动创建的可迭代对象。这个可迭代对象又支持迭代协议，在任何迭代工具中一次产出结果列表的一块。激活期间，该可迭代对象还保留

生成器状态——比如前面表达式中的变量 `x`，以及生成器的代码位置。最终效果与生成器函数很像，只是发生在推导式表达式的语境中：我们拿回一个“每次返回一部分结果后，还记得自己走到哪”的对象。与生成器函数一样，深入底层看看这些对象自动支持的协议，有助于祛魅（demystify）——顶层的 `iter` 调用在这里同样不是必需的，原因我们在后面展开：

```
>>> G = (x ** 2 for x in range(3))
>>> iter(G) is G                # iter(G) __iter__
True
>>> next(G)                     #
0
>>> next(G)
1
>>> next(G)
4
>>> next(G)
StopIteration
>>> G
<generator object <genexpr> at 0x109607920>
```

同样地，我们通常不会这样看到生成器表达式引擎盖下的 `next` 迭代机制，因为 `for` 循环之类的工具会自动替我们触发它：

```
>>> for num in (x ** 2 for x in range(3)):    # next()
    print(num, num / 2.0, sep=', ')
0, 0.0
1, 0.5
4, 2.0
```

正如我们已经看到的，每一种迭代工具都会这样做——包括 `for` 循环；我们之前用过的 `list` 调用；各种推导式；`map` 和 `filter` 这两个函数式工具；以及我们在第 14 章探索过

的其他迭代工具，其中包括 `sum`、`sorted`、`any` 和 `all` 内置函数。作为可迭代对象，生成器表达式可以用在所有这些迭代工具中，就像物理序列以及生成器函数的调用结果一样。例如，下面在字符串 `join` 方法调用和元组赋值——两者都是迭代工具——中部署了生成器表达式。回忆一下：`join` 配合空字符串分隔符，会把其可迭代对象产生的值拼接起来：

```
>>> ''.join(x.upper() for x in 'aaa,bbb,ccc'.split(','))
'AAABBBCCC'
>>> a, b, c = (x + '\n' for x in 'aaa,bbb,ccc'.split(','))
>>> a, c
('aaa\n', 'ccc\n')
```

注意上面 `join` 调用并不要求在生成器外面加额外的括号。在语法上，如果一个生成器表达式已经是“为其他用途所加的括号（比如函数调用的括号）里唯一的项目”，那么它外面的括号不是必须的。然而，在其他所有情形下括号都是必须的，即使它们看起来多余，比如下面第二个 `sorted` 调用：

```
>>> sum(x ** 2 for x in range(3))    #
5
>>> sorted(x ** 2 for x in range(3)) #
[0, 1, 4]
>>> sorted((x ** 2 for x in range(3)), reverse=True) #
[4, 1, 0]
```

与元组那经常可省略的括号类似，这里并没有公认的规则，不过生成器表达式不像元组那样具有“其他对象的固定集合”这样清晰的角色，所以额外的括号在这里看起来也许更加多余。

为什么要生成器表达式？ (Why generator expressions?)

与生成器函数一样，生成器表达式是一种内存空间优化——它们不要求像方括号列表推导式那样一次性构造出整个结果列表。也与生成器函数一样，它们把结果生产的工作切成更小的时间片——它们以零敲碎打的方式产出结果，而不是让调用方在单个调用中干等整套结果创建完成。另一方面，因为如今的生成器表达式通常比列表推导式运行得慢，它们可能最好留给“结果集非常庞大”或“程序无法等待完整结果生成”的场景。不过，关于性能的更权威的说法，还得等到下一章我们要写的计时脚本。在主观的加分项上，生成器表达式还提供显著的编码优势——正如接下来的小节所示。

代码分析

- `(x2 for x in range(5))` 与 `[x2 for x in range(5)]` 只差括号形状，但前者返回的是 `<generator object <genexpr>...>`——注意类型名是 `<genexpr>`，说明解释器为“生成器表达式”单独编译了一个匿名生成器函数（相当于一个隐式 `def roundedparensген()`）。`list(...)` 则立即驱动它并物化（materialize）成列表。
- `/2.0` 在 Python 3 中是“真除法”，所以 `1/2.0` 输出 `0.5`。
- 底层实现：生成器表达式在编译期被包进一个隐藏的嵌套帧（nested code object），运行时创建生成器对象。这也是为什么 `x` 有自己的“作用域”——循环变量其实住在生成器内部帧里。

·括号规则总结：作为函数调用/方法调用里唯一实参时括号可省（`sum(...)`、`join(...)`）；一旦后面还要跟其他实参（`reverse=True`），括号必须写全，否则 `sorted(...)` 会把逗号后的东西误认成第二个实参。

·设计思想："函数加一层 `list()` = 列表推导式"揭示了推导式与生成器的血缘关系：列表推导式本质上就是"对生成器表达式立即求值并建表"。把方括号换成括号，就把"立即求值"换成了"延迟求值"——同一语法，两种时空策略。

·初学者误区：①在只需要"遍历一次"的场合仍用列表推导式，白白多造一个列表；②以为 `sum(x2 for x in range(3))` 里的生成器"看不见"，其实它照常存在，只是寄生在括号里。

深度理解

·核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

·底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

·设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

·实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.13 Generator Expressions Versus map (生成器表达式与 map 的对比)

英文原文

One way to see the coding benefits of generator expressions is to compare them to other functional tools, as we did for list comprehensions. For example, generator expressions often are equivalent to map calls, because both generate result items on request. Like list comprehensions, though, generator expressions may be simpler to code when the operation applied is not a function call:

```
> >> list(map(abs, (-1, -2, 3, 4))) # Map function on tuple
[1, 2, 3, 4]
> >> list(abs(x) for x in (-1, -2, 3, 4)) # Generator expression
[1, 2, 3, 4]
> >> list(map(lambda x: x * 2, (1, 2, 3, 4))) # Nonfunction case
[2, 4, 6, 8]
> >> list(x * 2 for x in (1, 2, 3, 4)) # Simpler as generator?
[2, 4, 6, 8]
```

The same holds true for text-processing use cases like the join call we saw earlier—a list comprehension makes an extra temporary list of results, which is completely pointless in this context because the list is not retained, and map loses simplicity points compared to

o generator expression syntax when the operation being applied is not a call:

```
> >> line = 'aaa,bbb,ccc'>> ''.join([x.upper() for x in line.sp...  
  
'AAABBBCCC'  
> >> ''.join(x.upper() for x in line.split(',')) # Generates resul...  
  
'AAABBBCCC'  
> >> ''.join(map(str.upper, line.split(','))) # Generates results  
  
'AAABBBCCC'  
> >> ''.join(x * 2 for x in line.split(',')) # Simpler as generato...  
  
'aaaaaabbbbbcccccc'  
> >> ''.join(map(lambda x: x * 2, line.split(',')))  
  
'aaaaaabbbbbcccccc'
```

Both map and generator expressions can also be arbitrarily nested; when coded this way, a list call or other iteration tool starts the entire process of producing results. For example, the list comprehension in the following produces the same result as the map and generator equivalents that follow it, but makes two physical lists.

The others generate just one integer at a time with nested generators, and the generator expression form may more clearly reflect its intent:

```
> >> [x * 2 for x in [abs(x) for x in (-1, -2, 3, 4)]] # Nested...

[2, 4, 6, 8]
> >> list(map(lambda x: x * 2, map(abs, (-1, -2, 3, 4)))) # Nested...

[2, 4, 6, 8]
> >> list(x * 2 for x in (abs(x) for x in (-1, -2, 3, 4))) # Nester...

[2, 4, 6, 8]
```

Although the effect of all three of these is to combine operations, the generators do so without making multiple temporary lists. In fact, the next example both nests and combines generators—the nested generator expression is activated by `map`, which in turn is only activated by `list`:

```
> >> import math >> list(map(math.sqrt, (x ** 2 for x in range(...

[0.0, 1.0, 2.0, 3.0]
```

Technically speaking, the range on the right in the preceding is a value generator too, activated by the generator expression itself—forming three levels of value generation, which produce individual values from inner to outer only on request, and which "just works" because of Python's iteration tools and protocol. In fact, generator nestings can be arbitrarily mixed and deep, though some may be more valid than others:

```
> >> list(map(abs, map(abs, map(abs, (-1, 0, 1))))) # Nesting g...

[1, 0, 1]
> >> list(abs(x) for x in (abs(x) for x in (abs(x) for x in (-1, 0...

[1, 0, 1]
```

These last examples illustrate how general generators can be, but are also coded in an intentionally complex form to underscore that generator expressions have the same potential for abuse as the list comprehensions discussed earlier—as usual, you should keep them simple unless they must be complex.

When used well, though, generator expressions combine the expressiveness of list comprehensions with the space and time benefits of other iterables. The following nonnested alternatives, for example, provide simpler solutions to the preceding three listings' nested codings, but still leverage generators' strengths:

```
> >> list(abs(x) * 2 for x in (-1, -2, 3, 4)) # Same, unnested

[2, 4, 6, 8]
> >> list(math.sqrt(x ** 2) for x in range(4)) # Flat may be better

[0.0, 1.0, 2.0, 3.0]
> >> list(abs(x) for x in (-1, 0, 1))

[1, 0, 1]
```

中文翻译

要看出生成器表达式的编码好处，一个办法是像我们对列表推导式所做的那样，把它们与其他函数式工具比较。例如，

生成器表达式常常等价于 map 调用，因为两者都是按需生成结果项。不过，与列表推导式一样，当所应用的操作不是一个函数调用时，生成器表达式编码起来可能更简单：

```
>>> list(map(abs, (-1, -2, 3, 4)))          #  
[1, 2, 3, 4]  
>>> list(abs(x) for x in (-1, -2, 3, 4))    #  
[1, 2, 3, 4]  
>>> list(map(lambda x: x * 2, (1, 2, 3, 4))) #  
[2, 4, 6, 8]  
>>> list(x * 2 for x in (1, 2, 3, 4))       #  
[2, 4, 6, 8]
```

对 join 调用这类文本处理用例也是如此——列表推导式会额外制造一个临时结果列表，在这个语境中这完全是多余的，因为这个列表并不会被保留；而当应用的操作不是一个调用时，map 相比生成器表达式语法会在简洁性上失分：

```
>>> line = 'aaa,bbb,ccc'  
>>> ''.join([x.upper() for x in line.split(',')]) #  
'AAABBBCCC'  
>>> ''.join(x.upper() for x in line.split(','))   #  
'AAABBBCCC'  
>>> ''.join(map(str.upper, line.split(',')))      #  
'AAABBBCCC'  
>>> ''.join(x * 2 for x in line.split(','))       #  
'aaaaaabbbbbcccccc'  
>>> ''.join(map(lambda x: x * 2, line.split(',')))  
'aaaaaabbbbbcccccc'
```

map 和生成器表达式都可以被任意嵌套；这样编码时，一个 list 调用或其他迭代工具启动整个结果生产过程。例如，下面第一个列表推导式与紧随其后的 map 和生成器等价形式产生相同结果，但它制造了两个物理列表。另外两个用嵌套生成器每次只生成一个整数，而生成器表达式形式也许能

更清晰地反映其意图：

```
>>> [x * 2 for x in [abs(x) for x in (-1, -2, 3, 4)]] #  
[2, 4, 6, 8]  
>>> list(map(lambda x: x * 2, map(abs, (-1, -2, 3, 4)))) # map  
[2, 4, 6, 8]  
>>> list(x * 2 for x in (abs(x) for x in (-1, -2, 3, 4))) #  
[2, 4, 6, 8]
```

尽管这三种做法的效果都是组合操作，生成器们做到这一点却不需要制造多个临时列表。事实上，下一个示例既嵌套又组合了生成器——嵌套的生成器表达式由 `map` 激活，而 `map` 又只能由 `list` 激活：

```
>>> import math  
>>> list(map(math.sqrt, (x ** 2 for x in range(4)))) #  
[0.0, 1.0, 2.0, 3.0]
```

严格来说，上面右边的 `range` 本身也是一个值生成器，由生成器表达式激活——形成三层值生成，从内到外只在请求时才产出单个值，而这一切“就是能工作”，全靠 Python 的迭代工具和协议。事实上，生成器的嵌套可以任意混搭、任意深，虽然有些组合可能比其他组合更站得住脚：

```
>>> list(map(abs, map(abs, map(abs, (-1, 0, 1))))) #  
[1, 0, 1]  
>>> list(abs(x) for x in (abs(x) for x in (abs(x) for x in (-1, 0,...  
[1, 0, 1]
```

最后这些例子展示了生成器可以多么通用，但它们也被故意编码成复杂的形式，以强调：生成器表达式与前面讨论过的列表推导式一样有被滥用的可能——照例，你应该让它们保持简单，除非它们必须复杂。不过，用得好的时候，生成器表达式把列表推导式的表现力与其他可迭代对象的空间、时间优势结合起来。例如，下面这些非嵌套的替代方案为前

面三个清单的嵌套写法提供了更简单的解法，同时仍然利用了生成器的长处：

```
>>> list(abs(x) * 2 for x in (-1, -2, 3, 4)) #  
[2, 4, 6, 8]  
>>> list(math.sqrt(x ** 2) for x in range(4)) #  
[0.0, 1.0, 2.0, 3.0]  
>>> list(abs(x) for x in (-1, 0, 1))  
[1, 0, 1]
```

代码分析

·嵌套生成器的执行时序：list(map(math.sqrt, (x² for x in range(4)))) 中，list 驱动 map，map 向最内层生成器 next() 要一个值，生成器才向 range 要一个整数、算出平方、交回 map，map 再开方，list 收下。整条流水线"拉"着走（pull-driven），每时刻内存里只有一个整数在流动。

·abs(abs(abs(-1))) 式嵌套是"同一个函数反复套自己"，结果总是 1, 0, 1——作者故意用它演示"嵌套合法但没意义"。

·list(x² for x in (abs(x) for x in ...)) 比 map 嵌套更可读：for 在括号内分层展开，人脑按"外层 for 每取一项，内层 for 已备好"的顺序理解。

·设计思想：map 要求"先有函数"，而生成器表达式允许"当场写表达式"。操作是非函数调用（如 x²、abs(x)）时，map 必须伪造 lambda，生成器表达式则直写表达式——这是它"编码优势"的根源。

·初学者误区：以为嵌套生成器会"先算完内层再算外层"。

正好相反——除非被消费，内层根本不启动。三层生成器在一轮 `next()` 里层层唤醒，一次只产一个数。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.14 Generator Expressions Versus filter (生成器表达式与 filter 的对比)

英文原文

Generator expressions also support all the usual list comprehension syntax—including if clauses, which work like the filter call we met earlier. Because filter is an iterable that generates its results on request, a generator expression with an if clause is operationally equivalent. Again, the join in the following is an iteratio

n tool that suffices to force all forms to produce their results:

```
> >> line = 'aa bbb c'>> ''.join(x for x in line.split() if len(x) > 1)

'aabbb'
> >> ''.join(filter(lambda x: len(x) > 1, line.split())) # Equivalent

'aabbb'
```

The generator seems just marginally simpler than the filter here. As for list comprehensions, though, adding processing steps to filter results requires a map too, which makes filter noticeably more complex:

```
> >> ''.join(x.upper() for x in line.split() if len(x) > 1)

'AABBB'
> >> ''.join(map(str.upper, filter(lambda x: len(x) > 1, line.split())))

'AABBB'
```

In effect, generator expressions provide more general coding structures that do not rely on functions, but still delay results production. Also like list comprehensions, there is always a statement-based equivalent to a generator expression, though it sometimes renders substantially more code:

```
> >> res = ''>> for x in line.split(): # Statement equivalent?

    if len(x) > 1:                # This is also a join
        res += x.upper()
> >> res

'AABBB'
```

In this case, though, the statement form isn't quite the same—it cannot produce items one at a time, and it's also emulating the effect of the join that forces results to be produced all at once. The true equivalent to a generator expression would be a generator function with a yield, as the next section will show.

中文翻译

生成器表达式也支持所有常规的列表推导式语法——包括 if 子句，它的工作方式就像我们前面遇到的 filter 调用。因为 filter 是一个按需生成结果的可迭代对象，所以带 if 子句的生成器表达式在操作上是等价的。同样，下面的 join 是一个足以强迫各种形式产出结果的迭代工具：

```
>>> line = 'aa bbb c'
>>> ''.join(x for x in line.split() if len(x) > 1) # if
'aabbb'
```

在这里，生成器看起来只是比 filter 略微简单。不过，和列表推导式的情况一样，要给 filter 的结果加上处理步骤，还需要一个 map，这使得 filter 明显更复杂：

```
>>> ''.join(x.upper() for x in line.split() if len(x) > 1)
'AABBB'
```

实际上，生成器表达式提供了更通用的编码结构——不依赖函数，却仍然延迟结果的产生。与列表推导式一样，生成器表达式也总有一个基于语句的等价形式，尽管它有时渲染出多得多的代码：

```
>>> res = ''
>>> for x in line.split():      #
    if len(x) > 1:             # join
        res += x.upper()
>>> res
'AABBB'
```

不过，在这种情况下，语句形式并不完全一样——它不能一次只产出一项，而且它是在模仿那个强迫结果一次性全部产出的 join 的效果。生成器表达式的真正等价物应该是带 yield 的生成器函数，下一节将展示这一点。

代码分析

- filter 用 lambda 表达"保留条件"，if 子句直接写条件表达式——len(x) > 1 比 lambda x: len(x) > 1 少了函数包装层，这是生成器表达式在简单筛选上胜出的原因。
- "筛选 + 变换"两步走：生成器表达式一行 x.upper() for x in ... if ... 同时完成 filter 与 map；而 filter + map 得写成 map(str.upper, filter(...)) 的嵌套调用，可读性陡降。
- 执行轨迹：join 对生成器反复 next()；每轮生成器取一个词，先判 len(x) > 1，不过滤则直接跳过（join 完全不知道这个词存在过），通过才 upper() 后交出去。过滤发生在生成器内部，从源头避免无效词进入 join。
- 语句版 res += x.upper() 是"边遍历边拼字符串"，每次 += 都会创建新字符串（字符串不可变），效率低；生成器版则把"产出"与"消费"解耦。
- 设计思想：推导式语法的 if 天然内建"守卫 (guard)"语

义，等价于函数式语言中的 filter，但不需要匿名函数。用"表达式"取代"函数"，是把 Lisp 式工具翻译成 Python 更自然的方言。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.15 Generator Functions Versus Generator Expressions (生成器函数与生成器表达式对比)

英文原文

Let's recap what we've covered so far in this section: Generator functions — A function def statement that contains a yield statement is turned into a generator function. When called, it returns a new generator obj

ect with automatic retention of local scope and code position; an automatically created iter method that simply returns itself; and an automatically created next method that starts the function or resumes it where it last left off, and raises StopIteration when finished producing results.

Generator expressions — A comprehension expression enclosed in parentheses is known as a generator expression. When run, it returns a new generator object with the same automatically created method interface and state retention as a generator function call's results—with an iter method that simply returns itself; and a next method that starts the implied loop or resumes it where it last left off, and raises StopIteration when finished producing results.

The net effect is to automatically produce results on demand in all iteration tools that employ either of these.

As implied by preceding coverage, the same iteration can often be coded with either a generator function or a generator expression. The following generator expression, for example, repeats each character in a string four times:

```
> >> G = (c * 4 for c in 'hack') # Generator expression >> list...  
['hhhh', 'aaaa', 'cccc', 'kkkk']
```

The equivalent generator function requires slightly m

ore code, but its multiple-statement block will be able to code more logic and use more state information if needed. In fact, this is essentially the same as the prior chapter's trade-off between lambda and def—expression conciseness versus statement power:

```
> >> def timesfour(S): # Generator function

    for c in S:
        yield c * 4
> >> G = timesfour('hack') >> list(G) # Iterate automatically

['hhhh', 'aaaa', 'cccc', 'kkkk']
```

To their clients, the two are more similar than different. For instance, both expressions and functions support both automatic and manual iteration—the prior list call iterates automatically, and the following iterate manually:

```
> >> G = (c * 4 for c in 'hack') >> I = iter(G) # Iterate manually
'hhhh'
> >> next(I)

'aaaa'
> >> G = timesfour('hack') >> I = iter(G) # Iterate manually (function)
'hhhh'
> >> next(I)

'aaaa'
```

In either case, Python automatically creates a generator object, which has both the methods required by the

e iteration protocol, and state retention for variables in the generator's code and its current code location. Notice how we make new generators here to iterate again—as explained in the next section, generators are single-pass iterators.

First, though, here's the true statement-based equivalent of the expression at the end of the prior section: a function that yields values—though the difference is irrelevant if the code using it produces all results with a tool like join:

```
> >> line = 'aa bbb c'>> ''.join(x.upper() for x in line.split(...  
  
'AABBB'  
> >> def gensub(line): # Function  
  
    for x in line.split():  
        if len(x) > 1:  
            yield x.upper()  
> >> ''.join(gensub(line)) # But why generate?  
  
'AABBB'
```

While generators have valid roles, in cases like this the use of generators over the simple statement equivalent shown earlier may be difficult to justify, except on stylistic grounds: if you're just going to immediately join generated results anyhow, you might as well skip generators and use simple loops. On the other hand, trading four lines for the generator expression's one may to many seem fairly compelling stylistic grounds!

中文翻译

让我们回顾一下本小节迄今覆盖的内容：生成器函数（Generator functions）——包含 `yield` 语句的函数 `def` 会被改造成生成器函数。调用时，它返回一个新的生成器对象（generator object），自动保留局部作用域和代码位置；自动创建一个只返回自身的 `iter` 方法；自动创建一个 `next` 方法，启动函数或从上次离开处恢复它，并在结果生产完毕时抛出 `StopIteration`。生成器表达式（Generator expressions）——用圆括号包围的推导式表达式称为生成器表达式。运行时，它返回一个新的生成器对象，具备与生成器函数调用结果相同的自动创建的方法接口和状态保持——`iter` 方法只返回自身；`next` 方法启动隐含的循环或从上次离开处恢复它，并在结果生产完毕时抛出 `StopIteration`。最终效果是：在使用这两者之一的任何迭代工具中，结果都会自动按需产出。正如前面的介绍所暗示的，同一个迭代常常既可以用生成器函数、也可以用生成器表达式来编码。例如，下面的生成器表达式把字符串中每个字符重复四次：

```
>>> G = (c * 4 for c in 'hack') #
>>> list(G) #
['hhhh', 'aaaa', 'cccc', 'kkkk']
```

等价的生成器函数需要略微多一些代码，但它的多语句代码块在需要时能够编写更多逻辑、使用更多状态信息。事实上，这本质上就是上一章中 `lambda` 与 `def` 之间的取舍——表达式的简洁对语句的威力：

```
>>> def timesfour(S):                                #
        for c in S:
            yield c * 4
>>> G = timesfour('hack')
>>> list(G)                                           #
['hhhh', 'aaaa', 'cccc', 'kkkk']
```

对它们的客户来说，两者相似之处多于不同之处。例如，表达式和函数都同时支持自动迭代和手动迭代——前面的 `list` 调用是自动迭代，下面这些是手动迭代：

```
>>> G = (c * 4 for c in 'hack')
>>> I = iter(G)                                       #
>>> next(I)
'hhhh'
>>> next(I)
'aaaa'
>>> G = timesfour('hack')
>>> I = iter(G)                                       #
>>> next(I)
'hhhh'
>>> next(I)
'aaaa'
```

无论哪种情况，Python 都会自动创建一个生成器对象，它既具备迭代协议所需的方法，也为生成器代码中的变量及其当前代码位置提供状态保持。注意我们是如何制造新的生成器来再次迭代的——正如下一节所解释的，生成器是单遍（single-pass）迭代器。不过，先来看看上一节末尾那个表达式的真正基于语句的等价形式：一个产出值的函数——不过，如果使用它的代码用 `join` 之类的工具一次性地把全部结果都产出来，那么这种差异就无关紧要了：

```
>>> line = 'aa bbb c'
>>> ''.join(x.upper() for x in line.split() if len(x) > 1) #
'AABBB'
>>> def gensub(line):
    for x in line.split():
        if len(x) > 1:
            yield x.upper()
>>> ''.join(gensub(line)) #
'AABBB'
```

虽然生成器有其正当的角色，但在这样的情形下，用生成器取代前面展示的简单语句等价形式可能很难站得住脚，除非出于风格理由：如果你反正都要立刻 join 生成的结果，那不如跳过生成器，直接用简单循环。另一方面，用四行代码换取生成器表达式的一行，对很多人来说似乎也是相当有说服力的风格理由！

代码分析

- 两个方案产出的对象完全同构：(c 4 for c in 'hack') 编译后等价于一个匿名生成器函数，其内部就是 timesfour 的函数体。所以“表达式”和“函数”只是同一机制的两件外衣。
- iter(G) 在两种生成器上都是“返回自身”，next(I) 驱动同一状态机；对两者做 iter(G) is G 都会得到 True。
- 选择指南：逻辑能塞进一个表达式 → 用生成器表达式（一行）；需要多步逻辑、状态积累、或要在中间插入 try/except → 用生成器函数。
- gensub 的 join 语境下，惰性带来的收益为零（反正一次性消费），此时作者坦诚：简单循环同样合理——写作时“值得”与否要按场景判断，不要为用生成器而用生成器。

·设计思想：回顾表（recap）其实是给两种工具盖棺定论——它们共享同一协议接口、同一状态模型，区别只在于“语法容器”：一个是 `def` 块，一个是括号表达式。这也解释了为什么 `lambda` 不能 `yield`：`lambda` 是“表达式只能含表达式”的产物，而 `yield` 需要语句级的状态机语义。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.16 Generator Odds and Ends（生成器的边角话题）

英文原文

This section wraps up generators with a quick rundown of associated but lesser topics. After this, we'll mo

ve on to larger examples, but make sure you have a handle on the smaller bits before jumping ahead.

Generators are single-pass iterables (生成器是单遍可迭代对象)

First up, a subtle but important point: both generator functions and generator expressions are their own iterators and thus support just one active iteration—unlike some built-in types, you can't have multiple iterators of either generator positioned at different locations in the stream of results. Because of this, a generator's iterator is the generator itself; in fact, as suggested earlier, calling `iter` on a generator expression or function is an optional no-op:

```
> >> G = (c * 4 for c in 'hack') >> iter(G) is G # My iterator ...
```

```
True
```

If you do iterate over the results stream manually with multiple iterators, they will all point to the same position:

```
> >> G = (c * 4 for c in 'hack') # Make a new generator >> I1 = ...
```

```
'hhhh'
```

```
> >> next(I1)
```

```
'aaaa'
```

```
> >> I2 = iter(G) # Second iterator at same position! >> next(I2)
```

```
'cccc'
```

Moreover, once any iteration runs to completion, all are exhausted—we have to make a new generator to start again:

```
> >> list(I1) # Collect the rest of I1's items

['kkkk']
> >> next(I2) # Other iterators exhausted too

StopIteration
> >> I3 = iter(G) # Ditto for new iterators >> next(I3)

StopIteration
> >> I3 = iter(c * 4 for c in 'hack') # New generator to start ove...

'hhhh'
```

The same holds true for generator functions—the following `def` statement-based equivalent supports just one active iterator and is exhausted after one pass:

```
> >> def timesfour(S):

    for c in S:
        yield c * 4
> >> G = timesfour('hack') # Generator functions work the same way...

True
> >> I1, I2 = iter(G), iter(G) >> next(I1)

'hhhh'
> >> next(I1)

'aaaa'
> >> next(I2) # I2 at same position as I1

'cccc'
```

This is different from the behavior of some built-in objects like lists, which support multiple iterators and passes and even reflect their in-place changes in active iterators:

```
> >> L = [1, 2, 3, 4] >> I1, I2 = iter(L), iter(L) >> next(I1)
```

```
1
```

```
> >> next(I1)
```

```
2
```

```
> >> next(I2) # Lists support multiple iterators
```

```
1
```

```
> >> del L[2:] # Changes reflected in iterators >> next(I1)
```

```
StopIteration
```

Though not readily apparent in these simple examples, this can matter in your code: if you wish to scan a generator's values multiple times, you must either create a new generator for each scan or build a rescanable list out of its values—a single generator's values will be consumed and exhausted after a single pass. See this chapter's sidebar "Why You Will Care: Iteration Versus Python Morph" for a prime example of the sort of code impacted by this.

When we begin coding class-based iterables in Part V I, you'll also see that it's up to us to decide how many iterations we wish to support for our objects, if any. In general, objects that wish to support multiple scans will return supplemental class objects instead of the

mselves for iter. The next section previews more of the model.

Generation in built-ins and classes (内置与类中的生成)

Although we've focused on coding value generators ourselves in this section, don't forget that many built-in types behave the same way. As we saw in Chapter 14, for example, dictionaries and files generate results too:

```
for key in dictionary: ...    # See Chapter 14 et al.  
for line in file: ...
```

Though beyond this book's scope, many Python standard-library tools generate values too, including its directory walker—which at each level of a folder tree yields a tuple of the current directory, its subdirectories, and its files:

```
> >> import os >> for (root, subs, files) in os.walk('.'): # D...  
  
    for name in files:                                     # A Python 'fin...  
        if name.endswith('.py'):                         # Also os.path...  
            print(root, name)  
../Chapter02 script0.py  
../Chapter03 myfile.py  
../Chapter03 script1.py  
...  
etc  
...
```

Because the current Python-coded implementation of `os.walk` uses `yield` to return results, it's a normal ge

erator function, and hence iterable object, that generates a three-item tuple on each iteration:

```
> >> G = os.walk('.') # A single-scan iterator: iter(G) option...

('..', ['Chapter02', 'Chapter03', 'Chapter04', ... 'Chapter37'], [...
> >> next(G)

('../Chapter02', ['__pycache__', ['script0.py']])
```

By yielding results as it goes, the walker does not require its clients to wait for an entire tree to be scanned. See Python's manuals for more on this tool. For system-tools fans, also see the next chapter for an example that uses `os.popen`—a related iterable used to run a shell command and read its output. And for additional examples of built-in value generators and the iteration tools that use them, review Chapter 14.

Finally, although beyond the scope of this chapter, it is also possible to implement arbitrary user-defined generator objects with classes that conform to the iteration protocol. Such classes define `iter` and `next` methods explicitly to support the protocol:

```
class SomeIterable:
    def __iter__(...): ...    # On iter(): return self or suppleme...
    def __next__(...): ...    # On next(): coded here, or in anothe...
```

As the prior section suggested, these classes usually return their objects directly for single-iteration behavior, or a supplemental object with scan-specific state for multiple-scan support.

Alternatively, a user-defined iterable class's `iter` may use `yield` to transform itself into a generator, and a `getitem` indexing method is also available as a fallback option for iteration with trade-offs we'll explore later.

However this is coded, the iterator and generator story won't really be complete until we've seen how it also maps to classes. For now, we'll have to settle for postponing its conclusion—and its final sequel—until Chapter 30.

Comprehensions versus type calls and generators (推导式与类型调用和生成器的对比)

We've also been focusing on list comprehensions and generators in this chapter, but keep in mind that there are two other comprehension expression forms: the set and dictionary comprehensions we met earlier in Chapter 14 and Part II. For reference and closure, here's a summary of all the comprehension forms in Python:

```
> >> [x * x for x in range(10)] # List comprehension: builds li...
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81] # Fixed order, made all at ...
> >> (x * x for x in range(10)) # Generator expression: yields ite...

<generator object <genexpr> at 0x100a1f920>
> >> {x * x for x in range(10)} # Set comprehension: builds set

{0, 1, 64, 4, 36, 9, 16, 49, 81, 25} # Random order, no duplicat...
> >> {x: x * x for x in range(10)} # Dict comprehension: insert or...
```

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}
```

All of these forms use the same formal comprehension syntax we met earlier, but their enclosing delimiters (and key, for dictionaries) denote their differing roles. In a sense, list, set, and dictionary comprehensions are syntactic sugar for passing generator expressions to type names. Because all accept any iterable, a generator works here too:

```
> >> list(x ** 2 for x in range(10)) # Generator plus type name
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
> >> set(x * x for x in range(10)) # Similar to a set comprehension
{0, 1, 64, 4, 36, 9, 16, 49, 81, 25}
> >> dict((x, x * x) for x in range(10))
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}
```

But don't read too much into such equivalences. Because implementations may vary arbitrarily, you should generally collect performance data before adopting an alternative. In this case, generators are actually slower than the equivalent list comprehension today, as we'll prove in the next chapter. Programs that run lots of them may need to care.

Scopes and comprehension variables (作用域与推导式变量)

Now that we've seen all comprehension forms, Chapter 17's overview of the localization of loop variables i

n these expressions may make more sense. In all forms, the loop variable (or variables) after the for is local to the expression—it won't clash with names outside, but is also not available there, and this differs from for statements:

```
> >> X = 99 >> [X for X in range(5)] # All comprehensions local...
[0, 1, 2, 3, 4]
> >> X # Enclosing-scope X was not changed
99
> >> for X in range(5): pass # But loop statements do not localize...
4
```

As noted in Chapter 17, loop variables assigned in a comprehension really live in a further nested special-case scope, but other names referenced within these expressions follow the usual LEGB rules. In the following generator, for example, Z is localized in the comprehension, but Y and X are found in the enclosing local and global scopes as usual:

```
> >> X = 'aaa' >> def func():
    Y = 'bbb'
    print('-'.join(Z for Z in X + Y)) # Z=comprehension, Y=1...
> >> func()
a-a-a-b-b-b
```

One exception here: names assigned by the `:=` expression inside a comprehension do leak out of comprehension, and generally behave as though they were as

signed in the scope containing the comprehension it self:

```
> >> S = 'hack'>> [(temp:= S[i]) + temp.upper() for i in range(...  
['hH', 'aA', 'cC', 'kK']  
> >> temp  
  
'k'  
> >> i  
  
NameError: name 'i' is not defined. Did you mean: 'id'?
```

While this allows comprehensions to both reuse and export values, keep in mind that a lambda container's local scope effectively plugs the leak—temp in the following, for example, lives only in the lambda's scope :

```
> >> del temp >> f = lambda: [(temp:= S[i]) + temp.upper() for ...  
['hH', 'aA', 'cC', 'kK']  
> >> temp  
  
NameError: name 'temp' is not defined
```

We'll code such a := nesting in a sequence-scrambler example ahead, though some observers may deem it much less transparent than simple assignment statements, and other real-world roles must await your discovery. See Chapter 11 for more background on := named assignment.

Generating "infinite" (well, indefinite) results (生成"无限" (好吧, 无定限) 结果)

Finally, it was noted earlier in passing that generators can even produce "infinite" results that tools like list comprehensions cannot. This may sound more impressive that it is; really, this just means that a generator can keep yielding results from its retained state's local variables indefinitely—until its client grows tired of them:

```
> >> def squares(n):  
  
    while True:                # Generate results "forever"  
        yield n ** 2           # Or until no more next calls...  
        n += 1  
  
> >> G = squares(2) >> next(G)  
  
4  
  
> >> next(G)  
  
9  
  
> >> for i in range(10): print(next(G), end=' ')  
  
16 25 36 49 64 81 100 121 144 169
```

This pattern may be useful in limited roles like test-parameter generation, and other tools can't compete in this event; list comprehensions, for example, must collect all results at once. In the end, though, this is a narrow role, generators are mortal, and so are we—so let's move on to some code that's a bit more tangible in the next section.

中文翻译

本节用一个快速清单收尾生成器相关的次要话题。之后我们会转向更大的示例，但在跳过去之前，请确保你掌握了这些小零件。

生成器是单遍可迭代对象 (Generators are single-pass iterables)

首先是一个微妙但重要的点：生成器函数和生成器表达式都“自己就是自己的迭代器”，因此只支持一次激活的迭代——与某些内置类型不同，你无法为任何一个生成器建立多个、各自位于结果流不同位置的迭代器。正因为如此，生成器的迭代器就是生成器本身；事实上，正如前面暗示过的，在生成器表达式或函数上调用 `iter` 是一个可选的空操作：

```
>>> G = (c * 4 for c in 'hack')
>>> iter(G) is G           # G __next__
True
```

如果你真的用多个迭代器手动遍历结果流，它们将全部指向同一个位置：

```
>>> G = (c * 4 for c in 'hack') #
>>> I1 = iter(G)                #
>>> next(I1)
'hhhh'
>>> next(I1)
'aaaa'
>>> I2 = iter(G)                #
>>> next(I2)
'cccc'
```

而且，一旦任何一次迭代跑到尽头，所有迭代器都耗尽了一——我们必须制造一个新生成器才能重新开始：

```
>>> list(I1)                                # I1
['kkkk']
>>> next(I2)                                #
StopIteration
>>> I3 = iter(G)                            #
>>> next(I3)
StopIteration
>>> I3 = iter(c * 4 for c in 'hack') #
>>> next(I3)
'hhhh'
```

生成器函数也是如此——下面这个基于 `def` 语句的等价形式只支持一个激活的迭代器，并且一次遍历后就耗尽：

```
>>> def timesfour(S):
    for c in S:
        yield c * 4
>>> G = timesfour('hack')                #
>>> iter(G) is G                          #
True
>>> I1, I2 = iter(G), iter(G)
>>> next(I1)
'hhhh'
>>> next(I1)
'aaaa'
>>> next(I2)                              # I2 I1
'cccc'
```

这与列表等某些内置对象的行为不同——列表支持多个迭代器和多次遍历，甚至在激活的迭代器中反映其就地修改：

```
>>> L = [1, 2, 3, 4]
>>> I1, I2 = iter(L), iter(L)
>>> next(I1)
1
>>> next(I1)
2
>>> next(I2)                                     #
1
>>> del L[2:]                                     #
>>> next(I1)
StopIteration
```

虽然这些简单例子中不太明显，但这在你的代码中可能很重要：如果你想多次扫描生成器的值，你必须为每次扫描创建新生成器，或用它的值构建一个可重复扫描的 list——单个生成器的值在一次遍历后就会被消费干净。参见本章边栏" Why You Will Care: Iteration Versus Python Morph"，那里有受此影响的代码的绝佳实例。当我们在第六部分开始编写基于类的可迭代对象时，你还会看到：我们的对象要支持多少次迭代（如果支持的话）由我们自己决定。一般而言，想支持多次扫描的对象会为 iter 返回补充的类对象，而不是返回自身。下一节会更多预览这种模型。

内置与类中的生成 (Generation in built-ins and classes)

虽然本节我们一直聚焦于自己编写值生成器，但别忘了许多内置类型也是同样的行为。例如，正如我们在第 14 章看到的，字典 (dictionaries) 和文件 (files) 也会生成结果：

```
for key in dictionary: ...    # 14
for line in file: ...
```

虽然超出本书范围，许多 Python 标准库工具也会生成值，包括它的目录遍历器（directory walker）——它在文件夹树的每一层产生一个元组，包含当前目录、它的子目录和它的文件：

```
>>> import os
>>> for (root, subs, files) in os.walk('.'):      #
        for name in files:                      # Python 'fi...
            if name.endswith('.py'):            # os.path.j...
                print(root, name)
../Chapter02 script0.py
../Chapter03 myfile.py
../Chapter03 script1.py
...
...
```

由于当前 `os.walk` 的 Python 实现使用 `yield` 来返回结果，它是一个普通的生成器函数，因而是可迭代对象，每次迭代产生一个三元素元组：

```
>>> G = os.walk('.')                          # iter(G)
>>> next(G)
('.', ['Chapter02', 'Chapter03', 'Chapter04', ... 'Chapter37'], [...
>>> next(G)
('../Chapter02', ['__pycache__', 'script0.py'])
```

边走边产出结果，遍历器就不要求客户等待整棵树扫描完毕。关于此工具的更多内容请见 Python 手册。系统工具爱好者还可以看下一章中使用 `os.popen` 的示例——它是一个相关的可迭代对象，用来运行 shell 命令并读取其输出。至于内置值生成器及使用它们的迭代工具的更多示例，请复习第 14 章。最后，虽然超出本章范围，用符合迭代协议的类（classes）来实现任意的用户自定义生成器对象也是可能的。这类类显式定义 `iter` 和 `next` 方法来支持协议：

```
class SomeIterable:
    def __iter__(...): ...    # iter()
    def __next__(...): ...    # next()
```

如上一节所暗示的，这些类通常为了单次迭代行为直接返回它们的对象，或为了支持多次扫描返回带扫描特定状态的补充对象。或者，用户自定义可迭代类的 `iter` 可以使用 `yield` 把自己变成生成器；此外还有一个 `getitem` 索引方法可以作为迭代的回退（fallback）选项，其中包含我们稍后要探索的权衡。无论怎样编码，直到我们看到迭代器与生成器的故事如何映射到类上，这个故事才算真正完整。眼下，我们只能把它的结论——以及它的最终续集——推迟到第 30 章。

推导式与类型调用和生成器的对比（Comprehensions v ersus type calls and generators）

本章我们一直聚焦于列表推导式和生成器，但请记住还有另外两种推导式表达式形式：我们在第 14 章和第二部分早见过的集合与字典推导式。为了参考和收尾，下面给出 Python 中所有推导式形式的总结：

```
>>> [x * x for x in range(10)]    #
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]    #
>>> (x * x for x in range(10))    #
<generator object <genexpr> at 0x100a1f920>
>>> {x * x for x in range(10)}    #
{0, 1, 64, 4, 36, 9, 16, 49, 81, 25}    #
>>> {x: x * x for x in range(10)} #
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}
```

所有这些形式都使用我们前面见过的同一套正式推导式语法，但它们的外层定界符（字典还有键）标示着各自不同的角色。从某种意义上说，列表、集合和字典推导式是“把生成

器表达式传给类型名"的语法糖（syntactic sugar）。因为所有这些类型都接受任意可迭代对象，所以生成器在这里也行：

```
>>> list(x ** 2 for x in range(10))    #
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> set(x * x for x in range(10))      #
{0, 1, 64, 4, 36, 9, 16, 49, 81, 25}
>>> dict((x, x * x) for x in range(10))
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}
```

但不要过度解读这种等价性。因为实现可能任意变化，你在采纳某种替代写法之前通常应该收集性能数据。在这里，生成器如今实际上比等价的列表推导式慢，我们将在下一章证明这一点。跑大量生成器的程序可能需要在这一点上较真。

作用域与推导式变量（Scopes and comprehension variables）

既然我们已经见识了所有推导式形式，第 17 章关于这些表达式中循环变量局部化（localization）的概述就更有意义了。在所有这些形式中，for 之后的循环变量（或变量们）是表达式局部的——它不会与外部名字冲突，但在外部也不可，这与 for 语句不同：

```
>>> X = 99
>>> [X for X in range(5)]    #
[0, 1, 2, 3, 4]
>>> X                        # X
99
>>> for X in range(5): pass  #
>>> X
4
```

正如第 17 章所述，推导式中被赋值的循环变量实际上生活在一个更深层嵌套的特殊作用域里，但这些表达式内引用的其他名字遵循通常的 LEGB 规则。例如，在下面这个生成器中，Z 在推导式中被局部化，而 Y 和 X 照常在外层局部作用域和全局作用域中找到：

```
>>> X = 'aaa'
>>> def func():
    Y = 'bbb'
    print('-'.join(Z for Z in X + Y))    # Z=Y=X=
>>> func()
a-a-a-b-b-b
```

这里有一个例外：推导式内部由 := 表达式赋值的名字会泄漏到推导式之外，行为一般就像它们是在包含该推导式的作用域中被赋值的一样：

```
>>> S = 'hack'
>>> [(temp := S[i]) + temp.upper() for i in range(len(S))]
['hH', 'aA', 'cC', 'kK']
>>> temp
'k'
>>> i
NameError: name 'i' is not defined. Did you mean: 'id'?
```

虽然这让推导式既能复用值又能导出值，但要记住：一个 lambda 容器的局部作用域可以有效地堵住这个泄漏——例如，下面代码中的 temp 只活在 lambda 的作用域里：

```
>>> del temp
>>> f = lambda: [(temp := S[i]) + temp.upper() for i in range(len(...
>>> f()
['hH', 'aA', 'cC', 'kK']
>>> temp
NameError: name 'temp' is not defined
```

稍后我们会在一个序列乱序器 (sequence-scrambler) 示例中编写这样的 `:=` 嵌套，不过有些观察者可能认为它远不如简单赋值语句透明，其他现实世界的角色就留待你自己发现了。`:=` 命名赋值的更多背景见第 11 章。

生成“无限”（好吧，无定限）结果 (Generating “infinite” results)

最后，前面顺带提到过：生成器甚至能产生列表推导式这类工具无法产生的“无限”结果。这可能听起来比实际更惊人；其实，它只是意味着一个生成器可以无限期地从其保留状态的局部变量中不断产出结果——直到它的客户对它失去兴趣：

```
>>> def squares(n):
    while True:
        yield n ** 2
        n += 1
    # ""
    # next .....

>>> G = squares(2)
>>> next(G)
4
>>> next(G)
9
>>> for i in range(10): print(next(G), end=' ')
16 25 36 49 64 81 100 121 144 169
```

这种模式在测试参数生成等有限角色中可能很有用，其他工具在这个项目上无法匹敌；例如，列表推导式必须一次性收集全部结果。但说到底，这是一个狭窄的角色，生成器是有死的，我们也是有死的——所以让我们继续，在下一节看一些更实在的代码。

代码分析

·单遍性：生成器的状态（当前索引、已消费位置）就存在生成器对象自己身上；`iter(G)` 返回自身，所以两个“迭代器”其实是同一个对象的两张身份证，共享同一份状态——不存在“第二个独立游标”。列表则不同：`iter(L)` 每次都返回一个独立的新列表迭代器，各自持有自己的下标。

·执行轨迹演示：`I1` 取走‘hhhh’、‘aaaa’后，`I2` 只能从当前位置（‘cccc’）开始；`list(I1)` 把剩余耗尽后，`I2`、`I3` 全部 `StopIteration`——“一损俱损”。

·`del L[2:]` 在迭代器存活期间删除元素，`I1` 下一次 `next()` 直接抛 `StopIteration`（长度不够了）——列表迭代器是按索引“live”读取的。

·`os.walk` 是生成器：CPython 标准库里 `os.walk` 的实现就是一个 `def + yield` 的生成器函数，每轮 `yield` 一个 `(root, dirs, files)` 三元组。你不需要等整棵树扫完就能开始处理第一个目录——这正是惰性的价值。

·四种推导式的对比一目了然：方括号→列表（保序、一次性）；圆括号→生成器（惰性）；花括号单项→集合（无序、去重）；花括号带冒号→字典（插入序）。`list()/set()/dict()` 加生成器能重现同样的结果，证明“推导式 = 类型名 + 生成器”这个等价关系。

·`X = 99; [X for X in range(5)]`：推导式结束后 `X` 仍是 99；而 `for X in range(5)` 结束后 `X` 是 4。底层实现：推导式的循环变量被放进了编译期单独生成的一个“推导式专用作用域”（CPython 用独立的 `code object` 隔离），循环

语句则直接复用当前函数作用域。

- `temp := S[i]`：海象运算符（walrus）的赋值目标是推导式所在作用域，所以 `temp` 泄漏出来（值为 `'k'`）；普通循环变量 `i` 则不泄漏。用 `lambda` 包一层后，`temp` 被关进 `lambda` 的局部作用域。

- 无限生成器：`squares` 只有状态（`n` 的当前值）与代码位置，没有任何“终局条件”——只要 `next()` 就一直有值。列表推导式无从谈起，因为它必须先知道“全部”结果。这就是“无限”与“一次性建表”的分水岭。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。

- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。

- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。

- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。

- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.17 Example: Shuffling Sequences（示例：打乱序列）

英文原文

To demonstrate the power of iteration and generation tools in action, let's turn to some more complete examples. In Chapter 18, we wrote a testing function, based on earlier code in Chapter 13, that scrambled the order of arguments used to verify generalized intersection and union functions. There, it was noted that this might be better coded as a generator of values. Now that we've learned how to write generators, this serves to illustrate a practical application.

One note up front: because they slice and concatenate objects, all the examples in the section (including the permutations at the end) work only on sequences like strings and lists, not on arbitrary iterables like files, maps, and other generators. That is, some of these examples will be generators themselves, producing values on request, but they cannot process generators as their inputs. Generalization for broader categories is left as an open issue, though the code here will suffice unchanged if you wrap nonsequence generators in list calls before passing them in.

Scrambling Sequences (打乱序列)

First, let's review. As coded in Chapter 18, we can reorder a sequence with slicing and concatenation, moving the front item to the end on each loop; slicing instead of indexing the item allows `+` to work for arbitrary

y sequence types:

```
> >> L, S = [1, 2, 3], 'code'>> for i in range(len(S)): # Codin...

    S = S[1:] + S[:1]          # Move front item to the end
    print(S, end=' ')
odec deco ecod code
> >> for i in range(len(L)):

    L = L[1:] + L[:1]          # Slice so any sequence type works
    print(L, end=' ')
[2, 3, 1] [3, 1, 2] [1, 2, 3]
```

Alternatively, as we also saw in Chapter 13, we get the same results by moving an entire front section to the end, though the order of the results varies slightly:

```
> >> for i in range(len(S)):

    X = S[i:] + S[:i]          # Coding 2: for positions 0..3
    print(X, end=' ')          # Rear part + front part (same ef...
code odec deco ecod
```

Trace this code to see how it works; each version produces N results for an N-item sequence passed in.

Simple functions (简单函数)

As is, this code works on specific named variables only, and simply prints its result. As we've seen, it's easy to generalize this by turning it into a normal function that can both work on any object passed to its argument and return its result for arbitrary use. The following does so for the second coding alternative; since its first cut exhibits the classic list comprehension pattern

ern, we can also save some work by coding it as such in the second:

```
> >> def scramble(seq):

    res = []
    for i in range(len(seq)):
        res.append(seq[i:] + seq[:i])
    return res

> >> scramble('code')

['code', 'odec', 'deco', 'ecod']
> >> def scramble(seq):

    return [seq[i:] + seq[:i] for i in range(len(seq))]
> >> scramble('code')

['code', 'odec', 'deco', 'ecod']
> >> for x in scramble((1, 2, 3)):

    print(x, end=' ')

(1, 2, 3) (2, 3, 1) (3, 1, 2)
```

We could use recursion here as well, but it's probably overkill in this fixed and linear context.

Generator functions (生成器函数)

The preceding section's simple approach works, but must build an entire result list in memory all at once (not great on memory usage if it's massive), and requires the caller to wait until the entire list is complete (less than ideal if this takes a substantial amount of time). You should know by now that we can do better on both fronts by translating this to a generator func

tion that yields one result at a time, using either coding scheme:

```
> >> def scramble(seq):

    for i in range(len(seq)):
        seq = seq[1:] + seq[:1] # Generator function, coding 1
        yield seq                # Assignments work here

> >> def scramble(seq):

    for i in range(len(seq)):
        yield seq[i:] + seq[:i] # Generator function, coding 2
                                # Yield one item per iteration
```

Both of these alternatives produce values when used by an iteration tool like `list` or `for`, though the second version's results order (shown here) again varies slightly in ways that probably won't matter to future clients:

```
> >> list(scramble('code')) # list() generates all results

['code', 'odec', 'deco', 'ecod']
> >> list(scramble((1, 2, 3))) # Any sequence type works

[(1, 2, 3), (2, 3, 1), (3, 1, 2)]
> >> for x in scramble((1, 2, 3)): # for loops generate results

    print(x, end=' ')

(1, 2, 3) (2, 3, 1) (3, 1, 2)
```

Both generator functions retain their local scope state (`seq` and `i`) while active, minimize memory space requirements, and divide the work into shorter time slices. As full functions, they are also very general. Moreover, because iteration tools work the same whether

stepping through a real list or a generator of values, the function can select between the two codings freely, and even change strategies in the future without impacting callers.

Generator expressions (生成器表达式)

As we've also seen, generator expressions—comprehensions in parentheses instead of square brackets—also generate values on request and retain their local state. As expressions, they're not as flexible as full functions, but because they yield their values automatically, generator expressions can often be more concise in specific use cases like this:

```
> >> S = 'code'>> G = (S[i:] + S[:i] for i in range(len(S))) # ...  
['code', 'odec', 'deco', 'ecod']
```

A generator expression can't use the seq assignment statement of the first generator-function coding, but it can achieve the same effect by nesting the := named-assignment expression covered in Chapter 11, because := both changes assigned names in the containing scope and returns the assigned value so it appears in the results stream:

```
> >> S = 'code'>> G = (S:=(S[1:] + S[:1]) for i in range(len(S)) ...  
['odec', 'deco', 'ecod', 'code']
```

Either way, we're still operating on a specific variable here, S. To generalize a generator expression for an a

bitrary subject, wrap it in a simple lambda function that takes an argument and returns a generator that uses it (subtle bit: note that seq in the second of these is not local to the generator, but is local to the lambda that encloses it):

```
> >> F = lambda seq: (seq[i:] + seq[:i] for i in range(len(seq)...

['code', 'odec', 'deco', 'ecod']
> >> F = lambda seq: (seq:=(seq[1:] + seq[:1]) for i in range(len(...

['odec', 'deco', 'ecod', 'code']
> >> list(F([1, 2, 3]))

[[2, 3, 1], [3, 1, 2], [1, 2, 3]]
> >> F(S)

<generator object <lambda>.<locals>.<genexpr> at 0x100b23300>
> >> for x in F((1, 2, 3)): print(x, end=' ')

(2, 3, 1) (3, 1, 2) (1, 2, 3)
```

Tester client (测试客户端)

Finally, we can use either the generator function or its expression equivalent in Chapter 18's tester to produce scrambled arguments. As packaged in the module of Example 20-1, the sequence scramblers become reusable tools.

Example 20-1. scramble.py

```
"Generate shuffles of a sequence, by function or expression"
```

```
def scramble1(seq):
```

```
    for i in range(len(seq)):
```

```
        yield seq[i:] + seq[:i]      # Yield one shuffle per itera...
```

```
scramble2 = lambda seq: (seq[i:] + seq[:i] for i in range(len(seq))...
```

Though it requires a bit of page flipping to see how, moving the values generation out to an external tool like this also makes the testing function noticeably simpler:

```
> >> from scramble import scramble1 # Choose your scrambler >> ...
```

```
    for args in scramble1(items):      # Use either generator
```

```
        if trace: print(args)
```

```
        print(sorted(func(*args)))    # Test for this scramble
```

```
> >> tester(intersect, ('aab', 'abcde', 'ababab'), False)
```

```
['a', 'b']
```

```
['a', 'b']
```

```
['a', 'b']
```

To make this work for yourself, make sure all imported files are in the current directory: either copy `inter2.py` from Chapter 18's folder to Chapter 20's, or copy Example 20-1's `scramble.py` to Chapter 18's folder and work there (the examples package has already done the former). Alternatively, you can modify the module import search path's `PYTHONPATH` setting to include any folder—as you'll learn when we cover modules in full after the next chapter.

Permutating Sequences (排列序列)

Generators have many other real-world applications—consider parsing attachments in an email message or computing points to be plotted in a GUI. Moreover, other types of sequence scrambles serve central roles in other applications, from searches to mathematics. As is, our sequence scrambler of the prior section is a simple reordering, but some programs warrant the more exhaustive set of all possible orderings we get from permutations—produced using recursive functions in both list-builder and generator forms by the module file in Example 20-2.

Example 20-2. permute.py

```
"Permute sequences: as a list or generator of values"
def permute1(seq):
    if not seq:
        return [seq]
    else:
        res = []
        for i in range(len(seq)):
            rest = seq[:i] + seq[i+1:]
            for x in permute1(rest):
                res.append(seq[i:i+1] + x)
        return res
def permute2(seq):
    if not seq:
        yield seq
    else:
        for i in range(len(seq)):
            rest = seq[:i] + seq[i+1:]
            for x in permute2(rest):
```

Both of these functions produce the same results, though the second defers much of its work until it is asked for a result. This code is a bit advanced, especially the second of these functions (and to some Python newcomers might even be categorized as cruel and unusual punishment!). Still, as you'll learn in a moment, there are cases where the generator approach can be very useful—and even essential.

Study and test this code for more insight, and add prints to trace if it helps. If it's still a mystery, try to make sense of the first version first; remember that generator functions simply return objects with methods that handle next operations run by for loops at each level, and don't produce any results until iterated; and trace through some of the following examples to see how they're handled by this code.

Permutations produce more orderings than the original scrambler—for N items, we get $N!$ (factorial, not surprise) results instead of just N (24 for 4: 4 3 2 1). In fact, that's why we need recursion here: the number of nested loops is arbitrary, and depends on the length of the sequence permuted. Here's a demo of both shufflers at work:

```

> >> from scramble import scramble1 >> from permute import perm...

['abc', 'bca', 'cab']
> >> permute1('abc') # Permutations larger: N! = 6

['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
> >> list(permute2('abc')) # Generate all ordered combos

['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
> >> G = permute2('abc') # Iterate: iter() not needed >> next(G)

'abc'
> >> next(G)

'acb'
> >> for x in permute2('abc'): print(x) # Automatic iteration

...
prints six lines
...

```

The list and generator versions' results are the same, though the generator minimizes both space usage and delays for results. For larger items, the set of all permutations from both is much larger than the simpler scrambler's:

```

> >> permute1('hack') == list(permute2('hack'))

True
> >> len(list(permute2('hack'))), len(list(scramble1('hack'))))

(24, 4)
> >> list(scramble1('hack'))

['hack', 'ackh', 'ckha', 'khac']
> >> list(permute2('hack'))

```

```
['hack', 'hac', 'hcak', 'hcka', 'hkac', 'hkca', 'ahck', 'ahkc', '...  
'akhc', 'akch', 'chak', 'chka', 'cahk', 'cakh', 'ckha', 'ckah', '...  
'kahc', 'kach', 'kcha', 'kcah']
```

Per Chapter 19, there are nonrecursive alternatives here too, using explicit stacks or queues, and other sequence orderings are common (e.g., fixed-size subsets and combinations that filter out duplicates of differing order), but these require coding extensions we'll forgo here. Experiment further on your own for more insights.

Why generators here: Space, time, and more (这里为什么用生成器：空间、时间，以及更多)

Generators are a somewhat advanced tool, and might be better treated as an optional topic, but for the fact that they permeate the Python language today. As we've seen, fundamental built-in tools such as `range`, `map`, dictionary keys, and even files are now generators, so you must be familiar with the concept even if you don't write new generators of your own. Moreover, user-defined generators are increasingly common in Python code that you might come across today—in the Python standard library, for instance.

Though your code is yours to code, the same guidelines given for list comprehensions apply here as well: don't complicate your code with user-defined generators if they are not warranted. Especially for smaller p

programs and data sets, such tools may not make sense. Simple lists of results may suffice, may be easier to understand, will be garbage-collected automatically, and might be produced quicker (and are today: see the next chapter). Advanced tools like generators that rely on implicit "magic" can be fun to experiment with, but may be subpar when optional.

That being said, there are specific use cases that generators can address well. They can reduce memory footprint in some programs, reduce delays in others, and can occasionally make the impossible possible. Consider, for example, a program that must produce all possible permutations of a nontrivial sequence. Since the number of combinations is a factorial that explodes exponentially, the preceding `permute1` recursive list-builder function will either introduce a noticeable and perhaps interminable pause, or fail completely due to memory requirements.

By contrast, the `permute2` recursive generator returns each individual result quickly, and can handle very large result sets. Even at just 10 items, for instance, the difference starts becoming stark (per Python's `math` module):

```
> >> import math >> f'{math.factorial(10):,}' # 10 * 9 * 8 * 7 ...
'3,628,800'
> >> from permute import permute1, permute2 >> seq = list(range(10...
3628800
> >> p1[0], p1[-1]
([0, 1, 2, 3, 4, 5, 6, 7, 8, 9], [9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
```

In this case, the `permute1` list builder pauses for 17 seconds to build a 3.6-million-item list, but the generator can begin returning individual results immediately—neither `permute2` nor any next pause in the following:

```
> >> p2 = permute2(seq) # Returns generator immediately >> next...
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
> >> next(p2)
[0, 1, 2, 3, 4, 5, 6, 7, 9, 8]
```

While collecting all results from the generator is still slow, it's faster than the list builder, and not intended usage:

```
> >> p2 = list(permute2(seq)) # About 10 seconds, though still ...
True
```

Naturally, we might be able to optimize the list builder's code to run quicker (e.g., an explicit stack instead of recursion might change its performance), but for larger sequences, it's not an option at all—at just 50 items, the number of permutations wholly precludes b

uilding a results list, and would take far too long for mere mortals like us in any event (and larger values will overflow the preset recursion stack depth limit: see the preceding chapter). The generator, however, is still viable—despite the factorial size of the problem, it is able to produce individual results immediately:

```
> >> math.factorial(50)

304140932017133780436126081660647688443776415689605120000000000000

> >> p3 = permute2(list(range(50))) # permute1 is not an option he...

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18,...
 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 3...
 44, 45, 46, 47, 48, 49]
```

For more fun—and to yield results that are more variable and less obviously deterministic—we could use Python's random module of Chapter 5 to randomly shuffle the sequence to be permuted before the permuter begins its work. In the following, for example, each permute2 and next call returns immediately as before and permutes a random sequence, but permute1 hangs (and presumably perishes from memory starvation if allowed to run long enough):

```

> >> seq = list(range(20)) >> print(f'{math.factorial(20):,}') ...
2,432,902,008,176,640,000
> >> import random >> random.shuffle(seq) # Shuffle sequence random...

[10, 19, 5, 6, 2, 13, 1, 8, 11, 7, 14, 16, 4, 3, 0, 18, 9, 12, 17,...
> >> next(p)

[10, 19, 5, 6, 2, 13, 1, 8, 11, 7, 14, 16, 4, 3, 0, 18, 9, 12, 15,...
> >> random.shuffle(seq) >> p = permute2(seq) >> next(p)

[17, 15, 14, 7, 10, 8, 2, 6, 18, 19, 13, 4, 1, 12, 5, 0, 3, 9, 16,...
> >> next(p)

[17, 15, 14, 7, 10, 8, 2, 6, 18, 19, 13, 4, 1, 12, 5, 0, 3, 9, 11,...

```

In fact, we might be able to use the random shuffler instead of manual shuffles in some roles, as long as we either can assume that it won't repeat shuffles during the time we consume them, or test its results against prior shuffles to avoid repeats—and hope that we do not live in the strange universe where a random sequence repeats the same result an infinite number of times! When full coverage is important, manual shuffles offer better control.

The main point here is that generators can sometimes produce results from large solution sets when list builders cannot. Then again, it's not clear how common such use cases may be in the real world, and this doesn't necessarily justify the implicit flavor of value generation that we get with generator functions and expressions. As you'll see in Part VI, value generation c

an also be coded as iterable objects with classes. Class-based iterables can produce items on request too, and are more explicit than the magic objects and methods produced for generator functions and expressions.

Part of programming is finding a balance among trade-offs like these, and there are no absolute rules here. While the benefits of generators may sometimes justify their use, maintainability counts too. Like comprehensions, generators also offer an expressiveness and code economy that's hard to resist if you understand how they work—but you'll want to weigh this against the frustration of coworkers who might not.

中文翻译

为了展示迭代与生成工具实际应用中的威力，让我们看一些更完整的示例。在第 18 章，我们基于第 13 章更早的代码，写了一个用于验证广义交集（intersection）和并集（union）函数的测试函数，它会打乱用于测试的参数顺序。当时我们提到，这也许更适合写成值的生成器。现在既然我们学会了写生成器，正好用它来说明一个实际应用。先说一个注意事项：由于这些示例要切片（slice）和拼接（concatenate）对象，本节所有例子（包括最后的排列）只适用于字符串、列表这样的序列（sequences），不适用于文件、map、其他生成器这样的任意可迭代对象（iterables）。也就是说，这些示例中有些本身是生成器，按需产出值，但它们无法把生成器当作输入处理。向更宽类别的推广留作开放问题——不过，只要你在传入之前用 list 调用把非序列生

成器包一层，这里的代码原样就够用。

打乱序列 (Scrambling Sequences)

首先来复习一下。如第 18 章所写，我们可以用切片和拼接来重排一个序列，每次循环把最前面的项移到末尾；用切片而非下标索引取出该项，能让 + 对任意序列类型都有效：

```
>>> L, S = [1, 2, 3], 'code'
>>> for i in range(len(S)):      # 1 0..3
    S = S[1:] + S[:1]           #
    print(S, end=' ')
odec deco ecod code
>>> for i in range(len(L)):
    L = L[1:] + L[:1]           #
    print(L, end=' ')
[2, 3, 1] [3, 1, 2] [1, 2, 3]
```

或者，正如我们在第 13 章也见过的，把整个前段移到后面可以得到相同结果，不过结果的顺序略有差异：

```
>>> for i in range(len(S)):
    X = S[i:] + S[:i]           # 2 0..3
    print(X, end=' ')           # +
code odec deco ecod
```

跟踪这段代码看看它是如何工作的；对传入的 N 项序列，每个版本都产生 N 个结果。

简单函数 (Simple functions)

照现在这样，这段代码只对特定命名变量有效，而且只是打印结果。正如我们已经看到的，把它变成一个普通函数很容易推广——既能作用于传入参数的任意对象，又能返回结果供任意使用。下面针对第二种编码方案这样做；由于第一版呈现出经典的列表推导式模式，第二版我们也可以顺水推

舟写成列表推导式，省点事：

```
>>> def scramble(seq):
    res = []
    for i in range(len(seq)):
        res.append(seq[i:] + seq[:i])
    return res
>>> scramble('code')
['code', 'odec', 'deco', 'ecod']
>>> def scramble(seq):
    return [seq[i:] + seq[:i] for i in range(len(seq))]
>>> scramble('code')
['code', 'odec', 'deco', 'ecod']
>>> for x in scramble((1, 2, 3)):
    print(x, end=' ')
(1, 2, 3) (2, 3, 1) (3, 1, 2)
```

这里我们也可以用递归，但在这个固定且线性的语境中，递归大概算过度设计。

生成器函数 (Generator functions)

上一节的简单方法能工作，但必须一次性地在内存中构建整个结果列表（如果结果巨大，内存占用就不妙），而且要求调用方等待整个列表完成（如果这要花相当长时间，就不太理想）。你现在应该已经知道，我们可以在两方面都做得更好——把它改写成一次只产生一个结果的生成器函数，用任何一种编码方案都行：

```
>>> def scramble(seq):
    for i in range(len(seq)):
        seq = seq[1:] + seq[:1] # 1
        yield seq                #
>>> def scramble(seq):
    for i in range(len(seq)):
        yield seq[i:] + seq[:i] # 2
                                   #
```

这两个版本在被 list 或 for 之类的迭代工具使用时都会产生值，不过第二个版本的结果顺序（如下所示）又略有差异，这种差异大概不会影响将来的客户：

```
>>> list(scramble('code'))          # list()
['code', 'odec', 'deco', 'ecod']
>>> list(scramble((1, 2, 3)))      #
[(1, 2, 3), (2, 3, 1), (3, 1, 2)]
>>> for x in scramble((1, 2, 3)): # for
    print(x, end=' ')
(1, 2, 3) (2, 3, 1) (3, 1, 2)
```

两个生成器函数在激活期间都保留它们的局部作用域状态（seq 和 i），把内存空间需求降到最低，并把工作切分成更短的时间片。作为完整函数，它们也非常通用。此外，因为迭代工具不管是遍历真实列表还是值的生成器都一视同仁，函数可以在两种编码之间自由选择，甚至将来更换策略也不会影响调用方。

生成器表达式（Generator expressions）

正如我们同样看到的，生成器表达式——用圆括号代替方括号的推导式——也是按需生成值并保留局部状态。作为表达式，它们不如完整函数灵活，但由于它们自动产出值，在这种特定用例下生成器表达式常常可以更简洁：

```
>>> S = 'code'
>>> G = (S[i:] + S[:i] for i in range(len(S))) # 2
>>> list(G)
['code', 'odec', 'deco', 'ecod']
```

生成器表达式不能使用第一种生成器函数编码里的 `seq` 赋值语句，但通过嵌套第 11 章讲过的 `:=` 命名赋值表达式，它可以达到同样的效果——因为 `:=` 既会改变包含作用域中被赋名的名字，又返回被赋的值，使它出现在结果流中：

```
>>> S = 'code'
>>> G = (S:=(S[1:] + S[:1]) for i in range(len(S))) # 1
>>> list(G)
['odec', 'deco', 'ecod', 'code']
```

不管哪种方式，这里我们仍然操作的是一个特定变量 `S`。要把生成器表达式推广到任意对象，把它包在一个接受参数、返回使用该参数的生成器的简单 `lambda` 函数里即可（微妙之处：注意第二个版本里的 `seq` 并不是生成器局部的，而是包围它的 `lambda` 局部的）：

```
>>> F = lambda seq: (seq[i:] + seq[:i] for i in range(len(seq)))
>>> list(F(S))
['code', 'odec', 'deco', 'ecod']
>>> F = lambda seq: (seq:=(seq[1:] + seq[:1]) for i in range(len(s...
>>> list(F(S))
['odec', 'deco', 'ecod', 'code']
>>> list(F([1, 2, 3]))
[[2, 3, 1], [3, 1, 2], [1, 2, 3]]
>>> F(S)
<generator object <lambda>.<locals>.<genexpr> at 0x100b23300>
>>> for x in F((1, 2, 3)): print(x, end=' ')
(2, 3, 1) (3, 1, 2) (1, 2, 3)
```

测试客户端 (Tester client)

最后，我们可以用生成器函数或它的表达式等价形式，在第 18 章的 `tester`（测试器）里产生打乱参数。按照示例 20-1 的模块封装，序列打乱器变成了可复用的工具。

示例 20-1: `scramble.py`

```
"Generate shuffles of a sequence, by function or expression"
def scramble1(seq):
    for i in range(len(seq)):
        yield seq[i:] + seq[:i]      #
scramble2 = lambda seq: (seq[i:] + seq[:i] for i in range(len(seq)))
```

虽然需要翻几页书才能看清原理，但像这样把值的生成挪到外部工具中，也让测试函数明显变简单了：

```
>>> from scramble import scramble1 #
>>> from inter2 import intersect, union #
>>> def tester(func, items, trace=True):
        for args in scramble1(items):    #
            if trace: print(args)
            print(sorted(func(*args)))    #
>>> tester(intersect, ('aab', 'abcde', 'ababab'), False)
['a', 'b']
['a', 'b']
['a', 'b']
```

要自己运行它，请确保所有被导入的文件都在当前目录：要么把第 18 章文件夹里的 `inter2.py` 复制到第 20 章文件夹，要么把示例 20-1 的 `scramble.py` 复制到第 18 章文件夹并在那边工作（示例包已经做了前者）。或者，你可以修改模块导入搜索路径的 `PYTHONPATH` 设置来包含任何文件夹——这是我们在下一章之后全面讲解模块时才会学到的内容。

排列序列 (Permutating Sequences)

生成器还有很多其他现实世界应用——想想解析电子邮件中的附件，或者计算要在 GUI 中绘制的点。此外，其他类型的序列乱序在其他应用中扮演核心角色，从搜索到数学。就目前而言，上一节的序列打乱器只是一个简单重排；但有些程序需要更彻底的全排列 (permutations) 结果集——示例 20-2 的模块文件用递归函数以列表构建器和生成器两种形式实现了它。

示例 20-2: permute.py

```
"Permute sequences: as a list or generator of values"
def permute1(seq):
    if not seq:
        #
        return [seq]
    #
    else:
        res = []
        for i in range(len(seq)):
            rest = seq[:i] + seq[i+1:]
            #
            for x in permute1(rest):
                #
                res.append(seq[i:i+1] + x)
            #
        return res

def permute2(seq):
    if not seq:
        #
        yield seq
    #
    else:
        for i in range(len(seq)):
            rest = seq[:i] + seq[i+1:]
            #
            for x in permute2(rest):
                #
                yield seq[i:i+1] + x
            #
```

这两个函数产生相同的结果，不过第二个把大量工作推迟到被索要结果时才做。这段代码有点高级，尤其是第二个函数

(对某些 Python 新手来说, 甚至可能被归类为"残忍而不寻常的惩罚"!)。不过, 正如你马上会学到的, 有些场景下生成器方法非常有用——甚至不可或缺。为了更多洞见, 请研究并测试这段代码, 如果对你有帮助就加些 print 跟踪。如果它仍然是个谜, 先试着搞懂第一个版本; 记住: 生成器函数只是返回带有方法的对象, 这些方法处理每一层 for 循环运行的 next 操作, 在迭代之前不产生任何结果; 并跟踪下面一些示例, 看看这段代码是怎么处理它们的。排列产生的顺序比原始打乱器多得多——对 N 项, 我们得到 $N!$ (阶乘, 不是惊讶) 个结果, 而不只是 N 个 (4 项就是 24 个: 4 3 2 1)。事实上, 这正是我们需要递归的原因: 嵌套循环的数量是任意的, 取决于被排列序列的长度。下面演示两个乱序器的工作:

```
>>> from scramble import scramble1
>>> from permute import permute1, permute2
>>> list(scramble1('abc'))      # N = 3
['abc', 'bca', 'cab']
>>> permute1('abc')             # N! = 6
['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
>>> list(permute2('abc'))       #
['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
>>> G = permute2('abc')         # iter()
>>> next(G)
'abc'
>>> next(G)
'acb'
>>> for x in permute2('abc'): print(x) #
...
...
```

列表版本和生成器版本的结果相同, 不过生成器把空间占用和结果延迟都降到了最低。对更大的项, 两者产生的全排列

集合都比简单打乱器大得多：

```
>>> permute1('hack') == list(permute2('hack'))
True
>>> len(list(permute2('hack'))), len(list(scramble1('hack')))
(24, 4)
>>> list(scramble1('hack'))
['hack', 'ackh', 'ckha', 'khac']
>>> list(permute2('hack'))
['hack', 'hac k', 'hcak', 'hcka', 'hkac', 'hkca', 'ahck', 'ahkc', '...
'akhc', 'akch', 'chak', 'chka', 'cahk', 'cakh', 'ckha', 'ckah', '...
'kahc', 'kach', 'kcha', 'kcah']
```

按照第 19 章，这里也有非递归的替代方案——使用显式栈（stacks）或队列（queues）——其他序列顺序也很常见（比如固定大小的子集、过滤掉不同顺序重复的组合），但这些需要编码扩展，我们在这里就略过了。更多洞见请自行实验。

这里为什么用生成器：空间、时间，以及更多（Why generators here: Space, time, and more）

生成器是一个有点高级的工具，本可当作选修话题处理，无奈它们今天已经渗透进整个 Python 语言。正如我们所见，range、map、字典键（keys），甚至文件这类基础内置工具现在都是生成器，所以即使你自己不写生成器，也必须熟悉这个概念。而且，用户自定义生成器在今天你能遇到的 Python 代码中越来越常见——比如在 Python 标准库里。尽管代码是你自己写的，但给列表推导式的那套准则在这里同样适用：如果并不需要，就不要用用户自定义生成器把代码搞复杂。尤其是对较小的程序和数据集合，这类工具可能没有意义。简单的结果列表可能就够用、更容易理解、会被自动垃圾回收，而且可能产生得更快（今天确实如此：见下

一章)。生成器这类依赖隐式“魔法”的高级工具，拿来实验很有意思，但在可有可无的场合可能反而平庸。话虽如此，确实有一些生成器能很好应对的特定用例。它们可以在某些程序中降低内存占用，在另一些程序中减少延迟，有时还能把不可能变成可能。例如，考虑一个必须产生某个非平凡序列全部排列的程序。由于组合数量是呈指数爆炸的阶乘，前面那个 `permute1` 递归列表构建器要么引入明显甚至似乎无尽的停顿，要么因内存需求而彻底失败。相比之下，`permute2` 递归生成器能快速返回每一个单独结果，并能处理非常庞大的结果集。比如仅仅 10 项时，差距就开始变得刺眼（按 Python 的 `math` 模块计算）：

```
>>> import math
>>> f'{math.factorial(10):,}'          # 10 * 9 * 8 * 7 * 6 * 5 * 4 *...
'3,628,800'
>>> from permute import permute1, permute2
>>> seq = list(range(10))
>>> p1 = permute1(seq)                # 2.3GHz i9 macOS 17
>>> len(p1)                          # 360
3628800
>>> p1[0], p1[-1]
([0, 1, 2, 3, 4, 5, 6, 7, 8, 9], [9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
```

这种情况下，`permute1` 列表构建器停顿 17 秒来构建一个 360 万项的列表，但生成器可以立即开始返回单个结果——下面的 `permute2` 和每一次 `next` 都不停顿：

```
>>> p2 = permute2(seq)               #
>>> next(p2)                         #
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> next(p2)
[0, 1, 2, 3, 4, 5, 6, 7, 9, 8]
```

虽然从生成器收集全部结果仍然很慢，但比列表构建器快，

而且那不是它本来的用法：

```
>>> p2 = list(permute2(seq))          # 10
>>> p1 == p2                          #
True
```

自然，我们也许能优化列表构建器的代码让它跑得更快（例如，用显式栈代替递归可能改变它的性能），但对更大的序列，它根本不是一个选项——仅仅 50 项时，排列的数量就完全排除了构建结果列表的可能，而且无论如何都会花掉我们这些凡人太久的时间（更大的值还会溢出预设的递归栈深度限制：见上一章）。然而生成器仍然可行——尽管问题规模是阶乘级的，它也能立即产生单个结果：

```
>>> math.factorial(50)
30414093201713378043612608166064768844377641568960512000000000000
>>> p3 = permute2(list(range(50)))    # permute1
>>> next(p3)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18,...
 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 3...
 44, 45, 46, 47, 48, 49]
```

为了更有趣——并产出更多变、不那么明显确定性的结果——我们可以用第 5 章的 Python random 模块，在排列器开始工作前随机打乱要排列的序列。例如在下面代码中，每次 permute2 和 next 调用都像之前一样立即返回并排列一个随机序列，而 permute1 则会挂起（如果让它跑得足够久，大概会因为内存耗尽而消亡）：

```

>>> seq = list(range(20))
>>> print(f'{math.factorial(20):,}') # permute1
2,432,902,008,176,640,000
>>> import random
>>> random.shuffle(seq) #
>>> p = permute2(seq)
>>> next(p)
[10, 19, 5, 6, 2, 13, 1, 8, 11, 7, 14, 16, 4, 3, 0, 18, 9, 12, 17,...
>>> next(p)
[10, 19, 5, 6, 2, 13, 1, 8, 11, 7, 14, 16, 4, 3, 0, 18, 9, 12, 15,...
>>> random.shuffle(seq)
>>> p = permute2(seq)
>>> next(p)
[17, 15, 14, 7, 10, 8, 2, 6, 18, 19, 13, 4, 1, 12, 5, 0, 3, 9, 16,...
>>> next(p)
[17, 15, 14, 7, 10, 8, 2, 6, 18, 19, 13, 4, 1, 12, 5, 0, 3, 9, 11,...

```

事实上，在某些角色中我们也许可以用随机打乱器代替手动打乱——只要我们或者能假定在消费这些乱序期间它不会重复，或者把它的结果与先前的乱序比对以避免重复——并祈祷我们不是生活在“随机序列无限次重复同一结果”的奇异宇宙里！当全覆盖（full coverage）很重要时，手动打乱提供更好的控制。这里的要点是：在列表构建器无能为力时，生成器有时可以从庞大的解集中产出结果。话说回来，这种用例在现实世界中有多常见并不清楚，而且这也不必然为生成器函数和表达式那种隐式（implicit）的值生成风格背书。正如你在第六部分将看到的，值生成也可以用类编码为可迭代对象。基于类的可迭代对象同样可以按需产出项，并且比生成器函数和表达式产生的魔法对象与方法更显式（explicit）。编程的一部分就是在这样的权衡中找到平衡，这里没有绝对规则。虽然生成器的好处有时能证明其使用是合理的，但可维护性同样重要。和推导式一样，生成器也提供一种表现力和代码经济性——如果你理解它们如何工作

，这很难抗拒——但你需要把它与那些可能不理解它的同事的挫败感放在天平上称一称。

代码分析

- `permute` 递归原理：`permute1('abc')` 展开后是——外层选 $i=0$ 的 'a'，对剩余 'bc' 递归得到 ['bc', 'cb']，拼成 'abc'、'acb'；再 $i=1$ 取 'b'，对 'ac' 递归……总结果 $3! = 6$ 个。递归深度等于序列长度，“嵌套循环数量任意”正是要递归的原因。

- `permute2` 的执行时序：调用 `permute2(seq)` 只是创建生成器；第一次 `next()` 才真正进入函数体，跑到第一个 `yield`（最浅层递归的第一个结果）立即返回。每一层 `for x in permute2(rest)` 都在“现场”从更深层的生成器里逐个 `next()` 取数——所以整个递归栈其实是一串互相驱动的生成器，内存中同时只存在一条“结果链”。

- `f'{math.factorial(10);}'` 是 f-string 格式化，让整数按千位分组显示为 3,628,800。

- 性能对比的机理：`permute1(seq)` 建 3628800 个列表 → 17 秒 + 巨大内存；`permute2` 首秒内就能吐出第一个排列。50 项时 $50! \approx 3.04 \times 10^4$ ，任何列表方案物理上不可能，而生成器随时可取第一项。这就是“让不可能成为可能”。

- `random.shuffle(seq)` 就地打乱列表，`permute2` 的输出顺序随输入顺序变化——生成器使“取一个排列看效果”变得廉价，而 `permute1` 在这个规模下连第一步都迈不出去。

·设计思想：作者在结论处保持辩证——生成器不是银弹：维护成本、隐式性都是代价；显式类版本（第六部分）是另一条路。工程判断 = 权衡，不是教条。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.18 Example: Emulating zip and map （ 示例：模拟 zip 和 map）

英文原文

Let's take a quick look at one more example of generators in action that illustrates just how expressive they can be. Once you know about comprehensions, generators, and other iteration tools, you'll find that emulating many of Python's functional built-ins is both st

straightforward and instructive, and can be useful for custom roles. For example, we've already seen how the built-in `zip` and `map` functions combine iterables and project functions across them, respectively. With multiple iterable arguments, `map` projects the function across items taken from each iterable in much the same way that `zip` combines them, and both functions truncate at the shortest iterable's length:

```
> >> list(zip('abc', 'xyz123')) # zip combines items from itera...

[('a', 'x'), ('b', 'y'), ('c', 'z')]
> >> list(zip([1, 2, 3], [2, 3, 4, 5])) # N M-ary iterables: M N-a...

[(1, 2), (2, 3), (3, 4)]
> >> list(zip([-2, -1, 0, 1, 2])) # 1 5-ary iterable: 5 1-ary tuple...

[(-2,), (-1,), (0,), (1,), (2,)]
> >> list(map(abs, [-2, -1, 0, 1, 2])) # Single iterable: 1-ary fu...

[2, 1, 0, 1, 2]
> >> list(map(pow, [1, 2, 3], [2, 3, 4, 5])) # N iterables: N-ary ...

[1, 8, 81]
```

As covered earlier, both also work on any type of iterable, including files that read their lines automatically. Though they're ultimately used for different purposes, if you study these examples long enough, you might notice a relationship between `zip` results and `map` function arguments that our next example can exploit.

Coding Your Own map (编写你自己的 map)

Although the map and zip built-ins are fast and convenient, they're easy to implement in customizable code of our own. In the preceding chapter, for example, we wrote a function that emulated the map built-in for a single iterable argument. Per Example 20-3, it doesn't take much more work to allow for multiple iterables, as the built-in does.

Example 20-3. mymap-lists.py

```
"Emulate map: support multiple arguments, build a list result"
def mymap(func, *seqs):
    res = []
    for args in zip(*seqs):
        res.append(func(*args))
    return res
print(mymap(abs, [-2, -1, 0, 1, 2]))
print(mymap(pow, [1, 2, 3], [2, 3, 4, 5]))
```

This version relies upon argument syntax—it collects multiple sequence (really, iterable) arguments; unpacks them as zip arguments to combine; and then unpacks the combined zip results as arguments to the passed-in function. That is, we're using the fact that the zipping is essentially a nested operation in mapping. The test code at the bottom applies this to both one and two sequences to test—with the same results generated by the built-in map:

```
$ python3 mymap-list.py
[2, 1, 0, 1, 2]
[1, 8, 81]
```

Really, though, the preceding code exhibits the classic list comprehension pattern, building a list of operation results within a for loop. We can rewrite our mapper more concisely as an equivalent one-line list comprehension:

```
def mymap(func, *seqs):
    return [func(*args) for args in zip(*seqs)]
```

When this is run the result is the same as before, but the code is more concise and might run faster (more on performance in Chapter 21). Both of the preceding mymap versions build result lists all at once, though, and this can waste memory for larger lists. Now that we know about generator functions and expressions, it's simple to recode both these alternatives to produce results on demand instead, per Example 20-4.

Example 20-4. mymap-generate.py

```
"Emulate map: support multiple arguments, generate results on r...
def mymap_func(func, *seqs):
    for args in zip(*seqs):
        yield func(*args)
def mymap_expr(func, *seqs):
    return (func(*args) for args in zip(*seqs))
for mymap in (mymap_func, mymap_expr):
    print(list(mymap(abs, [-2, -1, 0, 1, 2])))
    print(list(mymap(pow, [1, 2, 3], [2, 3, 4, 5])))
```

Both these new versions produce the same results bu

t return generators designed to support the iteration protocol—the first yields one result at a time explicitly, and the second returns a generator expression's result to do the same implicitly. As generators, we must wrap them in list calls to force them to produce their values all at once:

```
$ python3 mymap-generate.py
[2, 1, 0, 1, 2]
[1, 8, 81]
[2, 1, 0, 1, 2]
[1, 8, 81]
```

No real work is done here until the list calls force the generators to run by activating the iteration protocol. The list calls allow for display in testing, but this is not generally desirable in other contexts. The generators returned by these functions themselves, as well as the generators returned by the `zip` built-in they use, produce results only on demand, and that's the whole point of generators—delaying results saves memory space, and avoids pausing callers for full results.

Coding Your Own `zip` and 2.X `map` (编写你自己的 `zip` 和 2.X 版 `map`)

Of course, much of the secret behind the success of the examples shown so far lies in their use of the `zip` built-in to combine arguments from multiple iterables. Using iteration tools, we can also code workalikes that emulate both today's truncating `zip`, as well as the former padding behavior of Python 2.X's `map` when p

assed a None for its function—a still potentially useful tool despite its demise in 3.X, and the sort of thing that custom code can provide. Per Example 20-5, zip and this padding map are related in utility and nearly the same in code.

Example 20-5. myfptools-list.py

```
# Emulate zip and padding map: build lists
def myzip(*seqs):
    seqs = [list(S) for S in seqs]
    res = []
    while all(seqs):
        res.append(tuple(S.pop(0) for S in seqs))
    return res

def mymapPad(*seqs, pad=None):
    seqs = [list(S) for S in seqs]
    res = []
    while any(seqs):
        res.append(tuple((S.pop(0) if S else pad) for S in seqs))
    return res

S1, S2 = 'abc', 'xyz123'
print(myzip(S1, S2))
print(mymapPad(S1, S2))
print(mymapPad(S1, S2, pad=99))
```

Both of these functions work on any type of iterable object because they run their arguments through the list built-in to force result generation (e.g., files would work as arguments, in addition to sequences like strings). Notice the use of the all and any built-ins here—as we saw briefly in Chapter 14, these return True if all or any items in an iterable are True (or equivalently, nonempty), respectively. These built-ins are used t

o stop looping when any or all of the listified arguments become empty after deletions. Also note the use of the keyword-only argument, `pad`; unlike the 2.X `map`, our version will allow any `pad` object to be specified. When these functions are run, the following results are printed—a zip and two padding maps:

```
$ python3 myfptools-list.py
[('a', 'x'), ('b', 'y'), ('c', 'z')]
[('a', 'x'), ('b', 'y'), ('c', 'z'), (None, '1'), (None, '2'), (None, '3')]
[('a', 'x'), ('b', 'y'), ('c', 'z'), (99, '1'), (99, '2'), (99, '3...)]
```

These functions aren't amenable to list comprehension translation because their loops are too specific. As before, though, while our `zip` and `map` workalikes currently build and return result lists, it's just as easy to turn them into generators with `yield` so that they each return one piece of their result set at a time. Example 20-6 shows how.

Example 20-6. `myfptools_generate.py`

```
# Emulate zip and padding map: generate results
def myzip(*seqs):
    seqs = [list(S) for S in seqs]
    while all(seqs):
        yield tuple(S.pop(0) for S in seqs)
def mymapPad(*seqs, pad=None):
    seqs = [list(S) for S in seqs]
    while any(seqs):
        yield tuple((S.pop(0) if S else pad) for S in seqs)
```

The results are the same as before, but this file assumes it will be used or tested elsewhere (it has no self-test code), and we need to use `list` again to force the

generators to yield their values for display in the REPL:

```
$ python3
> >> from myfptools_generate import myzip, mymapPad >> list(myzip(...

[('a', 'x'), ('b', 'y'), ('c', 'z')])
> >> list(mymapPad('abc', 'xyz123', pad=99))

[('a', 'x'), ('b', 'y'), ('c', 'z'), (99, '1'), (99, '2'), (99, '3...
```

Finally, here's an alternative implementation of our zip and map emulators—rather than deleting arguments from lists with the pop method, the following versions do their job by calculating the minimum and maximum argument lengths. Armed with these lengths, it's easy to code nested list comprehensions to step through argument index ranges:

```
def myzip(*seqs):
    minlen = min(len(S) for S in seqs)
    return [tuple(S[i] for S in seqs) for i in range(minlen)]
def mymapPad(*seqs, pad=None):
    maxlen = max(len(S) for S in seqs)
    index = range(maxlen)
    return [tuple((S[i] if len(S) > i else pad) for S in seqs) for...
```

Because these use len and indexing, they assume that arguments are sequences or similar, not arbitrary iterables, much like our earlier sequence scramblers and permuters. The outer comprehensions here step through argument index ranges, and the inner comprehensions (passed to tuple) step through the passed-in sequences to pull out arguments in parallel (though

not at the same time: see the next section!). When they're run, the results are as before. Most strikingly, generators and iterators seem to run rampant in this example. The arguments passed to `min` and `max` are generator expressions, which run to completion before the nested comprehensions begin iterating. Moreover, the nested list comprehensions employ two levels of delayed evaluation—the `range` built-in is an iterable, as is the generator expression argument to `tuple`. In fact, no results are produced here until the square brackets of the list comprehensions request values to place in the result list—which forces all the comprehensions and generators to run. To turn these functions themselves into generators instead of list builders, simply use parentheses instead of square brackets again. Here's the case for our `zip`:

```
def myzip(*seqs):
    minlen = min(len(S) for S in seqs)
    return (tuple(S[i] for S in seqs) for i in range(minlen))
S1, S2 = 'abc', 'xyz123'
print(list(myzip(S1, S2))) # Go!... [('a', 'x'), ('b', 'y'), ('c', ...
```

In this case, it takes a list call to activate the generators and other iterables to produce their results. Experiment with these on your own for more details. Developing further coding alternatives is left as a suggested exercise (but see the sidebar "Why You Will Care: Iteration Versus Python Morph" for an exploration of one such option). Here, we have time for just one more related tale from the generations saga.

中文翻译

让我们快速再看一个生成器实战示例，它说明生成器可以多么有表现力。一旦你了解了推导式、生成器和其他迭代工具，你会发现模拟许多 Python 函数式内置函数既直截了当又富有教益，还能用于自定义角色。例如，我们已经见识过内置 `zip` 和 `map` 函数如何分别组合可迭代对象、以及它们在它们之上投射函数。面对多个可迭代参数时，`map` 把函数投射到从每个可迭代对象取出的项上，方式和 `zip` 组合它们差不多，而且两个函数都在最短可迭代对象的长度处截断：

```
>>> list(zip('abc', 'xyz123'))           # zip
[('a', 'x'), ('b', 'y'), ('c', 'z')]
>>> list(zip([1, 2, 3], [2, 3, 4, 5]))    # N M M N ...
[(1, 2), (2, 3), (3, 4)]
>>> list(zip([-2, -1, 0, 1, 2]))          # 1 5 5 1 ...
[(-2,), (-1,), (0,), (1,), (2,)]
>>> list(map(abs, [-2, -1, 0, 1, 2]))      # 1
[2, 1, 0, 1, 2]
>>> list(map(pow, [1, 2, 3], [2, 3, 4, 5])) # N N
[1, 8, 81]
```

正如前面讲过的，两者也都能用于任何类型的可迭代对象，包括自动读取行的文件。虽然它们最终用途不同，但如果你把这些例子研究得够久，你可能会注意到 `zip` 的结果和 `map` 的函数参数之间存在一种关系——我们的下一个示例可以利用这一点。

编写你自己的 `map` (Coding Your Own `map`)

尽管 `map` 和 `zip` 内置函数又快又方便，但它们很容易用我们自己的可定制代码实现。例如，上一章我们写过一个模拟 `map` 内置函数的函数，只支持单个可迭代参数。按照示例

20-3，要像内置函数那样支持多个可迭代对象，也不需要多花多少功夫。

示例 20-3: mymap-lists.py

```
"Emulate map: support multiple arguments, build a list result"
```

```
def mymap(func, *seqs):
    res = []
    for args in zip(*seqs):
        res.append(func(*args))
    return res

print(mymap(abs, [-2, -1, 0, 1, 2]))
print(mymap(pow, [1, 2, 3], [2, 3, 4, 5]))
```

这个版本依赖参数语法——它收集多个序列（确切说是可迭代）参数；把收集的参数解包成 `zip` 的参数来组合；再把组合后的 `zip` 结果解包成传给传入函数的参数。也就是说，我们利用了这一事实：在映射中，`zip` 本质上是一个嵌套操作。底部的测试代码把它同时应用到一两个序列上进行测试——生成的结果与内置 `map` 相同：

```
$ python3 mymap-list.py
[2, 1, 0, 1, 2]
[1, 8, 81]
```

不过说真的，前面这段代码呈现出经典的列表推导式模式——在一个 `for` 循环中构建操作结果的列表。我们可以把映射器改写成等价的单行列表推导式，更简洁：

```
def mymap(func, *seqs):
    return [func(*args) for args in zip(*seqs)]
```

运行这个版本时结果和之前一样，但代码更简洁，而且可能跑得更快（性能详情见第 21 章）。不过，上面两个 `myma`

p 版本都是一次性构建结果列表，对较大的列表这可能浪费内存。既然我们现在知道了生成器函数和表达式，要照示例 20-4 把这两个版本改成按需产生结果，就很简单了。

示例 20-4: mymap-generate.py

```
"Emulate map: support multiple arguments, generate results on r...
def mymap_func(func, *seqs):
    for args in zip(*seqs):
        yield func(*args)

def mymap_expr(func, *seqs):
    return (func(*args) for args in zip(*seqs))

for mymap in (mymap_func, mymap_expr):
    print(list(mymap(abs, [-2, -1, 0, 1, 2])))
    print(list(mymap(pow, [1, 2, 3], [2, 3, 4, 5])))
```

这两个新版本产生相同结果，但返回的是为支持迭代协议而设计的生成器——第一个显式地一次 yield 一个结果，第二个返回生成器表达式的结果来隐式完成同样的事。作为生成器，我们必须用 list 调用包住它们，才能强迫它们一次性产出所有值：

```
$ python3 mymap-generate.py
[2, 1, 0, 1, 2]
[1, 8, 81]
[2, 1, 0, 1, 2]
[1, 8, 81]
```

在 list 调用通过激活迭代协议强迫生成器运行之前，这里没有任何实际工作发生。list 调用是为了在测试中显示结果，但在其他语境中一般不这么做。这些函数返回的生成器，以及它们所用的 zip 内置函数返回的生成器，都只按需产出结果——而这正是生成器的全部意义：延迟结果以节省内存

空间，并避免为完整结果而让调用方停顿。

编写你自己的 zip 和 2.X 版 map (Coding Your Own zip and 2.X map)

当然，迄今所示示例成功背后的很多秘密，在于它们使用 zip 内置函数来组合来自多个可迭代对象的参数。利用迭代工具，我们还可以编写工作等价的 (workalike) 代码，同时模拟今天的截断式 zip，以及 Python 2.X 的 map 传入 None 作为函数时的旧式填充 (padding) 行为——尽管它在 3.X 中已消亡，这个工具仍然潜在有用，正是自定义代码能提供的那种东西。按照示例 20-5，zip 和这个填充 map 在用途上相关，在代码上几乎相同。

示例 20-5: myfptools-list.py

```
# zip map

def myzip(*seqs):
    seqs = [list(S) for S in seqs]
    res = []
    while all(seqs):
        res.append(tuple(S.pop(0) for S in seqs))
    return res

def mymapPad(*seqs, pad=None):
    seqs = [list(S) for S in seqs]
    res = []
    while any(seqs):
        res.append(tuple((S.pop(0) if S else pad) for S in seqs))
    return res

S1, S2 = 'abc', 'xyz123'
print(myzip(S1, S2))
print(mymapPad(S1, S2))
```

```
print(mymapPad(S1, S2, pad=99))
```

这两个函数都能处理任何类型的可迭代对象，因为它们把参数送过 `list` 内置函数来强迫结果生成（例如，除了字符串这类序列，文件也能作为参数）。注意这里 `all` 和 `any` 内置函数的用法——正如我们在第 14 章简单见过的，如果可迭代对象中的全部项（`all`）或任一（`any`）项为 `True`（等价地说，非空），它们分别返回 `True`。这两个内置函数用来在删除之后、当列表化参数中的任何一个或全部变空时停止循环。还要注意仅关键字（`keyword-only`）参数 `pad` 的使用；与 2.X 的 `map` 不同，我们的版本允许指定任意填充对象。运行这些函数时，会打印以下结果——一个 `zip` 和两个填充 `map`：

```
$ python3 myfptools-list.py
[('a', 'x'), ('b', 'y'), ('c', 'z')]
[('a', 'x'), ('b', 'y'), ('c', 'z'), (None, '1'), (None, '2'), (No...
[('a', 'x'), ('b', 'y'), ('c', 'z'), (99, '1'), (99, '2'), (99, '3...
```

这些函数不适合翻译成列表推导式，因为它们的循环太特殊了。不过和之前一样，虽然我们的 `zip` 和 `map` 工作等价物目前是构建并返回结果列表，用 `yield` 把它们变成生成器同样容易——这样它们每次只返回结果集的一块。示例 20-6 展示了做法。

示例 20-6: myfptools_generate.py

```
# zip map
def myzip(*seqs):
    seqs = [list(S) for S in seqs]
    while all(seqs):
        yield tuple(S.pop(0) for S in seqs)

def mymapPad(*seqs, pad=None):
    seqs = [list(S) for S in seqs]
    while any(seqs):
        yield tuple((S.pop(0) if S else pad) for S in seqs)
```

结果和之前相同，但这个文件假定它会在别处被使用或测试（它没有自测代码），我们还需要再次用 `list` 来强迫生成器产出它们的值，以便在 REPL 中显示：

```
$ python3
>>> from myfptools_generate import myzip, mymapPad
>>> list(myzip('abc', 'xyz123'))
[('a', 'x'), ('b', 'y'), ('c', 'z')]
>>> list(mymapPad('abc', 'xyz123', pad=99))
[('a', 'x'), ('b', 'y'), ('c', 'z'), (99, '1'), (99, '2'), (99, '3...
```

最后，这里还有我们 `zip` 和 `map` 模拟器的一个替代实现——不通过 `pop` 方法从列表删除参数，下面的版本通过计算参数的最小和最大长度来完成工作。有了这些长度，编写嵌套列表推导式来遍历参数索引区间就很容易了：

```
def myzip(*seqs):
    minlen = min(len(S) for S in seqs)
    return [tuple(S[i] for S in seqs) for i in range(minlen)]

def mymapPad(*seqs, pad=None):
    maxlen = max(len(S) for S in seqs)
    index = range(maxlen)
    return [tuple((S[i] if len(S) > i else pad) for S in seqs) for...
```

因为这些版本使用 `len` 和下标索引，它们假定参数是序列或

类似物，而不是任意可迭代对象——和我们早先的序列打乱器和排列器很相似。这里的外层推导式遍历参数索引区间，内层推导式（传给 tuple 的）遍历传入的序列以并行取出参数（虽然并非“同时”：见下一节！）。运行它们时，结果与之前一样。最引人注目的是，生成器和迭代器在这个示例中似乎泛滥成灾。传给 min 和 max 的参数是生成器表达式，它们在嵌套推导式开始迭代之前就运行完毕了。此外，嵌套的列表推导式用了两层延迟求值——range 内置函数是一个可迭代对象，传给 tuple 的生成器表达式也是。事实上，直到列表推导式的方括号向结果列表索要放入的值之前，这里不会产生任何结果——这强迫所有推导式和生成器运行。要把这些函数自身从列表构建器变成生成器，只需再次把方括号换成圆括号。下面是我们 zip 的例子：

```
def myzip(*seqs):
    minlen = min(len(S) for S in seqs)
    return (tuple(S[i] for S in seqs) for i in range(minlen))

S1, S2 = 'abc', 'xyz123'
print(list(myzip(S1, S2))) # ... [('a', 'x'), ('b', 'y'), ('c...
```

这种情况下，需要一个 list 调用才能激活生成器和其他可迭代对象来产出结果。更多细节请自行实验。开发进一步的编码替代方案留作建议性练习（不过关于其中一种方案的探索，参见边栏“Why You Will Care: Iteration Versus Python Morph”）。在这里，我们只剩下时间讲“生成记”（generations saga）里最后一个相关故事。

代码分析

· mymap 的 seqs 魔法：zip(seqs) 把收集到的多个可迭代对象解包成 zip 的多个参数——正好对应“多参数 map

= zip 后逐组调用函数"这一洞察；func(args) 再把 zip 产生的一元组解包成函数的多个实参。整个 mymap 就是"转置 + 应用"。

·执行轨迹 (mymapfunc) : list(mymapfunc(pow, [1,2,3], [2,3,4,5])) → list 驱动生成器 → 生成器驱动 zip → zip 从两个列表各取一项得到 (1, 2) → pow(1, 2) = 1 → yield 给 list。三层惰性嵌套，每轮只碰一个中间元组。

· while all(seqs): all 对三个"列表化"后的序列求值——空列表为假，任一耗尽即停止，实现"最短截断"；mymapPad 用 any，任一非空就继续，空序列用 pad 填充 (S.pop(0) if S else pad)，实现"最长填充"。这正是旧版 2.X map 的 padding 语义。pop(0) 从列表头部弹出，复杂度 O(n)，每次弹头会移动后续元素——对小输入无所谓，这里仅为教学。

· min(len(S) for S in seqs) 是生成器表达式做聚合：min 内部对它逐个 next，跑完即弃；tuple(S[i] for S in seqs) 里的内层生成器在每次外层迭代时现场取所有序列的第 i 项——两层延迟求值被"方括号"一次性点燃。

·列表版与生成器版 myzip 的唯一差别：return [...] 变 return (...)。返回的生成器把"计算"延后到消费时。

·设计思想：亲手重造内置函数是最深刻的学习方式——你会在重造过程中被迫理解迭代协议、参数解包、惰性求值这些基础；而这些工作等价物还可以自由定制（比如任意 pad 值），这正是"用自定义代码服务特定角色"的价值。

深度理解

- 核心概念：把握本节在整章知识链中的位置，弄清它引入或巩固的关键术语与机制。
- 底层实现：对照 Python 运行时/迭代协议/对象模型，理解代码实际如何被解释器处理。
- 设计原因：Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题：可在真实项目中用本节技术解决哪些任务；何时该用、何时不该用。
- 初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.19 Asynchronous Functions: The Short Story (异步函数：短篇故事)

英文原文

Now that you've survived all the foregoing twists and turns of the functions story in Python, there's one last topic to go. Python 3.5 debuted an extension called asynchronous (usually shortened to `async`) functions, which are able to manually pause their execution while waiting for a result, in order to allow other such functions to run. Such tools are available in other languages (e.g., JavaScript), which provided some of the inspiration and blueprint for their appearance in Python.

n. Indeed, match, :=, f-strings, type hinting, and a host of other Python tools owe their presence to this same programming-languages arms race, whose logical outcome would make all languages the same. Origins aside, the pathological use case for the async extension is input/output (IO) operations: a function can pause itself until an IO transfer completes so that other parts of the program can run during the wait. While other tools like multiple threads can and still do address this need too, some may judge async functions to be a lighter-weight option. That said, there are downsides to this topic. For one thing, it has been morphing almost constantly since its first murmurs in Python 3.4, which makes it difficult to document in books with lifespans much longer than web pages and blogs. For another, it's something of an all-or-nothing proposition, because async code requires other async code. More fundamentally, async functions are part of the applications-level domain of parallel programming—a category that also includes multiprocessing and multithreading. As such, they are a heavyweight subject of interest to only a subset of Python's user base, and a lot to ask of newcomers to Python or programming in general. Especially given Python's already skyrocketing complexity, this may be best taken as an optional add-on topic, and out of scope here. Nevertheless, async functions are not truly optional in Python: they were added deeply to the language as new syn

tax, via the `async` and `await` reserved words, and are now fair game in code you may have to reuse (and job-interview tests you may have to suffer). For all these reasons, this section provides just enough of an overview to whet your appetite, but defers to Python's online resources for more details when you're ready to tackle this advanced topic.

Async Basics (异步基础)

In a nutshell, `async` functions enable a cooperative, nonpreemptive multitasking model, with coding trade-offs similar to those imposed by asynchronous network servers. For instance, tasks must generally be short-lived to avoid monopolizing the host device's processor. Though begun earlier, Python 3.5 made this part of the language with the `async def` statement and `await` expression, as well as the `async for` iterator and `async with` context manager. `Async` tools and syntax can be used only in functions defined with `async def`. Like the `yield` and `yield from` that this model extend, `async def` causes a function to be compiled specially: when called, instead of running its body, such a function returns an awaitable object that supports an expected method-based protocol. To invoke this object, functions `await` results explicitly, or use an `async for` or `async with` that does so implicitly. Moreover, an event loop must be launched to run the show at large. Although the event loop runs just one task at a time (th

ey're not truly parallel), it can switch to other tasks when a task runs and await for a pending result.

Running serial tasks with normal blocking calls (用普通阻塞调用运行串行任务)

As usual, this may be easier to grok in code than narrative, so let's turn to some examples to demo the salient bits. All of this section's example snippets live in the file `allasyncdemos.py` in the `examples` package, previewed in Example 20-7. To work along, either cut and paste from this file, or run the examples file in its entirety or with parts stubbed out with triple quotes. All examples here assume that the preamble at the top of Example 20-7 has been run.

Example 20-7. `all_async_demos.py` (preamble)

```
import time, asyncio
def now():
    return time.strftime('%H:%M:%S') # Local time, as hour:minute:second
...
all examples here
...
```

We need a handful of standard-library tools here: some in the `asyncio` standard-library module are required, and we'll display times with `time.strftime` and pause with `sleep` calls in both `time` and `asyncio` to simulate a long-running task. See Python's library manual for these tools' fine print that we'll skip here for space and scope. To get started, consider the following non

-async code, in which a main function calls another, producer, that simply pauses for two seconds using the `time.sleep` call before returning a result. The sleep stands in for a real blocking operation—like an IO call, network transfer, or user interaction. As usual with normal functions, main simply waits for each producer call to run to completion and return a result, before it proceeds with a next step:

```
def producer(label):
    time.sleep(2)                                # Pause for two seconds: ...
    return f'All done, {label}, {now()}'         # And return a result

def main():
    print('Start =>', now())
    print(producer(f'serial task 1'))            # Run three steps in seq...
    print(producer(f'serial task 2'))            # Waiting for each one t...
    print(producer(f'serial task 3'))            # Before doing anything ...
    print('Stop =>', now())

main()
```

When run, this code produces the following mundane results. As you can see, each producer call runs a two-second sleep before another starts, so the total time required is six seconds—two for each call run in series by main:

```
Start => [19:00:28]
All done, serial task 1, [19:00:30]
All done, serial task 2, [19:00:32]
All done, serial task 3, [19:00:34]
Stop => [19:00:34]
```

Running concurrent tasks with "await" and "async def" (用 await 与 async def 运行并发任务)

Now, let's do the same work with async functions. In the next listing, the producer is coded as an `async def` function, which makes Python compile it specially: it's now a coroutine—an object which, when called, doesn't produce its result, but instead returns an automatically created object that supports the coroutine method protocol. That protocol generally includes a method named `await`, which returns an iterator that produces coroutine results on `next`, but may instead use `anext` for generators. We can safely ignore all of these here. The `async def` syntax is always required for functions that use `await`, as well as `async for` and `async with` demoed ahead. Because `async def` is required to both use and define awaitable coroutines, it tends to spread viruslike throughout code (in fact, some tools now come with libraries in two separate forms, `asynchronous` and `not`). Also in the following, producer uses `await` to suspend itself until the sleep expires. `await` is similar to a `yield from`, but the event loop is free to pause the coroutine running the `await` and resume another if the awaited result is not ready. Notice that `await` is used as a statement in producer; it's also an expression that returns the awaited result, as in `main`, where it returns the result of the awaited objects:

```

async def producer(label):
    await asyncio.sleep(2)
    return f'All done, {label}, {now()}'

async def main():
    print('Start =>', now())
    task1 = asyncio.create_task(producer(f'async task 1'))
    task2 = asyncio.create_task(producer(f'async task 2'))
    task3 = asyncio.create_task(producer(f'async task 3'))
    print(await task1)
    print(await task2)
    print(await task3)
    print('Stop =>', now())

asyncio.run(main())

```

Crucially, the awaited `asyncio.sleep` call is like `time.sleep`, but is both nonblocking—it uses the event-loop's API to ensure that the caller doesn't pause for its completion—and awaitable—it uses `async def` to allow itself to be paused until a timeout elapses. Operations at the end of await chains must generally follow this model. Also crucially, `main` uses `asyncio.create_task` to wrap each producer object in a task—an object run by the event loop that suspends execution of the coroutine it wraps while the coroutine waits for completion of a future, and returns the result of its wrapped coroutine when awaited itself. Future objects signal progress in ways that are too low level for this overview; here, it's enough to know that each task suspends itself while its coroutine is sleeping. As a result, tasks can be overlapped in time, to run concurrently. In our code, all three tasks—and the producer coroutines the

y manage—finish at the same time, and the total program takes just two seconds instead of six:

```
Start => [19:00:34]
All done, async task 1, [19:00:36]
All done, async task 2, [19:00:36]
All done, async task 3, [19:00:36]
Stop => [19:00:36]
```

Naturally, we can use loops here to make the steps more automated. The following runs the same as the prior version—taking two total seconds, and two for each overlapped task:

```
async def producer(label):
    await asyncio.sleep(2)
    return f'All done, {label}, {now()}'

async def main():
    print('Start =>', now())
    tasks = []
    for i in range(3):
        tasks.append(asyncio.create_task(producer(f'async task {i+...
    for task in tasks:
        print(await task)
    print('Stop at', now())

asyncio.run(main())
```

In both `producer` and `main`, an `await`, much like a `yield from`, suspends execution of the coroutine that runs it until the awaited object returns its result. Together with the event loop and `tasks`, the net effect interleaves code.

How not to use async functions (怎样不要用 async 函数)

To understand the preceding example, it may help to see other codings that do not work. In this example, the `await`, `tasks`, and event loop are all essential: the code won't work without them. For one thing, calling `main` directly without routing it to the event loop fails right out of the gate with a warning. The call to `main` simply creates an awaitable object but does nothing with it (for brevity, snippets in this section show just differing code, followed by its results):

```
async def main():
    print(producer('xxx'))

main()
# Result...
.../all_async_blunders.py:12: RuntimeWarning: coroutine 'main' was...
```

For another, calling `producer` directly instead of `await`ing it fails too, and for the same reason. Much like generators, calling a coroutine returns the awaitable object, but doesn't run it; we must `await` the awaitable to make it go, just like we have to `run` next to get results from a generator:

```

async def main():
    print(producer('xxx'))

asyncio.run(main())
# Result...
<coroutine object producer at 0x10142e740>
.../all_async_blunders.py:22: RuntimeWarning: coroutine 'producer'...

```

More vitally, tasks are key to the concurrency here. If the example's main simply runs an await for a producer directly, it will pause for the sleep too. In the following miscoding, both coroutines await the sleep's end, and the two producers run serially—yielding a total of four seconds for the program at large instead of two:

```

async def main():
    print('Start =>', now())
    print(await producer('xxx'))
    print(await producer('yyy'))
    print('Stop =>', now())

asyncio.run(main())
# Result...
Start => [18:41:56]
All done, xxx, [18:41:58]
All done, yyy, [18:42:00]
Stop => [18:42:00]

```

This is also true if we defer the awaits by assignments—main continues to run after creating the producer coroutine awaitables, but still hangs for each later await that runs the coroutine's code, with no concurrency to be found:

```

async def main():
    print('Start =>', now())
    p1 = producer('xxx')
    p2 = producer('yyy')
    print('Await =>', now())
    print(await p1)
    print(await p2)
    print('Stop =>', now())

```

```

asyncio.run(main())

```

```

# Result...

```

```

Start => [18:42:00]

```

```

Await => [18:42:00]

```

```

All done, xxx, [18:42:02]

```

```

All done, yyy, [18:42:04]

```

```

Stop => [18:42:04]

```

That's why tasks are needed in the prior example: they enable coroutines to be paused and run by the event loop to overlap their execution. That is, they enable concurrency—along with alternatives like those of the next section.

Running concurrent tasks with "ascompleted" and "gather" (用 ascompleted 与 gather 运行并发任务)

But enough common mistakes: let's get back to coding concurrent coroutines the right way. Besides creating tasks, we can use the `gather` method to wait for all of our coroutines automatically, or `ascompleted` to wait for results as they finish. The latter can be used in a normal for to watch for completions:

```

async def producer(label):
    await asyncio.sleep(2)
    return f'All done, {label}, {now()}'

async def main():
    print('Start =>', now())
    coros = [producer(f'async task {i+1}') for i in range(3)]
    for nextdone in asyncio.as_completed(coros):
        print(await nextdone)
    print('Stop at', now())

asyncio.run(main())

```

This also finishes in two total seconds, though coroutines results may filter in in any order (as `as_completed` can also be used with `async for` of the next section in ways covered by Python's manuals):

```

Start => [19:00:38]
All done, async task 2, [19:00:40]
All done, async task 3, [19:00:40]
All done, async task 1, [19:00:40]
Stop at [19:00:40]

```

Alternatively, the `gather` call schedules tasks for coroutines, awaits all its arguments, and returns a list of task return values in the same order as its arguments when all have finished:

```

async def producer(label):
    await asyncio.sleep(2)
    return f'All done, {label}, {now()}'

async def main():
    print('Start =>', now())
    coro1 = producer(f'async task 1')
    coro2 = producer(f'async task 2')
    coro3 = producer(f'async task 3')
    results = await asyncio.gather(coro1, coro2, coro3)
    print(results)
    print('Stop at', now())

asyncio.run(main())

```

Results vary because they are in a list (formatted for display here), but it's still just two seconds for the entire gig:

```

Start => [19:00:40]
['All done, async task 1, [19:00:42]',
 'All done, async task 2, [19:00:42]',
 'All done, async task 3, [19:00:42]']
Stop at [19:00:42]

```

For any line counters in the audience, a starred list comprehension can be used to create awaitables for gather more succinctly, and produces the same two-second result as the preceding version:

```

async def producer(label):
    await asyncio.sleep(2)
    return f'All done, {label}, {now()}'

async def main():
    print('Start =>', now())
    print(await asyncio.gather(*[producer(f'async task {i+1}') for...
    print('Stop at', now())

asyncio.run(main())

```

While this code works, Python's manuals today include a note that recommends `TaskGroup` over `gather` on the grounds of better exception handling. Should you care to follow that advice, you'll need to move on to the next section.

Running concurrent tasks with "async with" and "async for" (用 `async with` 与 `async for` 运行并发任务)

As yet another concurrency option, the combination of the `async with` statement and a manager object can automatically handle both protocols and exceptions, and wait for tasks to finish. As half of this pair, `async with` internally awaits the awaitables returned by its subject's `aenter` and `aexit` methods, the same way that the normal `with` statement calls the context-manager protocol's `enter` and `exit`. Having said that, we unfortunately won't study either the normal `with` or its method protocols until Part VII, because they require knowledge of classes and OOP (yet again, Python's t

oolset assumes you must already know Python to use Python!), so you'll have to take this on faith for now. In the following code, though, it's enough to know that using a TaskGroup asynchronous context manager in concert with async with will automatically await results from all of its tasks before the statement exits, as well as handle some thorny issues involving exceptions:

```
async def producer(label):
    await asyncio.sleep(2)
    return f'All done, {label}, {now()}'

async def main():
    print('Start =>', now())
    async with asyncio.TaskGroup() as tg:
        tasks = [tg.create_task(producer(f'async task {i+1}')) for i in range(3)]
        for task in tasks:
            print(task.result())
    print('Stop at', now())

asyncio.run(main())
```

The net effect runs all three producer calls concurrently in two seconds as before:

```
Start => [19:00:44]
All done, async task 1, [19:00:46]
All done, async task 2, [19:00:46]
All done, async task 3, [19:00:46]
Stop at [19:00:46]
```

Finally, when a function is coded with `async def` and also uses `yield`, it's considered an asynchronous generator. When called, it returns an asynchronous iterable

e object which uses the `anext` method noted earlier, and can be used in an `async for` statement to execute the body of the function. (And no, there are no `async` variants of other statements like `while` or `if`; `with` and `for` were the only targets deemed worthy of the `async` twist—so far.) Internally, `async for` uses methods `aiter` and `anext` much like the normal iteration protocol's `iter` and `next` that we studied in Chapter 14. The `anext` method returns an awaitable that produces a next value, and raises `StopAsyncIteration` to signal the end of values, instead of `StopIteration`. `async for` simply gets a next value by awaiting a call to `anext`, and running the loop's block of code for every value obtained this way. This encroaches on classes and OOP again, but statement users may ignore the details. To avoid confusion, keep in mind that `async for` awaits the next item in the iterable it scans—which is not the same as awaiting for something else in the loop body of a normal `for`, as in prior examples here. Moreover, `async for` does not automatically parallelize anything—it's really just like a normal `for`, but adds an `await` for each next item obtained from its iterable, instead of issuing a potentially blocking call. `async for`'s implicit `await`s between loops allow it to be interleaved with other ready-to-run tasks, but don't help much otherwise:

■

```

async def producer(label):
    for i in range(3):
        await asyncio.sleep(2)
        yield f'All done, {label} {i+1}, {now()}'

async def main():
    print('Start =>', now())
    async for reply in producer('async task'):
        print(reply)
    print('Stop at', now())

asyncio.run(main())

```

As coded here, the result is the same six-second showing of the original serial version, because there's nothing else to run in this simple demo during the sleeps:

```

Start => [19:00:46]
All done, async task 1, [19:00:48]
All done, async task 2, [19:00:50]
All done, async task 3, [19:00:52]
Stop at [19:00:52]

```

And lest this section hasn't yet managed to send you screaming into the night, it's also possible to use `await` and `async for` within a comprehension that's coded in an `async def`—but this must remain a story for another day:

```

async def hmm():
    result = [i async for i in asynciter() if i > 0]    # This code...
    result = [await func() for func in coroutines]      # It's also...

```

The Async Wrap-Up (异步收尾)

That's all the time and space we have for this topic (humanely, perhaps). Python's async functions may be more useful than type hinting (which, as we saw in Chapter 6, is both completely unused and at odds with Python's core ideas), but a full survey of either topic could fill chapters or books, and would be of interest to only a subset of Python users. Moreover, this text isn't covering Python's multithreading or multiprocessing tools at all, even though they are at least as valid as async functions for parallelizing tasks. The fact that async alone was afforded dedicated syntax in the language both subjectively favors just one option in this domain and raises the bar for Python newcomers. That said, your goals and views may differ, so please see Python's standard manuals for the full story if and when you wish to wade deeper into the async sea. Here, we must step back from the ledge of optional and stunningly convoluted extensions, and get back to the fundamentals that nearly all Python programs and programmers use.

中文翻译

既然你已经在 Python 的函数故事里熬过了前面所有的曲折反转，这里还剩最后一个主题。Python 3.5 首次引入了叫异步 (asynchronous, 通常简称为 async) 函数的扩展，它们能在等待一个结果时手动暂停自己的执行，以便让其他

同类函数运行。这类工具在其他语言（比如 JavaScript）中早就存在，它们为 `async` 出现在 Python 中提供了部分灵感和蓝图。事实上，`match`、`:=`、`f-string`、类型提示（`type hinting`）以及一大堆其他 Python 工具的存在，都要归功于这场编程语言军备竞赛——其逻辑结局会让所有语言变得雷同。抛开起源不谈，`async` 扩展的“病态”用例（`pathological use case`）是输入/输出（IO）操作：一个函数可以暂停自己直到一次 IO 传输完成，以便程序的其他部分在等待期间运行。虽然多线程等其他工具也能满足这个需求，有些人可能会认为 `async` 函数是更轻量的选项。话虽如此，这个话题也有缺点。其一，自从它在 Python 3.4 初现端倪以来，它几乎一直在变，这让它很难写进寿命远比网页和博客长的书籍里。其二，它有点“要么全有、要么全无”的味道，因为 `async` 代码要求其他代码也是 `async` 的。更根本地说，`async` 函数属于并行编程（`parallel programming`）这一应用层领域——这个类别还包括多进程（`multiprocessing`）和多线程（`multithreading`）。因此，它是一个重头话题，只有一部分 Python 用户群体会感兴趣，对 Python 或编程的新手来说要求也太高。尤其考虑到 Python 已经急剧攀升的复杂度，它最好被当作可选的附加话题，这里不展开。尽管如此，`async` 函数在 Python 中并非真正可选：它们作为新语法、通过 `async` 和 `await` 保留字被深深植入语言，现在在你可能不得不复用的代码（以及你可能不得不遭受的求职面试题）里都是公平游戏。出于所有这些原因，本节只给出足以吊起你胃口的概述，细节就交给 Python 的在线资源——等你准备好攻克这个高级话题时再去看。

异步基础 (Async Basics)

简而言之，`async` 函数启用一种协作式 (cooperative)、非抢占式 (nonpreemptive) 的多任务模型，其编码权衡与异步网络服务器强加的那些权衡相似。例如，任务通常必须短命，以免独占宿主设备的处理器。虽然更早就有雏形，Python 3.5 用 `async def` 语句和 `await` 表达式，以及 `async for` 迭代器和 `async with` 上下文管理器，把它正式变成了语言的一部分。`Async` 工具和语法只能用在用 `async def` 定义的函数里。和它扩展自的 `yield`、`yield from` 一样，`async def` 会让函数被专门编译：调用时，这样的函数不运行函数体，而是返回一个支持预期“基于方法的协议”的可等待对象 (awaitable)。要调用这个对象，函数要么显式地 `await` 结果，要么使用隐式完成此事的 `async for` 或 `async with`。此外，还必须启动一个事件循环 (event loop) 来总体掌舵。虽然事件循环一次只运行一个任务（它们并非真正并行），但当某个任务为一个未就绪的结果执行 `await` 时，它可以在任务之间切换。

用普通阻塞调用运行串行任务 (Running serial tasks with normal blocking calls)

照例，用代码理解可能比用文字更容易，所以让我们看几个示例来演示要点。本节所有示例片段都存在于示例包的文件 `allasyncdemos.py` 中，示例 20-7 是预览。跟着做时，你可以从该文件剪切粘贴，或整体运行示例文件、或用三引号注释掉部分代码。这里所有示例都假定示例 20-7 顶部的序言 (preamble) 已经运行过。

示例 20-7: `all_async_demos.py` (序言)

```
import time, asyncio
def now():
    return time.strftime('%H:%M:%S') # ::
...
...
```

这里我们需要几个标准库工具：asyncio 标准库模块中的一些工具是必需的；我们还会用 time.strftime 显示时间，并用 time 和 asyncio 两边的 sleep 调用来模拟一个长时间运行的任务。这些工具的细枝末节请见 Python 库手册，出于篇幅和范围我们这里略过。作为开始，考虑下面这段非 async 代码：一个 main 函数调用另一个函数 producer，后者用 time.sleep 调用简单地暂停两秒，然后返回结果。这个 sleep 代表一个真实的阻塞操作——比如一次 IO 调用、网络传输或用户交互。和普通函数一样，main 只是等待每个 producer 调用运行完毕并返回结果，然后才进行下一步：

```
def producer(label):
    time.sleep(2) #
    return f'All done, {label}, {now()}' #

def main():
    print('Start =>', now())
    print(producer(f'serial task 1')) #
    print(producer(f'serial task 2')) #
    print(producer(f'serial task 3')) #
    print('Stop =>', now())

main()
```

运行时，这段代码产生下面这些平淡无奇的结果。如你所见，每个 producer 调用都在下一个开始之前运行两秒的 sleep，所以总耗时是六秒——main 串行运行的每个调用各占

两秒：

```
Start => [19:00:28]
All done, serial task 1, [19:00:30]
All done, serial task 2, [19:00:32]
All done, serial task 3, [19:00:34]
Stop => [19:00:34]
```

用 `await` 与 `async def` 运行并发任务 (Running concurrent tasks with "await" and "async def")

现在，我们用 `async` 函数做同样的工作。在下一个清单中，`producer` 被写成 `async def` 函数，这让 Python 专门编译它：它现在是一个协程 (coroutine) —— 一个被调用时不产生结果、而是返回一个支持协程方法协议的自动创建对象的对象。该协议一般包含一个叫 `await` 的方法，它返回一个在 `next` 时产出协程结果的迭代器；不过对生成器也可能改用 `anext`。这里我们都可以安全地忽略这些。使用 `await` 的函数必须写 `async def` 语法，后面演示的 `async for` 和 `async with` 也一样。因为 `async def` 对使用和定义可等待协程两者都是必需的，它往往会像病毒一样在代码中蔓延（事实上，现在有些工具会以两种分离的形式提供库：异步版和同步版）。此外在下面代码中，`producer` 用 `await` 把自己挂起，直到 `sleep` 到期。`await` 类似 `yield from`，但如果被等待的结果尚未就绪，事件循环可以自由地暂停运行该 `await` 的协程、恢复另一个协程。注意 `await` 在 `producer` 中是作为语句使用的；它也是一个返回被等待结果的表达式，就像 `main` 中那样——在那里它返回被等待对象的结果：

```

async def producer(label):
    # await async
    await asyncio.sleep(2)          # / sleep
    return f'All done, {label}, {now()}' # await

async def main():
    print('Start =>', now())
    task1 = asyncio.create_task(producer(f'async task 1'))
    task2 = asyncio.create_task(producer(f'async task 2'))
    task3 = asyncio.create_task(producer(f'async task 3'))
    print(await task1)              #
    print(await task2)
    print(await task3)
    print('Stop =>', now())

asyncio.run(main())                #

```

关键的是，被等待的 `asyncio.sleep` 调用像 `time.sleep`，但它既非阻塞——它用事件循环的 API 确保调用方不会为它的完成而暂停——又可等待——它用 `async def` 允许自己被暂停直到超时流逝。await 链末端的操作通常必须遵循这个模型。同样关键的是，main 用 `asyncio.create_task` 把每个 `producer` 对象包进一个任务（task）——这是由事件循环运行的对象，当它包裹的协程等待一个未来对象（future）完成时，它挂起该协程的执行，并且当自身被 `await` 时返回所包裹协程的结果。未来对象以对本概述来说过于底层的方式发出进展信号；这里只要知道“每个任务在自己的协程睡觉时挂起自己”就足够了。于是，任务可以在时间上重叠，并发（concurrently）运行。在我们的代码中，三个任务——以及它们管理的 `producer` 协程——同时完成，整个程序只花两秒而不是六秒：

```
Start => [19:00:34]
All done, async task 1, [19:00:36]
All done, async task 2, [19:00:36]
All done, async task 3, [19:00:36]
Stop => [19:00:36]
```

自然，我们可以用循环让步骤更自动化。下面这个与前一版本运行相同——总共两秒，每个重叠任务各两秒：

```
async def producer(label):
    await asyncio.sleep(2)
    return f'All done, {label}, {now()}'

async def main():
    print('Start =>', now())
    tasks = []
    for i in range(3):
        tasks.append(asyncio.create_task(producer(f'async task {i+...
    for task in tasks:
        print(await task)
    print('Stop at', now())

asyncio.run(main())
```

在 `producer` 和 `main` 中，`await` 都像 `yield from` 一样，挂起运行它的协程的执行，直到被等待对象返回其结果。结合事件循环和任务，最终效果是代码的交错（interleave）运行。

怎样不要用 `async` 函数（How not to use `async functions`）

要理解前面的示例，看看其他不能工作的写法也许有帮助。在这个示例中，`await`、任务和事件循环都是不可或缺的：没有它们代码就无法工作。其一，不把它路由到事件循环就直接调用 `main` 会立刻失败并发出警告。对 `main` 的调用只

是创建一个可等待对象，却对它什么都不做（为简洁起见，本节片段只展示不同的代码，后面跟它的结果）：

```
async def main():
    print(producer('xxx'))

main()
# ...
.../all_async_blunders.py:12: RuntimeWarning: coroutine 'main' was...
```

其二，直接调用 `producer` 而不是 `await` 它也会失败，原因相同。和生成器很像：调用一个协程返回可等待对象，但并不运行它；我们必须 `await` 这个可等待对象才能让它跑起来，就像我们必须运行 `next` 才能从生成器取得结果一样：

```
async def main():
    print(producer('xxx'))

asyncio.run(main())
# ...
<coroutine object producer at 0x10142e740>
.../all_async_blunders.py:22: RuntimeWarning: coroutine 'producer'...
```

更重要的是，任务在这里是并发的关键。如果示例的 `main` 直接对 `producer` 运行一个 `await`，它也会为 `sleep` 暂停。在下面这个错误写法中，两个协程都等待 `sleep` 结束，两个 `producer` 串行运行——整个程序总共四秒而不是两秒：

```
async def main():
    print('Start =>', now())
    print(await producer('xxx'))
    print(await producer('yyy'))
    print('Stop =>', now())

asyncio.run(main())
# ...
Start => [18:41:56]
All done, xxx, [18:41:58]
All done, yyy, [18:42:00]
Stop => [18:42:00]
```

如果我们用赋值推迟 `await` 也一样——`main` 在创建 `producer` 协程可等待对象后继续运行，但每次稍后运行协程代码的 `await` 时仍然会挂住，找不到任何并发：

```
async def main():
    print('Start =>', now())
    p1 = producer('xxx')
    p2 = producer('yyy')
    print('Await =>', now())
    print(await p1)
    print(await p2)
    print('Stop =>', now())

asyncio.run(main())
# ...
Start => [18:42:00]
Await => [18:42:00]
All done, xxx, [18:42:02]
All done, yyy, [18:42:04]
Stop => [18:42:04]
```

这就是为什么前面的示例需要任务：任务让协程能被事件循环暂停和运行，从而重叠它们的执行。也就是说，它们实现了并发——与下一节的替代方案并列。

用 as_completed 与 gather 运行并发任务 (Running concurrent tasks with "as_completed" and "gather")

好了，常见的错误说够了：让我们回到用正确方式编写并发协程。除了创建任务，我们还可以用 gather 方法自动等待所有协程，或用 as_completed 在结果完成时等待它们。后者可以在普通 for 中使用，以观察完成情况：

```
async def producer(label):
    await asyncio.sleep(2)
    return f'All done, {label}, {now()}'

async def main():
    print('Start =>', now())
    coros = [producer(f'async task {i+1}')] for i in range(3)]
    for nextdone in asyncio.as_completed(coros):
        print(await nextdone)
    print('Stop at', now())

asyncio.run(main())
```

这也总共两秒完成，不过协程结果可能以任意顺序陆续进来（as_completed 也可以与下一节的 async for 结合使用，方式见 Python 手册）：

```
Start => [19:00:38]
All done, async task 2, [19:00:40]
All done, async task 3, [19:00:40]
All done, async task 1, [19:00:40]
Stop at [19:00:40]
```

或者，gather 调用为协程调度任务，await 它所有的参数，并在全部完成时返回一个与参数顺序相同的任务返回值列表：

```

async def producer(label):
    await asyncio.sleep(2)
    return f'All done, {label}, {now()}'

async def main():
    print('Start =>', now())
    coro1 = producer(f'async task 1')
    coro2 = producer(f'async task 2')
    coro3 = producer(f'async task 3')
    results = await asyncio.gather(coro1, coro2, coro3)
    print(results)
    print('Stop at', now())

asyncio.run(main())

```

结果因位于列表中而有所不同（此处为显示做了格式化），但整个活儿仍然只需两秒：

```

Start => [19:00:40]
['All done, async task 1, [19:00:42]',
 'All done, async task 2, [19:00:42]',
 'All done, async task 3, [19:00:42]']
Stop at [19:00:42]

```

对于在场的任何行数统计者：可以用带星号的列表推导式更简洁地为 `gather` 创建可等待对象，产生与前一版本相同的两秒结果：

```

async def producer(label):
    await asyncio.sleep(2)
    return f'All done, {label}, {now()}'

async def main():
    print('Start =>', now())
    print(await asyncio.gather(*[producer(f'async task {i+1}') for i in range(3)]))
    print('Stop at', now())

asyncio.run(main())

```

虽然这段代码能工作，但今天的 Python 手册里有一条注释，以更好的异常处理为由推荐 TaskGroup 而不是 gather。如果你想遵循那条建议，就需要移步到下一节了。

用 `async with` 与 `async for` 运行并发任务 (Running concurrent tasks with "async with" and "async for")

作为又一种并发选项，`async with` 语句与一个管理器对象的组合可以自动处理协议和异常，并等待任务完成。作为这对组合的一半，`async with` 内部 await 其主体的 `aenter` 和 `aexit` 方法返回的可等待对象，就像普通 `with` 语句调用上下文管理器协议的 `enter` 和 `exit` 一样。话虽如此，遗憾的是我们要到第七部分才会研究普通的 `with` 或其方法协议，因为它们需要类和 OOP 的知识（又是这样：Python 的工具集假定你必须先懂 Python 才能用 Python!），所以你现在只能先相信这一点。不过在下面代码中，只需知道：把 TaskGroup 异步上下文管理器与 `async with` 搭配使用，会在语句退出前自动等待其所有任务的结果，并处理一些棘手的异常问题：

```
async def producer(label):
    await asyncio.sleep(2)
    return f'All done, {label}, {now()}'

async def main():
    print('Start =>', now())
    async with asyncio.TaskGroup() as tg:
        tasks = [tg.create_task(producer(f'async task {i+1}')) for i in range(10)]
        for task in tasks:
            print(task.result())
    print('Stop at', now())
```

```
asyncio.run(main())
```

最终效果和之前一样，在两秒内并发运行全部三个 producer 调用：

```
Start => [19:00:44]
All done, async task 1, [19:00:46]
All done, async task 2, [19:00:46]
All done, async task 3, [19:00:46]
Stop at [19:00:46]
```

最后，当一个函数用 `async def` 编写并且也使用 `yield` 时，它被认为是异步生成器（asynchronous generator）。调用时，它返回一个使用前面提到的 `anext` 方法的异步可迭代对象，可以在 `async for` 语句中用来执行该函数体。（而且，没有其他语句的 `async` 变体，比如 `while` 或 `if`；`with` 和 `for` 是仅有的两个被认为值得加 `async` 变体的目标——至少到目前为止。）在内部，`async for` 使用 `aiter` 和 `anext` 方法，很像我们第 14 章研究的正常迭代协议的 `iter` 和 `next`。`anext` 方法返回一个可等待对象，它产生下一个值，并通过抛出 `StopAsyncIteration`（而不是 `StopIteration`）来标志值的结束。`async for` 只是通过 `await` 一次 `anext` 调用来取得下一个值，并对每个这样获得的值运行循环代码块。这再次侵入了类和 OOP 的领地，但使用语句的人可以忽略这些细节。为避免混淆，请记住：`async for` `await` 的是它所扫描可迭代对象中的下一个项——这不同于前面示例中在普通 `for` 循环体里 `await` 别的东西。而且，`async for` 不会自动并行化任何东西——它其实就像普通 `for`，只是为从可迭代对象取得的每个下一项加了一次 `await`，而不是发出一个可能阻塞的调用。`async for` 在循环之间的隐式 `await` 使它能与其它就绪任务交错，但除此之外帮助不大：

```

async def producer(label):
    for i in range(3):
        await asyncio.sleep(2)
        yield f'All done, {label} {i+1}, {now()}'

async def main():
    print('Start =>', now())
    async for reply in producer('async task'):
        print(reply)
    print('Stop at', now())

asyncio.run(main())

```

照现在这样编码，结果和最初串行版本一样是六秒的演出，因为在这个简单演示中，sleep 期间没有任何其他东西可运行：

```

Start => [19:00:46]
All done, async task 1, [19:00:48]
All done, async task 2, [19:00:50]
All done, async task 3, [19:00:52]
Stop at [19:00:52]

```

万一本节还没能让你尖叫着跑进夜色里——在 async def 编写的推导式内部使用 await 和 async for 也是可能的——但这只能留到改日再讲了：

```

async def hmm():
    result = [i async for i in asynciter() if i > 0]    #
    result = [await func() for func in coroutines]     # .....

```

异步收尾 (The Async Wrap-Up)

这个话题我们只有这些时间和篇幅了（也许算仁慈）。Python 的 async 函数也许比类型提示（type hinting）更有用（我们在第 6 章看到，类型提示既完全用不上，又违背 Python 的核心思想），但这两个话题的完整综述都能填满几

章甚至几本书，而且只会让一部分 Python 用户感兴趣。此外，本书完全没有涉及 Python 的多线程或多进程工具，尽管它们对任务并行化来说至少与 `async` 函数一样有效。语言只给 `async` 单独提供了专用语法，这一事实既主观地偏爱了这个领域中的某个选项，也提高了 Python 新手的门槛。话虽如此，你的目标和观点可能不同，所以如果你想在 `async` 之海里游得更深，请参阅 Python 的标准手册了解完整故事。在这里，我们必须从“可选但惊人复杂”的扩展悬崖边退回来，回到几乎所有 Python 程序和程序员都会用到的基础知识上。

代码分析

- `async def` 与生成器的同构性：`await` 与 `yield from` 的机制同源——都是“暂停当前执行，等待他人结果”。区别只在“等谁”：`yield from` 等子生成器，`await` 等一个 `future`，并把暂停期间的控制权交给事件循环去调度其他任务。
- 执行时序对比（串行 vs 并发）：串行版三个 `time.sleep(2)` 逐个阻塞，总计 6 秒；`async` 版 `createtask` 立刻创建三个任务，`asyncio.sleep` 是非阻塞等待，事件循环在三个协程间轮转：所有 `sleep` 并行计时，2 秒后三者同时完成——时间从 6 秒变 2 秒。
- “调用协程 ≠ 运行协程”：`main()` 只是创建协程对象（并触发 `RuntimeWarning`）；必须 `asyncio.run(main())` 把协程交给事件循环。这和“调用生成器函数不执行函数体”完全一致——都是惰性对象的编译产物。

- `asyncio.gather([producer(...) for i in range(3)])`: 星号解包让列表推导式产出的三个协程变成 `gather` 的三个位置参数; `gather` 自己调度并汇总为结果列表 (保持参数顺序)。
 - `async with asyncio.TaskGroup()`: 进入时创建任务组, 退出时 (`aexit`) 自动 `await` 所有任务、收集异常; `task.result()` 取各任务返回值。
 - 异步生成器: `async def + yield` → `async for` 通过 `anext` (可等待) 取下一项; 结束时抛 `StopAsyncIteration`。单任务演示仍是 6 秒——并发需要"有别的東西可切换", 没有其他任务时 `await` 就是白等。
 - 设计思想: 作者有意把 `async` 压缩成"短篇故事"——它是生成器思想的延伸, 但属于并行编程的重磅话题; 本书读者只需建立概念地图, 细节留到专题资料。
-

深度理解

- 核心概念: 把握本节在整章知识链中的位置, 弄清它引入或巩固的关键术语与机制。
- 底层实现: 对照 Python 运行时/迭代协议/对象模型, 理解代码实际如何被解释器处理。
- 设计原因: Python 为何提供该写法或 API——通常是为了表达力、惰性求值、可读性或性能权衡。
- 实际问题: 可在真实项目中用本节技术解决哪些任务; 何时该用、何时不该用。

·初学者误区：最常见的混淆点（与相似工具的边界、求值时机、可变/共享状态等）。

20.20 Chapter Summary (本章小结)

英文原文

This chapter wrapped up our coverage of built-in comprehension and iteration tools. It explored list comprehensions in the context of functional tools, and presented generator functions and expressions as additional iteration protocol tools. As a finale, we also explored some larger examples of iterations, comprehensions, and generations in action, and briefly glimpsed the asynchronous-functions extension to pique further study. Though we've now seen all the built-in iteration tools, the subject will resurface when we explore user-defined iterable classes in Chapter 30. The next chapter is something of a continuation of the theme of this one—it rounds out this part of the book with a case study that times the performance of the tools we've studied here, and serves as a more realistic example at the midpoint of this book (yes, you're half done!). Before we move ahead to benchmarking comprehensions and generators, though, this chapter's quizzes give you a chance to review what you've learned about them here.

中文翻译

本章收官了我们关于内置推导式与迭代工具的覆盖。它在函数式工具的语境中探讨了列表推导式，并把生成器函数和表达式作为额外的迭代协议工具呈现。作为压轴，我们还探讨了一些迭代、推导式和生成在实际应用中的更大示例，并匆匆瞥了一眼异步函数扩展，以激起进一步学习的兴趣。虽然我们现在已经见过所有内置迭代工具，但当我们第 30 章探索用户自定义可迭代类时，这个话题还会再次浮现。下一章多少是本章主题的延续——它用一个“给本章学过的工具计时”的案例研究为本书这部分收尾，也作为本书中点的更现实示例（没错，你已经读完一半了！）。不过在动身去给推导式和生成器做基准测试之前，本章的测验给你一个机会复习本章学到的知识。

20.21 Test Your Knowledge: Quiz (自测：测验)

英文原文

1. What is the difference between enclosing a list comprehension in square brackets and parentheses? 2. How are generators and iterators related? 3. How can you tell if a function is a generator function? 4. What does a yield statement do? 5. How are map calls and list comprehensions related? Compare and contrast the two. 6. What do `async def` and `await` mean in a Python script?

中文翻译

1. 把列表推导式用方括号括起来和用圆括号括起来有什么区别？2. 生成器 (generators) 和迭代器 (iterators) 有什么关系？3. 你如何判断一个函数是不是生成器函数？4. yield 语句做什么？5. map 调用和列表推导式有什么关系？比较并对比二者。6. 在 Python 脚本中，async def 和 await 是什么意思？

20.22 Test Your Knowledge: Answers (自测：答案)

英文原文

1. List comprehensions in square brackets produce the result list all at once in memory. When they are enclosed in parentheses instead, they are actually generator expressions—they have the same internal syntax and a similar meaning but do not produce the result list all at once. Instead, generator expressions return a generator object, which yields one item in the result at a time when used in an iteration tool or iterated manually with next. 2. Generators are iterable objects that support the iteration protocol automatically—they have an iterator with a next method (run by next) that repeatedly advances to the next item in a series of results and raises an exception at the end of the series. In Python, we can code generator functions with

def and yield, generator expressions with parentheses, and generator objects with classes that define a special method named `iter` (discussed later in the book).

3. A generator function has a `yield` statement (with or without its from extension) somewhere in its code. Generator functions are otherwise identical to normal functions syntactically, but they are compiled specially to return an iterable generator object when called. That object retains state and code location between values. (This means that deleting `yield` makes a function normal, but code deletions can cause all sorts of issues.)

4. When present, this statement makes Python compile the function specially as a generator; when called, the function returns a generator object that supports the iteration protocol. When the `yield` statement is run, it sends a result back to the caller and suspends the function's state; the function can then be resumed after the last `yield` statement, in response to a `next` built-in or `next` method call issued by the caller. In more advanced roles, the generator `send` method similarly resumes the generator, but can also pass a value that shows up as the `yield` expression's value. Generator functions may also have a `return` statement, which terminates the generator (and attaches an optional value to the automatic `StopIteration` exception).

5. The `map` call is similar to a list comprehension—both produce a series of values, by collecting the results of applying an operation to each i

tem in a sequence or other iterable, one item at a time. The primary difference is that `map` applies a function call to each item, and list comprehensions apply arbitrary expressions. Because of this, list comprehensions are more general; they can apply a function call expression like `map`, but `map` requires a function to apply other kinds of expressions. List comprehensions also support extended syntax—nested for loops, and `if` clauses that subsume the `filter` built-in. `map` also differs by producing a generator of values; the list comprehension materializes the result list in memory all at once, and this may matter for large lists.

6. The `async def` is used to define an asynchronous function (usually called a coroutine), and `await` waits for another asynchronous object to produce its value, possibly yielding control back to an event loop to allow other code to run during the wait. `async def` is required to use `await`, `async for`, and `async with`, and both an event loop and scheduling calls in module `asyncio` are generally required to achieve concurrency in this model. There's more to the `async` story than told here; see Python's manuals for next steps on this topic, as well as its multithreading and multiprocessing alternatives in Python's standard library.

中文翻译

1. 方括号列表推导式会一次性地在内存中生成结果列表。当它们改用圆括号括起来时，它们实际上是生成器表达式——

—内部语法相同、含义相似，但不会一次性生成结果列表。相反，生成器表达式返回一个生成器对象，当在迭代工具中使用、或用 `next` 手动迭代时，它一次只产出结果中的一项。

2. 生成器是自动支持迭代协议的可迭代对象——它们带有一个含 `next` 方法（由 `next` 驱动）的迭代器，该方法反复推进到结果序列中的下一项，并在序列末尾抛出一个异常。在 Python 中，我们可以用 `def` 和 `yield` 编写生成器函数，用圆括号推导式编写生成器表达式，用定义名为 `iter` 的特殊方法的类编写生成器对象（书后讨论）。

3. 生成器函数的代码某处有一个 `yield` 语句（带或不带它的 `from` 扩展）。除此之外，生成器函数在语法上与普通函数完全相同，但会被专门编译，以便在调用时返回一个可迭代的生成器对象。该对象在两次值之间保留状态和代码位置。（这意味着删掉 `yield` 会让函数变回普通函数，但删代码可能引发各种问题。）

4. 一旦出现，这个语句就让 Python 把函数专门编译成生成器；调用时，函数返回一个支持迭代协议的生成器对象。当 `yield` 语句运行时，它把一个结果送回调用方并挂起函数的状态；之后，响应调用方发出的 `next` 内置函数或 `next` 方法调用，函数可以在最后一个 `yield` 语句之后恢复。在更高级的角色中，生成器 `send` 方法类似地恢复生成器，但还能传入一个值，作为 `yield` 表达式的值出现。生成器函数也可以有 `return` 语句，它终止生成器（并把一个可选值挂到自动抛出的 `StopIteration` 异常上）。

5. `map` 调用与列表推导式相似——两者都产生一系列值：把某个操作应用到序列或其他可迭代对象中的每一项、一次收集一项的结果。主要区别在于 `map` 对每一项应用函数调用，而列表推导式应用任意表达式。正因如此，列表推导式更通用；它可以应用像 `map` 那样的函数调用表达式，而 `map` 要

应用其他种类的表达式则必须有函数。列表推导式还支持扩展语法——嵌套 for 循环，以及囊括了 filter 内置函数的 if 子句。map 还有一个不同点：它产生一个值的生成器；列表推导式则一次性地把结果列表物化（materialize）在内存中，对大型列表这可能很关键。6. async def 用来定义异步函数（通常称为协程），await 等待另一个异步对象产生它的值，可能会把控制权交回事件循环，以便在等待期间允许其他代码运行。async def 是使用 await、async for 和 async with 的前提，在这个模型中要实现并发，通常还需要 asyncio 模块里的事件循环和调度调用。async 的故事远不止这里讲的这些；更多后续内容请看 Python 手册，以及 Python 标准库中的多线程和多进程替代方案。

20.23 Why You Will Care: Iteration Versus Python Morph (你会关心的原因：迭代对阵 Python 的演化)

英文原文

In Chapter 14, we noted that some built-ins (like map) support only a single traversal and are empty after it occurs, and this book promised to show you an example of why that can be important in practice. Now that we've studied iteration topics in full, it can make good on this promise—and demo the negative impacts of Python changes on your code at the same time. In earlier editions of this book, the following clever coding for zip emulation, adapted from a version in Py

thon's docs at the time, worked because map returned a physical list instead of a generator of values:

```
def myzip(*args):                # Before 3.0...
    iters = map(iter, args)
    while iters:                 # Guarantee >=1, force looping
        res = [next(i) for i in iters] # Any empty? StopIteration...
        yield tuple(res)          # Else return res, suspend state
                                # Exit=return=StopIteration
```

```
>>> list(myzip('ab', 'lmn'))
```

```
KeyboardInterrupt
```

Except this broke in Python 3.0. This code relied on that fact that map's result supported multiple scans. When it mutated into a single-scan generator in 3.0, as soon as the list comprehension traversed iters once, the generator was exhausted but still True; later list comprehensions returned []; and this code would loop until killed by a Ctrl+C. To fix, the prior edition added a list around map to make it iterable:

```
def myzip(*args):                # Before 3.7...
    iters = list(map(iter, args)) # <= Make iters iterable, 3...
    while iters:                 # map() is a one-scan generator
        res = [next(i) for i in iters]
        yield tuple(res)
```

```
>>> list(myzip('ab', 'lmn'))
```

```
RuntimeError: generator raised StopIteration
```

Except this broke in Python 3.7. This code relied on the fact that an uncaught StopIteration from any iterable in iters was propagated through this function and sufficed to terminate its generation of values. Python 3.7 changed to suppress this "bubbling" propagation

of `StopIteration` through a generator and issue a `RuntimeError` in response. The workaround is to explicitly catch the `StopIteration` and return—even though `return` simply raises `StopIteration`:

```
def myzip(*args):                                # After 3.7, and before ?...
    iters = list(map(iter, args))
    while iters:
        try:                                     # <== Catch StopIteration, 3.7+
            res = [next(i) for i in iters]        # StopIteration won't ...
        except StopIteration:                     # How generators should exit
            return
        yield tuple(res)                          # But exit=return=StopIteration!

>>> list(myzip('ab', 'lmn'))
[('a', 'l'), ('b', 'm')]
```

You'll have to wait until Part VII for more on the `try` statement, and may have to similarly translate exceptions to returns this way in rare generators that rely on the former bubbling behavior. And while this final version works today, we're not placing any bets on the future. The takeaways here: wrapping `map` calls in `list` calls is not just for display; generators no longer propagate exit exceptions from code they run; and Python mods can and do break code—both subtly and repeatedly!

中文翻译

在第 14 章，我们提到一些内置函数（如 `map`）只支持单次遍历，遍历之后即告空竭，本书承诺过要给你看一个实例，说明这在实践中为何可能重要。既然我们现在已经完整研

究了迭代话题，正好可以兑现这个承诺——并同时演示 Python 的变更对你的代码的负面影响。在本书更早的版本中，下面这段聪明的 zip 模拟代码（改编自当时 Python 文档中的一个版本）能工作，是因为 map 返回的是一个物理列表，而不是值的生成器：

```
def myzip(*args):                # 3.0 .....
    iters = map(iter, args)
    while iters:                  # >=1
        res = [next(i) for i in iters] # StopIteration=
        yield tuple(res)            # res
                                    # ==StopIteration
```

```
>>> list(myzip('ab', 'lmn'))
KeyboardInterrupt
```

但它在 Python 3.0 中坏掉了。这段代码依赖这样一个事实：map 的结果支持多次扫描。当它在 3.0 中变异为单次扫描的生成器后，一旦列表推导式遍历 iters 一次，生成器就耗尽了、却仍然是 True；之后的列表推导式返回 []；这段代码会一直循环，直到被 Ctrl+C 干掉。为了修复，旧版本在 map 外面加了一个 list，让它可重新迭代：

```
def myzip(*args):                # 3.7 .....
    iters = list(map(iter, args)) # <= iters 3.0+
    while iters:                  # map()
        res = [next(i) for i in iters]
        yield tuple(res)
```

```
>>> list(myzip('ab', 'lmn'))
RuntimeError: generator raised StopIteration
```

但它在 Python 3.7 中又坏掉了。这段代码依赖这样一个事实：iters 中任何可迭代对象抛出的未捕获 StopIteration 会穿透这个函数传播出去，足以终止它的值生成。Python 3.7 改为抑制 StopIteration 穿过生成器的这种“冒泡”（bu

bbling) 传播, 并抛出 `RuntimeError` 作为回应。解决办法是显式捕获 `StopIteration` 然后 `return`——尽管 `return` 本身就会抛 `StopIteration`:

```
def myzip(*args):                                # 3.7 ?.....
    iters = list(map(iter, args))
    while iters:
        try:                                     # <== StopIteration3.7+
            res = [next(i) for i in iters]       # StopIteration
        except StopIteration:                   #
            return
        yield tuple(res)                        # ==StopIteration
```

```
>>> list(myzip('ab', 'lmn'))
[('a', 'l'), ('b', 'm')]
```

关于 `try` 语句的更多内容, 你得等到第七部分; 并且, 在少数依赖旧式冒泡行为的生成器里, 你可能也得用这种方式把异常翻译成 `return`。虽然这个最终版本今天能用, 但我们不对未来打任何赌。这里的要点是: 把 `map` 调用包进 `list` 调用不只是为了显示; 生成器不再传播它们所运行代码的退出异常; 而 Python 的改动确实会、而且确实在破坏代码——既隐蔽又反复!

深度理解

·核心概念: 这是全书最有价值的"版本变迁实录"之一——同一个函数因 Python 语义演化被打破两次, 每次都必须改写。它把本章的"单遍迭代"概念与真实的 Python 历史接上了轨。

·第一次打破 (3.0): `map` 从"返回列表"变成"返回单遍生成器"。`while iters:` 检查的是"映射结果是否非空", 但生成器对象永远为真——循环体里 `next(i)` 早已把生成器耗

尽，第二轮得到的 `res` 恒为 `[]`，于是无限循环空转。修复：`list(map(...))` 物化。

·第二次打破 (3.7)：PEP 479 规定，生成器内部未捕获的 `StopIteration` 不再向上传播以结束外层生成器，而是转成 `RuntimeError`。旧代码"让 `next(i)` 抛 `StopIteration` 自然退出 `myzip`"的技巧失效。修复：显式 `except StopIteration: return`。

·实际问题：这正是"迭代工具只能单遍"的现实后果——`os.walk`、`map`、`zip`、生成器都是单遍的，写可复用工具时若想多次扫描，必须显式物化 (`list`) 或重建。

·初学者误区：把"今天能用"当作"永远能用"。语言在演化 (3.0、3.7、3.11.....)，严谨的工程师要理解所依赖的语义、并对升级保持警觉。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

·大数据流式处理：逐行处理 GB 级日志时，用生成器表达式 (`(line for line in f if 'ERROR' in line)`) 替代列表推导式，内存占用从 $O(\text{文件大小})$ 降到 $O(1)$ 。

·无限序列与按需数据：ID 生成器、轮询器、测试参数生成器 (如本章 `squares` 模式) ——用 `itertools.count()`、`itertools.cycle()` 等无限迭代器配合 `islice` 截取。

- 流水线式处理：map/filter/生成器串联成"数据管道"，每个环节惰性，如 `sum(len(line) for line in open(f))`。
- Web/网络 IO：async def + await (asyncio/aiohttp) 并发处理网络请求；yield from 的进阶 await 模型正是生成器思想的延续。
- 自造迭代工具：仿照本章 mymap/myzip 写定制 zip 填充、跨数据库逐行搬运等专用迭代器。
- 排列与组合搜索：回溯算法（如八皇后、旅行商）用生成器版递归，第一时间返回部分解，避免全量组合爆炸。

与其他语言 (Java/C++) 的区别

特性	Python	Java	C++
惰性序列	生成器：yield / 生成器表达式，语言内建	Stream API (map/filter 惰性) 但无 yield	迭代器 + ranges; 协程需第三方 (C++20 有 coroutine 但复
状态保存	自动：帧冻结/恢复，局部变量自动保留	需显式类或 lambda 捕获	需手动 struct 或 lambda 状态
推导式	list/set/dict/gen 四种内建语法	无推导式，Stream 需链式调用	无; range-based for + 算法
迭代协议	鸭子类型：实现 iter/next 即可	必须实现 Iterator<T> 接口 (编译期强约束)	必须满足 std::iterator 概念 (模板)
无限序列	自然支持 (while True: yield)	Stream.iterate 有限支持	需自定义迭代器并小心边界
单遍性	生成器单遍、列表多遍，语义显式	Stream 单次消费	取决于迭代器实现

式

Python 发展历史背景

- 生成器源自 Python 2.2 (2001) 引入的 yield，灵感来自 Icon 语言的"生成器表达式"概念；Python 2.4 (2004

) 加入生成器表达式。

· 列表推导式更早, Python 2.0 (2000) 从 Haskell 借鉴;
2.4 增加 if 过滤; 2.7/3.0 增加集合、字典推导式。

· Python 2.5 加入 send/throw/close, 生成器升级为类协程; 2.6 增加 yield from 前的雏形; 3.3 正式引入 yield from。

· PEP 380 (yield from)、PEP 342 (send)、PEP 479 (StopIteration 不再冒泡, 3.7) ——本章边栏的两次"打破"正是 PEP 479 的直接后果。

· Python 3.5 的 PEP 492 引入 async def/await, 把"基于生成器的协程"提升为语言级语法; 3.11 的 asyncio.TaskGroup 标志着结构化并发的到来。

高级开发者需要掌握的相关知识

· 迭代协议的完整拼图: iter/next/StopIteration, 以及 getitem 回退机制 (第 14 章); 第 30 章讲类实现可迭代对象。

· itertools 工具箱: chain、tee、islice、zip_longest、product——生成器生态的瑞士军刀。

· 协程进阶: send/yield from 双向通信、asyncio 事件循环、TaskGroup 结构化并发、异步生成器 (async for)

。

· 性能剖析: 第 21 章的计时脚本 (timeit) 方法——推导式与生成器的真实速度差异要"测"而不是"猜"。

· 内存模型: 生成器帧 (frame) 的分配与释放、generator

r.close() 与 GeneratorExit、weakref 与生成器的终结。

学习建议 (Learning Advice)

·重要程度：★★★★★ (5/5) 。推导式与生成器是 Python 日常编码最高频的"高级"特性之一，标准库到处是生成器 (range、map、zip、os.walk、文件对象)，不理解它就无法真正读懂 Python 代码。

·应该掌握到什么程度：

·必会：四种推导式 (list/set/dict/gen) 的语法与场景选择；yield 生成器函数的读写；惰性 vs 立即求值的内存差异；单遍性陷阱。

·应知：send/throw/close、yield from、海象运算符 (:=) 在推导式中的作用域泄漏。

·了解即可：async 的详细编程（动手做一两个 asyncio 小项目后即可达到入门水平）。

·后续应该学习哪些相关内容：

1. 第 21 章：性能计时案例研究——亲手跑基准，验证"推导式快、生成器省内存"的量化结论。

2. 第 30 章：基于类的可迭代对象与生成器对象的对比——理解"显式类"与"隐式生成器"的权衡。

3. 第 22 章后：模块化打包（把 scramble/permute 这类工具写成可复用模块）。

4. Python 标准库 itertools 与 functools 文档，以及 asyncio 官方教程（进阶 async）。

5. 复习第 14 章（迭代协议）与第 17 章（作用域 LEGB），打通"生成器状态保存"与"作用域"两条线索。